QSC C# Wrapper – QSCCS

Technical Specification · QSCABI 0.10 · November 9, 2026

Quantum Resistant Cryptographic Solutions Corporation

This engineering specification provides an implementor-level description of the QSCCS managed cryptography and secure communications wrapper, its public C# contracts and native QSCABI bindings, build and deployment requirements, managed ownership rules, examples, security constraints and qualification procedures.

QSCCS is QRCS commercial source code governed by the QRCS Commercial Application Software License (QRCS-CASL) and any applicable signed commercial agreement. This document does not replace the controlling product license.

Contents

1 Foreword 3
2 Introduction and Supplied Source Baseline 3
3 References and Engineering Conventions 4
4 Architecture and .NET 8 Integration 6
5 Build and Runtime Deployment 6
6 QSCABI Native Loader and Contract Validation 7
7 Version, Capabilities and Compiled Backend 8
8 Managed Status, Errors and Result Types 9
9 Secret Buffers, SafeHandle and Resource Lifetime 9
10 Encoding, Entropy and Memory Utilities 10
11 SHA-2, SHA-3 and SHAKE 10
12 MAC and Key Derivation 10
13 Symmetric Ciphers and AEAD 11
14 Post-Quantum Key Encapsulation and Signatures 11
15 Classical Curves and Public-Key Operations 12
16 Explicit Algorithms and Selector Contracts 12
17 Streaming Digest, XOF, MAC and CTR 12
18 X.509 Certificates, Chains and Trust Stores 13
19 X.509 Issuance, CSR and CRL 13

20 TLS 1.3 Client and Server Lifecycle 14
21 Hardware Provider Callback Bridge 14
22 Example Implementation Patterns 15
23 Tests and Qualification Evidence 16
24 Security and Cross-Platform Constraints 17
25 Complete Managed C# API Reference 18
26 QSCABI Native LibraryImport Reference 102
27 Implementor Release Acceptance Checklist 117
Annex A Public Managed Types and Enums 118
Annex B Test Case and Source Inventory 121
Annex C Source Findings and Packaging Notes 125

1: Foreword

QSCCS is a .NET 8 C# wrapper over the external QSCABI 0.10 C interface. It exposes managed byte-span and array cryptographic operations, strongly typed enums and selectors, X.509 and TLS object lifecycles, and an optional hardware callback bridge without exposing QSC internal C structures as application contracts.

QSCCS is not a pure managed cryptography implementation. Its primitives, protocol engines, native allocation and CPU-specific code are supplied by QSC through QSCABI. The developer must provide the compatible native library, verify ABI stability, preserve secret-buffer and native-handle ownership, and qualify the compiled QSC backend on each deployment target.

This specification documents the supplied QSCCS(4).zip source. Function and property signatures in the managed API reference are extracted from public declarations; the native appendix reproduces exactly 329 LibraryImport names and their C# signatures. The paper distinguishes source-level evidence from runtime qualification, external documentation and distribution prerequisites.

2: Introduction and Supplied Source Baseline

2.1 Project purpose and boundaries

The managed wrapper uses a two-tier API: convenience methods that return owned managed values and raise QscException on an error, and Try-style methods returning QscStatus with caller-owned Span<byte> output. Native state is managed by SafeHandle and IDisposable. Interop classes declare fixed C calling signatures and route native selection through QscAbiLibraryLoader. SECRAND and native QSC-only APIs absent from QSCABI are intentionally excluded.

2.2 Source audit

Item	Verified source property
Target framework	QSCCS/QSCCS.csproj targets net8.0 and enables unsafe code and XML documentation.
Public managed API	526 public members (methods, constructors, properties and accessible fields) across 48 contributing source files; 81 public class/struct/enum/record type declarations.
Native ABI	329 distinct [LibraryImport] entry points, including the separate private qscabi_status_string import; no duplicates detected.
Qualified test surface	17 named directly called console sections and 112 explicit RunCase invocations in QSCCSTest/Program.cs.

Item	Verified source property
Managed tests	17 *Tests.cs sources; source annotations include 113 [Fact] methods and one [Theory] method, whose runtime cases may expand via InlineData.
Projects	QSCCS/QSCCS.csproj and QSCCSTest/QSCCSTest.csproj are present; supplied ZIP does not contain QSCCS.sln, Directory.Build.props, examples/, scripts/, or a native QSCABI binary.
Commercial license	Repository C# source headers contain QRCS-CASL restrictions; the full signed commercial agreement is not included in this ZIP.
Build and runtime evidence	Only source and the DOCX were inspected/generated; .NET compiler and native QSCABI were not run during this task.

Table 2.1 Supplied QSCCS project evidence

2.3 Historical document reconciliation

The bundled guides describe a reorganized Visual Studio solution, an examples/ project, scripts/audit_native_imports.py and a native staging folder. Those assets are not present in this particular archive, so their documented workflows require the complete corresponding repository or separately supplied native dependency. Likewise, the testing guide describes a 14-section/100-case earlier state; the supplied Program.cs has 17 sections and 112 explicit invocations, which is the current source evidence used by this paper.

3: References and Engineering Conventions

3.1 Supplied project sources

- QSCCS/QSCCS.csproj — Managed target framework and package metadata.
- QSCCS/Common — QscStatus, QscException, QscVersion, QscMemory and QscResult.
- QSCCS/Interop — P/Invoke declarations, native loader, Native API struct types and SafeHandle classes.
- QSCCS/Symmetric, Hash, Xof, Mac, Kdf — Managed one-shot cryptographic operations.
- QSCCS/Pqc, Asymmetric, Explicit — PQC, classical curves and selected native algorithm profiles.
- QSCCS/Streaming — Stateful SafeHandle-backed digest, XOF, MAC, CTR and customized SHAKE.
- QSCCS/Tls and QSCCS/X509 — Managed transport/PKI object lifecycle and trust operations.
- QSCCS/Hardware — C# delegate trampolines and owned provider/key facades.
- QSCCSTest/Program.cs — Fixed-order console test orchestration, pass/fail/skip output and no-pause policy.
- docs/MANAGED_API_IMPLEMENTOR_GUIDE.md — Project-authored usage, error, ownership, type and method reference.
- docs/QSCCS_010_ABI_REFERENCE.md — Project-authored ABI 0.10 native bindings reference.

- docs/QSCCS_010_TEST_EXAMPLE_MATRIX.md — Project-authored planned acceptance gates for ABI 0.10.

3.2 External implementor references

.NET 8 overview — https://learn.microsoft.com/dotnet/core/whats-new/dotnet-8/overview

Source-generated P/Invoke / LibraryImport — https://learn.microsoft.com/dotnet/standard/native-interop/pinvoke-source-generation

SafeHandle — https://learn.microsoft.com/dotnet/api/system.runtime.interopservices.safehandle

Span<T> and memory — https://learn.microsoft.com/dotnet/standard/memory-and-spans/

QSC cryptographic SDK — https://github.com/Steppenwolfe65/QSC

NIST FIPS 203 ML-KEM — https://doi.org/10.6028/NIST.FIPS.203

NIST FIPS 204 ML-DSA — https://doi.org/10.6028/NIST.FIPS.204

NIST FIPS 205 SLH-DSA — https://doi.org/10.6028/NIST.FIPS.205

RFC 8446 TLS 1.3 — https://www.rfc-editor.org/rfc/rfc8446

RFC 5280 X.509 PKI — https://www.rfc-editor.org/rfc/rfc5280

3.3 Normative implementation conventions

Byte lengths are represented by managed int/nuint only after checked conversions; a native size_t is mapped to C# nuint. A ReadOnlySpan<byte> is borrowed only for a call, and a Span<byte> output remains caller-owned. A byte[] return value is allocated managed memory; a private-key array must be explicitly zeroed. QscStatus.Success is necessary before reading any output and cannot be replaced by a non-null pointer check.

4: Architecture and .NET 8 Integration

```
C# application / managed services / test console
    | QRCS.QSCCS managed API: Span, byte[], enums, IDisposable
    v
QSCCS managed wrapper (.NET 8; source-generated LibraryImport)
    | P/Invoke with explicit C ABI, native-size widths and SafeHandle
    v
QSCABI 0.10 native library (external DLL / .so / .dylib)
    | C ABI exported qscabi_* functions
    v
QSC native cryptographic backend and TLS/X.509 engines
```

4.1 No direct dependency on QSC internal structures

QSCCS makes no public ABI contract with QSC private headers or C structure layouts. Public managed classes use copied values or opaque SafeHandle instances. Changing QSC internals without a matching QSCABI build can still affect behavior, compiled ISA requirements and cryptographic parameters, but the managed application must not bypass the C bridge.

4.2 Convenience versus Try style

Interface pattern	Behavior and ownership
byte[] Compute(...), Generate(...)	New managed result; throws QscException for native failure; caller must erase secret material.
QscStatus Try*(ReadOnlySpan<byte>, Span<byte>, ...)	No successful output on an error; caller retains and sizes the mutable destination.
QscX509Certificate / QscTlsConnection	IDisposable native owner; underlying resource released by corresponding SafeHandle.
QscVersion / capabilities	Native metadata and compiled-feature facts, not device presence or certification.
QRCS.QSCCS.Interop	Mostly internal declarations; not a general public consumer API.

Table 4.1 QSCCS programming conventions

5: Build and Runtime Deployment

5.1 Projects supplied in the archive

The archive includes the managed QSCCS library project and an x64-targeted QSCCSTest executable using xUnit and Microsoft.NET.Test.Sdk. The full solution, bundled examples and native QSCABI library described by docs/BUILD_GUIDE.md were not included. Building QSCCSTest on Windows triggers an AfterBuild native DLL copy target; it errors if qscabi.dll cannot be found at the selected path. The C# wrapper project itself does not add C/C++ header include paths.

```
# Source-supported build commands with .NET SDK 8 and native QSCABI:
dotnet restore QSCCS/QSCCS.csproj
dotnet build QSCCS/QSCCS.csproj -c Release
# When a matching native QSCABI x64 DLL is available:
dotnet run --project QSCCSTest/QSCCSTest.csproj -c Release -- --no-pause

# Commands documented by the full-repository guide, but QSCCS.sln
# and examples/ are NOT included in this uploaded ZIP:
# dotnet build QSCCS.sln -c Release
# dotnet run --project examples/QSCSDKCS.csproj -c Release
```

5.2 Native dependency discovery and staging

Windows x64 test staging resolves an optional QscAbiNativeDll or QscAbiNativeDir override, then a solution-relative x64/<Configuration>/qscabi.dll, then a sibling QSCABI/x64/<Configuration>/qscabi.dll. Output is copied beside the managed executable. Debug and Release paths are computed from the actual MSBuild Configuration; the archive does not hardcode Debug into the Release selection. An invalid or wrong-architecture DLL causes a native load failure.

5.3 Target runtime layouts

Target	Conventional native dependency path
Windows x64	runtimes/win-x64/native/qscabi.dll
Linux x64	runtimes/linux-x64/native/libqscabi.so
macOS x64	runtimes/osx-x64/native/libqscabi.dylib
macOS ARM64	runtimes/osx-arm64/native/libqscabi.dylib

Table 5.1 Recommended package layout from included guide

These directories are documented packaging targets, not bundled binaries or evidence that each operating-system build exists. Validate the interpreter/runtime, QSCABI calling convention, C runtime dependencies, native compiled profile and QSC source revision for each delivered package.

6: QSCABI Native Loader and Contract Validation

6.1 Library resolution and lock-in

QscAbiLibraryLoader registers a NativeLibrary resolver through a module initializer and offers SetNativeLibraryDirectory before the first P/Invoke. It rejects late rebinding after a successful native load and does not silently downgrade to older ABI versions. The loader checks the declared managed imports and native version/compatibility early to expose missing exports as load errors rather than deferred crashes or unpredictable missing-method behavior.

6.2 329 native symbols

All 329 distinct source-declared qscabi_* entry points were counted across the Interop/*.cs files. The `qscabi_status_string` import is declared using `QscAbiNative.LibraryName` in QscAbiStatusNative.cs, whereas the other imports conventionally use `LibraryName`. Search rules limited to only the shorter token would count 328 and give an incorrect false failure. The precise 329-name parity is presented in Section 26, which includes that status-string call.

6.3 ABI compatibility limits

QSCABI 0.10 compatibility is required; matching names alone are not sufficient. C# LibraryImport signatures must preserve correct C integer widths, unsigned native size_t lengths, pointer indirection, callback calling conventions, exported symbol spelling, structure field packing and exception-free native callbacks. A changed native status enumeration or opaque-handle ownership model requires coordination with the native interface.

7: Version, Capabilities and Compiled Backend

QscVersion provides the QSCABI major/minor/patch, version strings, QSC linked library revision, compiled backend and QSC build profile, native feature bitset and minimum-version compatibility. Use the native build-profile string to distinguish an actually compiled AVX2/AES-NI profile from the CPU feature report. A CPU that advertises AVX-512 does not imply that QSCABI was compiled for or dispatches to AVX-512. A compiled hardware capability bit does not establish the presence or trustworthiness of physical hardware.

```
using QRCS.QSCCS;

if (!QscVersion.IsAbiCompatible(0, 10))
{
    throw new InvalidOperationException("QSCABI 0.10 required.");
}
Console.WriteLine(QscVersion.AbiVersion);
Console.WriteLine(QscVersion.BuildBackend);
Console.WriteLine(QscVersion.QscBuildProfile);
```

8: Managed Status, Errors and Result Types

QscStatus mirrors native results 0 through 18, including authentication, verification, buffer capacity, platform unavailability, invalid handle/state, TLS failure and X.509 failure. QscException carries a status and diagnostic message; ThrowIfFailed raises on a non-success result. QscResult supports status-based workflows without treating failed output as initialized data.

Status category	Required caller response
Success	Use output only after checking exact written length and any application-level trust policy.
InvalidArgument / InvalidSize / BufferTooSmall	Validate parameter sets, array lengths, empty buffers and sizes before retrying.
AuthenticationFailure / VerificationFailure	Drop output and reject the ciphertext or signature; never recover by trusting partial plaintext.
Unsupported / PlatformUnavailable	Fail closed or use an explicitly approved alternative; do not silently downgrade.

Status category	Required caller response
InvalidHandle / BadState / Closed	Stop using disposed or malformed native state and inspect lifecycle.
TlsFailure / X509Failure	Distinguish transport failure from trust or identity validation failure.

Table 8.1 Managed error policy

9: Secret Buffers, SafeHandle and Resource Lifetime

9.1 Span, memory erasure and secret arrays

ReadOnlySpan<byte> input is pinned or borrowed only for the call. Span<byte> destinations and byte[] outputs belong to the managed caller, and memory is not automatically erased by the .NET garbage collector. For secret-bearing results, apply CryptographicOperations.ZeroMemory or QscMemory.Clear in a finally block where possible. A struct holding a key may copy the underlying key bytes or references; inspect the actual wrapper ownership semantics rather than assuming a value type is inherently secure.

9.2 SafeHandle and IDisposable

QscAbiSafeHandles.cs defines distinct owning handles for X.509 certificate/chain/store, TLS context/connection/listener/server and 0.10 streaming state. The corresponding public objects call Dispose and should be enclosed in using declarations. A SafeHandle release call is not a substitute for checking earlier authentication failures; do not reuse a disposed or closed handle.

```
using var context = QscTlsContext.Create();
// Configure trust anchors and policy before opening connections.
// Each native owner is disposed automatically at scope exit.
```

9.3 Threading and error pathways

The native loader uses process-level synchronization for initialization. Stateless managed wrappers can be used from independent threads, but mutable TLS, X.509, hardware, CTR, XOF and digest objects are not guaranteed concurrent-safe. Use separate handles or external serialization. C-style hardware callbacks must catch managed exceptions and convert them into native status; allowing exceptions to unwind through unmanaged code is prohibited.

10: Encoding, Entropy and Memory Utilities

QscEncoding wraps hex, Base64 and PEM with allocating and Try forms. qscabi_*_encoded_size helpers should drive buffer capacity, while exact produced lengths

and parsed content must still be checked. QscEntropy supports ACP, CSP, RDP and CJP providers and exposes availability predicates. Not every provider is available on every host; no fabricated predictable fallback is acceptable for production cryptography. QscMemory exposes secure clearing and native byte comparison, which cannot replace MAC verification or certificate trust.

11: SHA-2, SHA-3 and SHAKE

QscSha2 and QscSha3 expose fixed digest sizes, Compute and TryCompute families; QscShake supports SHAKE128/256/512 variable-output digest generation through QSCABI. The input is an arbitrary byte message. Output length and domain separation are protocol decisions; an unkeyed hash/XOF does not authenticate a sender. The native capabilities and exact hash lengths must be checked when a profile is selected dynamically.

12: MAC and Key Derivation

QscHmac, QscKmac, QscQmac, QscPoly1305, QscPbmac1 and QscHkdf/QscPbmac1Kdf implement keyed authentication and derivation calls from QSCABI. Use fresh, properly scoped keys and unambiguous domain/label separation. Poly1305 one-time subkeys cannot safely be reused across independent records. HKDF extract and expand have different salt/info roles and should bind the application transcript and target purpose. Password-based derivation policy must control salt generation, work factor and secure handling of password bytes.

13: Symmetric Ciphers and AEAD

Family	Managed wrapper	Security contract
AES CTR	QscAes	Keystream confidentiality only; counters nonrepeating; no inherent authentication.
AES GCM	QscAes	AEAD with associated data; reject tampering before accepting plaintext.
ChaCha20	QscChaCha	Raw stream cipher; use with authenticated protocol only.
ChaCha20-Poly1305	QscChaChaPoly1305	AEAD, unique key/nonce pair and 16-byte tag per format.
CSX	QscCsx	QSC authenticated cipher; respect exact QSC mode and tag size.
RCS 256/512	QscRcs	Native selected key profile; control nonce, authentication and output lengths.

Table 13.1 Symmetric wrapper responsibilities

GCM, ChaCha-Poly1305, CSX and RCS decryption must be considered successful only if native tag verification succeeds. Do not expose partially filled output on authentication failure. Raw AES CTR and ChaCha encryption do not provide authenticity, replay defense or key confirmation. Never reuse a nonce under the same authenticated-encryption key. Use QSCABI size getters rather than hardcoding profile-sensitive lengths.

14: Post-Quantum Key Encapsulation and Signatures

QscMlKem, QscHqc and QscMcEliece provide KEM key generation, encapsulation and decapsulation. QscMlDsa, QscFalcon and QscSlhDsa provide signed-message generation and verification. A KEM shared secret does not identify the remote peer; an attached signed-message representation is not the same as a detached signature, and verify operations must preserve the exact encoded message and selected parameter set.

The included project testing guide isolates Falcon while native full-suite reliability is still being assessed. The presence of the C# QscFalcon surface is not an endorsement of certification or cross-platform qualification. Use the native algorithm/version capabilities and the application's verified key-provenance policy before attempting any communication handshake.

15: Classical Curves and Public-Key Operations

QscEcdh, QscEddh, QscEcdsa and QscEddsa expose keypair generation, seeded operations, public-key derivation, key agreement, signing and verification as supported by native QSCABI. Curve choice, seed/private-key lengths, public-key format and attached versus detached signature encoding must match peer protocol rules. ECDH/EDDH agreement is unauthenticated without certificate, signature or transcript binding.

16: Explicit Algorithms and Selector Contracts

ABI 0.10 adds QscExplicitKem and QscExplicitCurve for runtime algorithm/curve selectors, plus QscExplicitSignature and QscExplicitSlhDsa with parameter-specific sizes. Explicit KEM enumerators include ML-KEM profiles and explicit curve selections include NIST P-256/P-384/P-521 and X25519/X448. The source uses exact native-provided length metadata for key pairs and agreement results. Do not substitute a parameter set based only on array length.

```
using QRCS.QSCCS.Explicit;
using var keypair = QscExplicitKem.Generate(QscKemAlgorithm.Mlkem512);
using var enc = QscExplicitKem.Encapsulate(
    QscKemAlgorithm.Mlkem512, keypair.PublicKey);
byte[] secret = QscExplicitKem.Decapsulate(
    QscKemAlgorithm.Mlkem512, enc.Ciphertext, keypair.PrivateKey);
// Compare/zero the recovered secret in a finally block.
```

For signatures, the attached EdDSA/classical message layout and detached FIPS 205 SLH-DSA layout have different verification contracts. Explicit SLH-DSA adds prehash choice, context bytes and randomization; bind these values to the negotiated protocol and reject unsupported combinations. Size metadata and enum discriminants must match the native QSCABI header.

17: Streaming Digest, XOF, MAC and CTR

Managed state	Call sequence	Failure behavior
QscIncrementalDigest	construct → Update* → FinalizeDigest → Dispose	Single finalization; reject operations after disposal.
QscIncrementalXof	construct → Absorb* → FinalizeXof → Squeeze* → Dispose	No absorb after finalization; output length explicit.
QscIncrementalMac	construct key → Update* → FinalizeMac → Dispose	Authentication tag generated once; key state destroyed at close.
QscCtrStream	construct key/IV → Transform* → Dispose	No integrity protection; prevent counter reuse.
QscCustomizedShake	one-shot Compute with customization	Customization is cryptographic domain separation.

Table 17.1 Incremental state model

Each stateful streaming object owns a SafeHandle and is not thread-safe unless the application provides synchronization. Output length and native status must be validated after every call. Streaming CTR must not be mistaken for stream-AEAD; authentication requires a separate protected construction or another authenticated cipher.

18: X.509 Certificates, Chains and Trust Stores

QscX509Certificate owns a native parsed certificate and supplies subject, issuer, SAN, export and hostname checks. QscX509Chain owns an ordered native chain. QscX509TrustStore owns trusted roots and performs verification. Loading a certificate or matching a string is not sufficient to establish trusted identity; validate the issuer path, time, key usage, purpose, revocation and exact intended hostname. Use using/Dispose to release all objects.

19: X.509 Issuance, CSR and CRL

QscX509Issuance.cs adds managed wrapper types for CSR creation/verification, CRL preparation and native P-256 DER signing callbacks. Certificate signing/issuance is a privileged CA operation, and a syntactically valid CSR must still be authorized for the requested identity. The native callback must receive validated data, produce the requested signature format and catch all managed exceptions before returning to unmanaged code. Keep the issuer certificate/key material valid through the complete native operation.

The default ABI 0.10 CSR test checks rejection of an unsigned request. That negative test does not demonstrate a trusted issued certificate, real CA lifecycle, revocation policy or certificate-path interoperability.

20: TLS 1.3 Client and Server Lifecycle

20.1 Context and peer setup

QscTlsContext configures role, trusted anchors, chain and private-key file paths, cipher/group/signature preference, ALPN, SNI, client-authentication and session-ticket policy. Native TLS preference identifiers are 16-bit wire values. A configured context is not a completed authenticated TLS handshake; the caller must check native connection state and peer verification before transmitting application data.

20.2 Client, listener and server

```
using var ctx = QscTlsContext.Create();
ctx.SetCipherSuites(new ushort[] { 0x1302, 0x1301 });
ctx.SetNamedGroups(new ushort[] { 0x0017 });
ctx.SetSignatureIds(new ushort[] { 0x0403 });
// Load trusted CA files and protected identity files.
// Establish a QscTlsConnection, authenticate the peer, then send frames.
// Dispose connection/listener/server before disposing context.
```

QscTlsConnection exposes native connect, send/receive, frame I/O, shutdown, cancellation and negotiated peer information. QscTlsListener and QscTlsServer own accept/bind and managed server loops. The caller handles timeouts, partial reads, cancel propagation, connection authorization, ALPN protocol dispatch and orderly shutdown. The default test sources emphasize lifecycle and configuration; live network/TLS interchange is not established in this archive.

20.3 mTLS, file loaders and trust

mTLS requires correctly configured server trust anchors, client certificate/key files, certificate-chain verification and application authorization. QSCABI source paths for file-

based TLS identity and trust must be protected by filesystem permissions. Loading a file does not prove the peer has a valid certificate; session authorization requires inspecting peer chain/hostname and policy results after handshake.

21: Hardware Provider Callback Bridge

The C# hardware module defines QscHardwareCallbackSet and a managed QscHardwareProvider / hardware key ownership model. Provider callbacks are retained as rooted delegates for the entire provider lifetime, with native pointer types and size/version-qualified capability structures. Trampolines catch exceptions and convert them to failure results rather than unwinding managed exceptions across a C callback boundary.

Hardware IsEnabled indicates compiled bridge support, not a present physical TPM/HSM/secure element, certified implementation or hardware-backed key provenance. Key release and provider close are distinct, and a provider with active keys must not be destroyed prematurely. Caller-supplied nint backend context memory must remain valid through provider teardown. Physical device disconnect and attestation trust require separate tests.

22: Example Implementation Patterns

The included docs/EXAMPLES_IMPLEMENTOR_GUIDE.md documents a broader QSCSDKCS command catalog. Since examples/ is not in this ZIP, those commands are reference workflows rather than independently executable examples from this snapshot. The following minimal patterns use methods observed in the managed source and should be integrated with actual key management and error handling.

22.1 Hashing an untrusted message

```
using QRCS.QSCCS.Hash;
byte[] digest = QscSha2.Compute256("message"u8);
// Digest alone does not authenticate the sender.
```

22.2 Secret lifecycle for KEM agreement

```
using System.Security.Cryptography;
using QRCS.QSCCS.Explicit;
using var local = QscExplicitKem.Generate(QscKemAlgorithm.Mlkem512);
using var sealedSecret = QscExplicitKem.Encapsulate(
    QscKemAlgorithm.Mlkem512, local.PublicKey);
byte[] recovered = QscExplicitKem.Decapsulate(
    QscKemAlgorithm.Mlkem512, sealedSecret.Ciphertext, local.PrivateKey);
try
{
    if (!CryptographicOperations.FixedTimeEquals(
        recovered, sealedSecret.SharedSecret))
    {
        throw new CryptographicException("KEM secret mismatch");
    }
}
finally
{
    CryptographicOperations.ZeroMemory(recovered);
}
```

22.3 Implementation checklist

- Verify ABI 0.10, 329 imports and compiled QSC build profile before any cryptographic work.
- Prefer the managed safe API rather than calling the Interop namespace directly.
- Check every QscStatus and exact output length; never inspect failed authentication output.
- Dispose native state using SafeHandle-based managed owners.
- Wipe secret-bearing byte[] and temporary buffers after use.
- Separate KEM agreement from peer identity proof, and TLS handshake completion from certificate authorization.
- Test invalid sizes, tags, signatures, handles, callbacks, network failures and wrong native binary profiles.

23: Tests and Qualification Evidence

23.1 Console runner

The uploaded QSCCSTest/Program.cs contains 17 directly invoked section methods and 112 explicit named RunCase calls. It prints a QRCS-style information header, CPU capability report, right-aligned PASS/FAIL/SKIP result tags and final qualification summary. ConsoleSkipAttribute marks certain throw-oriented tests as SKIP in interactive debugging without changing their separate xUnit semantics. `--no-pause` disables the optional interactive close prompt.

23.2 Source tests and coverage

The uploaded test files use xUnit Fact and Theory attributes, with parameterized InlineData cases that may expand beyond their method count. The console runner deliberately invokes each case using direct named statements and does not use a method registry or reflection-based test dispatch. The presence of the test definitions is not a PASS result; actual native and .NET test results must come from an executed, logged build against the matching QSCABI DLL.

23.3 Qualification boundaries

The new ABI 0.10 tests exercise import count, metadata, P-256/X25519 agreement, ML-KEM512, Ed25519/SLH-DSA, incremental SHAKE/HMAC/AES-CTR, TLS preferences, unsigned CSR rejection and native struct sizes. They do not collectively prove an authorized CA, a real TLS peer handshake, an attached HSM, certification of QRCS symmetric designs or platform-wide portability. No tests or compiled builds were executed for this documentation assignment.

24: Security and Cross-Platform Constraints

- Maintain binary agreement between QSCABI C exports and managed LibraryImport signature, including uint32, nuint, bool, pointer indirection and calling convention.
- Never select an AVX/AES-NI/AVX-512 QSC native package solely from managed CPU metadata without verifying platform profile and runtime support.
- Use named native algorithm parameters for cryptographic sizes; avoid profile-incompatible static key lengths.
- Keep every mutable TLS/X.509/stream/hardware object single-thread-owned, synchronized externally or separately allocated.
- Use QscMemory.Clear/CryptographicOperations.ZeroMemory for secret-bearing managed arrays, including error pathways.
- Use independent certificate trust policy and remote peer authorization after successful cryptographic verification.
- Do not treat source symbols or feature bits as certification, attestation or real network interoperability.
- Preserve QRCS-CASL and authorized distribution restrictions, and do not modify protected external QSC/QSCABI dependencies as a documentation shortcut.

25: Complete Managed C# API Reference

This section lists 526 accessible managed C# declarations from 48 source files in the uploaded QSCCS/ tree, including overloads, constructors, properties and callback configuration members. It excludes private/internal methods and the separate P/Invoke layer. All signatures are reproduced from source declarations, with method-specific XML documentation summaries, parameter descriptions and return contracts where those tags exist. Public type declarations are indexed in Annex A.

25.1 Asymmetric/QscEcdh.cs (12 members)

Declaration source: QSCCS/Asymmetric/QscEcdh.cs. A throwing operation and a status-returning Try variant are distinct contracts; private native handles remain implementation details.

QscEcdh.PublicKeySize [property]

public static int PublicKeySize

Purpose: Gets the public-key size, in bytes, required by the native QSCABI algorithm profile.

QscEcdh.PrivateKeySize [property]

public static int PrivateKeySize

Purpose: Gets the private-key size, in bytes, required by the native QSCABI algorithm profile.

QscEcdh.SharedSecretSize [property]

public static int SharedSecretSize

Purpose: Gets the shared-secret size, in bytes, produced by encapsulation and decapsulation.

QscEcdh.SeedSize [property]

public static int SeedSize

Purpose: Gets the deterministic key-generation seed size, in bytes, required by the native algorithm profile.

QscEcdh.GenerateKeyPair [method]

public static QscAsymmetricKeyPair GenerateKeyPair()

Purpose: Generates a new key pair using QSCABI randomness.

Returns: A managed key-pair object containing newly allocated public and private key arrays.

QscEcdh.GenerateSeededKeyPair [method]

public static QscAsymmetricKeyPair GenerateSeededKeyPair(ReadOnlySpan<byte> seed)

Purpose: Generates a deterministic key pair from a caller-supplied seed.

Parameters: seed: The deterministic seed. The length must equal SeedSize.

Returns: A managed key-pair object derived from the seed.

QscEcdh.PublicFromPrivate [method]

public static byte[] PublicFromPrivate(ReadOnlySpan<byte> privateKey)

Purpose: Derives a public key from a private key when the native algorithm exposes that operation.

Parameters: privateKey: The private key. The length must equal PrivateKeySize.

Returns: The derived public key.

QscEcdh.KeyExchange [method]

public static byte[] KeyExchange(ReadOnlySpan<byte> privateKey, ReadOnlySpan<byte> publicKey)

Purpose: Computes a shared secret from a local private key and a peer public key.

Parameters: privateKey: The local private key. The length must equal PrivateKeySize.; publicKey: The peer public key. The length must equal PublicKeySize.

Returns: The computed shared secret.

QscEcdh.TryGenerateKeyPair [method]

public static unsafe QscStatus TryGenerateKeyPair(Span<byte> publicKey, Span<byte> privateKey)

Purpose: Attempts to generate a key pair into caller-owned buffers.

Parameters: publicKey: The destination buffer for the public key. The buffer must be at least PublicKeySize bytes.; privateKey: The destination buffer for the private key. The buffer must be at least PrivateKeySize bytes.

Returns: QscStatus.Success on success; otherwise a QSCABI status code describing invalid sizes, insufficient output space, unsupported operations, or verification failure.

QscEcdh.TryGenerateSeededKeyPair [method]

public static unsafe QscStatus TryGenerateSeededKeyPair(ReadOnlySpan<byte> seed, Span<byte> publicKey, Span<byte> privateKey)

Purpose: Attempts to generate a deterministic key pair into caller-owned buffers.

Parameters: seed: The deterministic seed. The length must equal SeedSize.; publicKey: The destination buffer for the public key. The buffer must be at least PublicKeySize bytes.; privateKey: The destination buffer for the private key. The buffer must be at least PrivateKeySize bytes.

Returns: QscStatus.Success on success; otherwise a QSCABI status code describing invalid sizes, insufficient output space, unsupported operations, or verification failure.

QscEcdh.TryPublicFromPrivate [method]

public static unsafe QscStatus TryPublicFromPrivate(ReadOnlySpan<byte> privateKey, Span<byte> publicKey)

Purpose: Attempts to derive a public key from a private key into a caller-owned buffer.

Parameters: privateKey: The private key. The length must equal PrivateKeySize.; publicKey: The destination buffer for the public key. The buffer must be at least PublicKeySize bytes.

Returns: QscStatus.Success on success; otherwise a QSCABI status code describing invalid sizes, insufficient output space, unsupported operations, or verification failure.

QscEcdh.TryKeyExchange [method]

public static unsafe QscStatus TryKeyExchange(ReadOnlySpan<byte> privateKey, ReadOnlySpan<byte> publicKey, Span<byte> secret)

Purpose: Attempts to compute a shared secret into a caller-owned buffer.

Parameters: privateKey: The local private key. The length must equal PrivateKeySize.; publicKey: The peer public key. The length must equal PublicKeySize.; secret: The destination buffer for the shared secret. The buffer must be at least SharedSecretSize bytes.

Returns: QscStatus.Success on success; otherwise a QSCABI status code describing invalid sizes, insufficient output space, unsupported operations, or verification failure.

25.2 Asymmetric/QscEcdsa.cs (15 members)

Declaration source: QSCCS/Asymmetric/QscEcdsa.cs. A throwing operation and a status-returning Try variant are distinct contracts; private native handles remain implementation details.

QscEcdsa.PublicKeySize [property]

public static int PublicKeySize

Purpose: Gets the public-key size, in bytes, required by the native QSCABI algorithm profile.

QscEcdsa.PrivateKeySize [property]

public static int PrivateKeySize

Purpose: Gets the private-key size, in bytes, required by the native QSCABI algorithm profile.

QscEcdsa.SignatureSize [property]

public static int SignatureSize

Purpose: Gets the detached signature size, in bytes, for the native signature profile.

QscEcdsa.SeedSize [property]

public static int SeedSize

Purpose: Gets the deterministic key-generation seed size, in bytes, required by the native algorithm profile.

QscEcdsa.SignedMessageSize [method]

public static int SignedMessageSize(int messageLength)

Purpose: Calculates the signed-message buffer size required for a message of the supplied length.

Parameters: messageLength: The non-negative message length, in bytes.

Returns: The required signed-message buffer size, in bytes.

QscEcdsa.GenerateKeyPair [method]

public static QscAsymmetricKeyPair GenerateKeyPair()

Purpose: Generates a new key pair using QSCABI randomness.

Returns: A managed key-pair object containing newly allocated public and private key arrays.

QscEcdsa.GenerateSeededKeyPair [method]

public static QscAsymmetricKeyPair GenerateSeededKeyPair(ReadOnlySpan<byte> seed)

Purpose: Generates a deterministic key pair from a caller-supplied seed.

Parameters: seed: The deterministic seed. The length must equal SeedSize.

Returns: A managed key-pair object derived from the seed.

QscEcdsa.PublicFromPrivate [method]

public static byte[] PublicFromPrivate(ReadOnlySpan<byte> privateKey)

Purpose: Derives a public key from a private key when the native algorithm exposes that operation.

Parameters: privateKey: The private key. The length must equal PrivateKeySize.

Returns: The derived public key.

QscEcdsa.Sign [method]

public static byte[] Sign(ReadOnlySpan<byte> message, ReadOnlySpan<byte> privateKey)

Purpose: Signs a message using the supplied private key.

Parameters: message: The message bytes to sign. The buffer is read during the call and is not retained.; privateKey: The private signing key. The length must equal PrivateKeySize.

Returns: A signed-message buffer containing the signature and recoverable message representation produced by QSCABI.

QscEcdsa.Verify [method]

public static byte[] Verify(ReadOnlySpan<byte> signedMessage, ReadOnlySpan<byte> publicKey)

Purpose: Verifies a signed message and recovers the authenticated message bytes.

Parameters: signedMessage: The signed-message buffer returned by Sign(ReadOnlySpan{byte}, ReadOnlySpan{byte}).; publicKey: The public verification key. The length must equal PublicKeySize.

Returns: The recovered message bytes when verification succeeds.

QscEcdsa.TryGenerateKeyPair [method]

public static unsafe QscStatus TryGenerateKeyPair(Span<byte> publicKey, Span<byte> privateKey)

Purpose: Attempts to generate a key pair into caller-owned buffers.

Parameters: publicKey: The destination buffer for the public key. The buffer must be at least PublicKeySize bytes.; privateKey: The destination buffer for the private key. The buffer must be at least PrivateKeySize bytes.

Returns: QscStatus.Success on success; otherwise a QSCABI status code describing invalid sizes, insufficient output space, unsupported operations, or verification failure.

QscEcdsa.TryGenerateSeededKeyPair [method]

public static unsafe QscStatus TryGenerateSeededKeyPair(ReadOnlySpan<byte> seed, Span<byte> publicKey, Span<byte> privateKey)

Purpose: Attempts to generate a deterministic key pair into caller-owned buffers.

Parameters: seed: The deterministic seed. The length must equal SeedSize.; publicKey: The destination buffer for the public key. The buffer must be at least PublicKeySize bytes.; privateKey: The destination buffer for the private key. The buffer must be at least PrivateKeySize bytes.

Returns: QscStatus.Success on success; otherwise a QSCABI status code describing invalid sizes, insufficient output space, unsupported operations, or verification failure.

QscEcdsa.TryPublicFromPrivate [method]

public static unsafe QscStatus TryPublicFromPrivate(ReadOnlySpan<byte> privateKey, Span<byte> publicKey)

Purpose: Attempts to derive a public key from a private key into a caller-owned buffer.

Parameters: privateKey: The private key. The length must equal PrivateKeySize.; publicKey: The destination buffer for the public key. The buffer must be at least PublicKeySize bytes.

Returns: QscStatus.Success on success; otherwise a QSCABI status code describing invalid sizes, insufficient output space, unsupported operations, or verification failure.

QscEcdsa.TrySign [method]

public static unsafe QscStatus TrySign(ReadOnlySpan<byte> message, ReadOnlySpan<byte> privateKey, Span<byte> signedMessage, out int written)

Purpose: Attempts to sign a message into a caller-owned signed-message buffer.

Parameters: message: The message bytes to sign.; privateKey: The private signing key. The length must equal PrivateKeySize.; signedMessage: The destination buffer for the signed-message representation.; written: Receives the number of bytes written to .

Returns: QscStatus.Success on success; otherwise a QSCABI status code describing invalid sizes, insufficient output space, unsupported operations, or verification failure.

QscEcdsa.TryVerify [method]

public static unsafe QscStatus TryVerify(ReadOnlySpan<byte> signedMessage, ReadOnlySpan<byte> publicKey, Span<byte> message, out int written)

Purpose: Attempts to verify a signed message and recover the authenticated message bytes.

Parameters: signedMessage: The signed-message representation to verify.; publicKey: The public verification key. The length must equal PublicKeySize.; message: The destination buffer for the recovered message.; written: Receives the number of bytes written to .

Returns: QscStatus.Success on success; otherwise a QSCABI status code describing invalid sizes, insufficient output space, unsupported operations, or verification failure.

25.3 Asymmetric/QscEddh.cs (12 members)

Declaration source: QSCCS/Asymmetric/QscEddh.cs. A throwing operation and a status-returning Try variant are distinct contracts; private native handles remain implementation details.

QscEddh.PublicKeySize [property]

public static int PublicKeySize

Purpose: Gets the public-key size, in bytes, required by the native QSCABI algorithm profile.

QscEddh.PrivateKeySize [property]

public static int PrivateKeySize

Purpose: Gets the private-key size, in bytes, required by the native QSCABI algorithm profile.

QscEddh.SharedSecretSize [property]

public static int SharedSecretSize

Purpose: Gets the shared-secret size, in bytes, produced by encapsulation and decapsulation.

QscEddh.SeedSize [property]

public static int SeedSize

Purpose: Gets the deterministic key-generation seed size, in bytes, required by the native algorithm profile.

QscEddh.GenerateKeyPair [method]

public static QscAsymmetricKeyPair GenerateKeyPair()

Purpose: Generates a new key pair using QSCABI randomness.

Returns: A managed key-pair object containing newly allocated public and private key arrays.

QscEddh.GenerateSeededKeyPair [method]

public static QscAsymmetricKeyPair GenerateSeededKeyPair(ReadOnlySpan<byte> seed)

Purpose: Generates a deterministic key pair from a caller-supplied seed.

Parameters: seed: The deterministic seed. The length must equal SeedSize.

Returns: A managed key-pair object derived from the seed.

QscEddh.PublicFromPrivate [method]

public static byte[] PublicFromPrivate(ReadOnlySpan<byte> privateKey)

Purpose: Derives a public key from a private key when the native algorithm exposes that operation.

Parameters: privateKey: The private key. The length must equal PrivateKeySize.

Returns: The derived public key.

QscEddh.KeyExchange [method]

public static byte[] KeyExchange(ReadOnlySpan<byte> privateKey, ReadOnlySpan<byte> publicKey)

Purpose: Computes a shared secret from a local private key and a peer public key.

Parameters: privateKey: The local private key. The length must equal PrivateKeySize.; publicKey: The peer public key. The length must equal PublicKeySize.

Returns: The computed shared secret.

QscEddh.TryGenerateKeyPair [method]

public static unsafe QscStatus TryGenerateKeyPair(Span<byte> publicKey, Span<byte> privateKey)

Purpose: Attempts to generate a key pair into caller-owned buffers.

Parameters: publicKey: The destination buffer for the public key. The buffer must be at least PublicKeySize bytes.; privateKey: The destination buffer for the private key. The buffer must be at least PrivateKeySize bytes.

Returns: QscStatus.Success on success; otherwise a QSCABI status code describing invalid sizes, insufficient output space, unsupported operations, or verification failure.

QscEddh.TryGenerateSeededKeyPair [method]

public static unsafe QscStatus TryGenerateSeededKeyPair(ReadOnlySpan<byte> seed, Span<byte> publicKey, Span<byte> privateKey)

Purpose: Attempts to generate a deterministic key pair into caller-owned buffers.

Parameters: seed: The deterministic seed. The length must equal SeedSize.; publicKey: The destination buffer for the public key. The buffer must be at least PublicKeySize bytes.; privateKey: The destination buffer for the private key. The buffer must be at least PrivateKeySize bytes.

Returns: QscStatus.Success on success; otherwise a QSCABI status code describing invalid sizes, insufficient output space, unsupported operations, or verification failure.

QscEddh.TryPublicFromPrivate [method]

public static unsafe QscStatus TryPublicFromPrivate(ReadOnlySpan<byte> privateKey, Span<byte> publicKey)

Purpose: Attempts to derive a public key from a private key into a caller-owned buffer.

Parameters: privateKey: The private key. The length must equal PrivateKeySize.; publicKey: The destination buffer for the public key. The buffer must be at least PublicKeySize bytes.

Returns: QscStatus.Success on success; otherwise a QSCABI status code describing invalid sizes, insufficient output space, unsupported operations, or verification failure.

QscEddh.TryKeyExchange [method]

public static unsafe QscStatus TryKeyExchange(ReadOnlySpan<byte> privateKey, ReadOnlySpan<byte> publicKey, Span<byte> secret)

Purpose: Attempts to compute a shared secret into a caller-owned buffer.

Parameters: privateKey: The local private key. The length must equal PrivateKeySize.; publicKey: The peer public key. The length must equal PublicKeySize.; secret: The destination buffer for the shared secret. The buffer must be at least SharedSecretSize bytes.

Returns: QscStatus.Success on success; otherwise a QSCABI status code describing invalid sizes, insufficient output space, unsupported operations, or verification failure.

25.4 Asymmetric/QscEddsa.cs (15 members)

Declaration source: QSCCS/Asymmetric/QscEddsa.cs. A throwing operation and a status-returning Try variant are distinct contracts; private native handles remain implementation details.

QscEddsa.PublicKeySize [property]

public static int PublicKeySize

Purpose: Gets the public-key size, in bytes, required by the native QSCABI algorithm profile.

QscEddsa.PrivateKeySize [property]

public static int PrivateKeySize

Purpose: Gets the private-key size, in bytes, required by the native QSCABI algorithm profile.

QscEddsa.SignatureSize [property]

public static int SignatureSize

Purpose: Gets the detached signature size, in bytes, for the native signature profile.

QscEddsa.SeedSize [property]

public static int SeedSize

Purpose: Gets the deterministic key-generation seed size, in bytes, required by the native algorithm profile.

QscEddsa.SignedMessageSize [method]

public static int SignedMessageSize(int messageLength)

Purpose: Calculates the signed-message buffer size required for a message of the supplied length.

Parameters: messageLength: The non-negative message length, in bytes.

Returns: The required signed-message buffer size, in bytes.

QscEddsa.GenerateKeyPair [method]

public static QscAsymmetricKeyPair GenerateKeyPair()

Purpose: Generates a new key pair using QSCABI randomness.

Returns: A managed key-pair object containing newly allocated public and private key arrays.

QscEddsa.GenerateSeededKeyPair [method]

public static QscAsymmetricKeyPair GenerateSeededKeyPair(ReadOnlySpan<byte> seed)

Purpose: Generates a deterministic key pair from a caller-supplied seed.

Parameters: seed: The deterministic seed. The length must equal SeedSize.

Returns: A managed key-pair object derived from the seed.

QscEddsa.PublicFromPrivate [method]

public static byte[] PublicFromPrivate(ReadOnlySpan<byte> privateKey)

Purpose: Derives an EdDSA public key from a private signing key.

Parameters: privateKey: The private signing key. The length must equal PrivateKeySize.

Returns: A newly allocated public verification key.

QscEddsa.Sign [method]

public static byte[] Sign(ReadOnlySpan<byte> message, ReadOnlySpan<byte> privateKey)

Purpose: Signs a message using the supplied private key.

Parameters: message: The message bytes to sign. The buffer is read during the call and is not retained.; privateKey: The private signing key. The length must equal PrivateKeySize.

Returns: A signed-message buffer containing the signature and recoverable message representation produced by QSCABI.

QscEddsa.Verify [method]

public static byte[] Verify(ReadOnlySpan<byte> signedMessage, ReadOnlySpan<byte> publicKey)

Purpose: Verifies a signed message and recovers the authenticated message bytes.

Parameters: signedMessage: The signed-message buffer returned by Sign(ReadOnlySpan{byte}, ReadOnlySpan{byte}).; publicKey: The public verification key. The length must equal PublicKeySize.

Returns: The recovered message bytes when verification succeeds.

QscEddsa.TryGenerateKeyPair [method]

public static unsafe QscStatus TryGenerateKeyPair(Span<byte> publicKey, Span<byte> privateKey)

Purpose: Attempts to generate a key pair into caller-owned buffers.

Parameters: publicKey: The destination buffer for the public key. The buffer must be at least PublicKeySize bytes.; privateKey: The destination buffer for the private key. The buffer must be at least PrivateKeySize bytes.

Returns: QscStatus.Success on success; otherwise a QSCABI status code describing invalid sizes, insufficient output space, unsupported operations, or verification failure.

QscEddsa.TryGenerateSeededKeyPair [method]

public static unsafe QscStatus TryGenerateSeededKeyPair(ReadOnlySpan<byte> seed, Span<byte> publicKey, Span<byte> privateKey)

Purpose: Attempts to generate a deterministic key pair into caller-owned buffers.

Parameters: seed: The deterministic seed. The length must equal SeedSize.; publicKey: The destination buffer for the public key. The buffer must be at least PublicKeySize bytes.; privateKey: The destination buffer for the private key. The buffer must be at least PrivateKeySize bytes.

Returns: QscStatus.Success on success; otherwise a QSCABI status code describing invalid sizes, insufficient output space, unsupported operations, or verification failure.

QscEddsa.TryPublicFromPrivate [method]

public static unsafe QscStatus TryPublicFromPrivate(ReadOnlySpan<byte> privateKey, Span<byte> publicKey)

Purpose: Attempts to derive an EdDSA public key from a private signing key into a caller-owned buffer.

Parameters: privateKey: The private signing key. The length must equal PrivateKeySize.; publicKey: The destination buffer for the public key. The buffer must be at least PublicKeySize bytes.

Returns: QscStatus.Success on success; otherwise a QSCABI status code describing invalid sizes or insufficient output space.

QscEddsa.TrySign [method]

public static unsafe QscStatus TrySign(ReadOnlySpan<byte> message, ReadOnlySpan<byte> privateKey, Span<byte> signedMessage, out int written)

Purpose: Attempts to sign a message into a caller-owned signed-message buffer.

Parameters: message: The message bytes to sign.; privateKey: The private signing key. The length must equal PrivateKeySize.; signedMessage: The destination buffer for the signed-message representation.; written: Receives the number of bytes written to .

Returns: QscStatus.Success on success; otherwise a QSCABI status code describing invalid sizes, insufficient output space, unsupported operations, or verification failure.

QscEddsa.TryVerify [method]

public static unsafe QscStatus TryVerify(ReadOnlySpan<byte> signedMessage, ReadOnlySpan<byte> publicKey, Span<byte> message, out int written)

Purpose: Attempts to verify a signed message and recover the authenticated message bytes.

Parameters: signedMessage: The signed-message representation to verify.; publicKey: The public verification key. The length must equal PublicKeySize.; message: The destination buffer for the recovered message.; written: Receives the number of bytes written to .

Returns: QscStatus.Success on success; otherwise a QSCABI status code describing invalid sizes, insufficient output space, unsupported operations, or verification failure.

25.5 Common/QscException.cs (4 members)

Declaration source: QSCCS/Common/QscException.cs. A throwing operation and a status-returning Try variant are distinct contracts; private native handles remain implementation details.

QscException.QscException [constructor]

public QscException(QscStatus status) : base(CreateMessage(status))

Purpose: Initializes a new instance of the QscException class using the native status string returned by QSCABI.

Parameters: status: The QSCABI status that caused the exception.

QscException.QscException [constructor]

public QscException(QscStatus status, string message) : base(message)

Purpose: Initializes a new instance of the QscException class with a custom diagnostic message.

Parameters: status: The QSCABI status that caused the exception.; message: The exception message to expose to the caller.

QscException.QscException [constructor]

public QscException(QscStatus status, string message, Exception innerException) : base(message, innerException)

Purpose: Initializes a new instance of the QscException class with a custom diagnostic message and an inner exception.

Parameters: status: The QSCABI status that caused the exception.; message: The exception message to expose to the caller.; innerException: The exception that caused this wrapper exception.

QscException.Status [property]

public QscStatus Status

Purpose: Gets the native QSCABI status associated with this exception.

25.6 Common/QscMemory.cs (2 members)

Declaration source: QSCCS/Common/QscMemory.cs. A throwing operation and a status-returning Try variant are distinct contracts; private native handles remain implementation details.

QscMemory.Clear [method]

public static void Clear(Span<byte> data)

Purpose: Clears a caller-owned byte buffer using QSC native clearing semantics.

Parameters: data: The managed byte buffer to clear in place.

QscMemory.AreEqual [method]

public static bool AreEqual(ReadOnlySpan<byte> left, ReadOnlySpan<byte> right)

Purpose: Compares two byte spans using QSC constant-time comparison semantics.

Parameters: left: The first value to compare.; right: The second value to compare.

Returns: when both spans have the same length and the same byte contents; otherwise, .

25.7 Common/QscResult.cs (5 members)

Declaration source: QSCCS/Common/QscResult.cs. A throwing operation and a status-returning Try variant are distinct contracts; private native handles remain implementation details.

QscResult.QscResult [constructor]

public QscResult(QscStatus status)

Purpose: Initializes a new instance of the QscResult structure.

Parameters: status: The status represented by the result.

QscResult.Status [property]

public QscStatus Status

Purpose: Gets the operation status.

QscResult.Succeeded [property]

public bool Succeeded

Purpose: Gets a value indicating whether the result represents success.

QscResult.Failed [property]

public bool Failed

Purpose: Gets a value indicating whether the result represents failure.

QscResult.Success [property]

public static QscResult Success

Purpose: Gets a successful result value.

25.8 Common/QscVersion.cs (13 members)

Declaration source: QSCCS/Common/QscVersion.cs. A throwing operation and a status-returning Try variant are distinct contracts; private native handles remain implementation details.

QscVersion.Major [property]

public static uint Major

Purpose: Gets the major version of the loaded QSCABI bridge.

QscVersion.Minor [property]

public static uint Minor

Purpose: Gets the minor version of the loaded QSCABI bridge.

QscVersion.Patch [property]

public static uint Patch

Purpose: Gets the patch version of the loaded QSCABI bridge.

QscVersion.IsAbiCompatible [method]

public static bool IsAbiCompatible(uint major, uint minimumMinor)

Purpose: Gets whether the loaded native QSCABI implements the specified major and minimum minor version.

Parameters: major: Required ABI major version.; minimumMinor: Minimum minor version required.

Returns: True when the loaded native ABI is compatible.

QscVersion.Capabilities [property]

public static QscAbiCapabilities Capabilities

Purpose: Gets the features compiled into the native QSCABI library.

QscVersion.HasCapabilities [method]

public static bool HasCapabilities(QscAbiCapabilities required)

Purpose: Checks whether a complete set of compiled-feature flags is present.

Parameters: required: Features that must all be compiled in.

Returns: True when all requested bits are available.

QscVersion.AbiVersion [property]

public static string AbiVersion

Purpose: Gets the complete QSCABI version string.

QscVersion.QscVersionString [property]

public static string QscVersionString

Purpose: Gets the version string of the native QSC implementation linked into QSCABI.

QscVersion.BuildPlatform [property]

public static string BuildPlatform

Purpose: Gets the native platform string reported by QSCABI.

QscVersion.BuildCompiler [property]

public static string BuildCompiler

Purpose: Gets the compiler string reported by QSCABI.

QscVersion.EnabledFeatures [property]

public static string EnabledFeatures

Purpose: Gets the native feature string reported by QSCABI.

QscVersion.BuildBackend [property]

public static unsafe string BuildBackend

Purpose: Gets the loaded native QSCABI build backend identifier.

QscVersion.QscBuildProfile [property]

public static unsafe string QscBuildProfile

Purpose: Gets the linked QSC native build and intrinsic-selection profile.

25.9 Encoding/QscEncoding.cs (12 members)

Declaration source: QSCCS/Encoding/QscEncoding.cs. A throwing operation and a status-returning Try variant are distinct contracts; private native handles remain implementation details.

QscEncoding.HexEncode [method]

public static string HexEncode(ReadOnlySpan<byte> data)

Purpose: Encodes binary data as uppercase hexadecimal text.

Parameters: data: The caller-owned binary data to encode.

Returns: The uppercase hexadecimal representation of .

QscEncoding.TryHexEncode [method]

public static unsafe QscStatus TryHexEncode(ReadOnlySpan<byte> data, out string output)

Purpose: Attempts to encode binary data as uppercase hexadecimal text.

Parameters: data: The caller-owned binary data to encode.; output: Receives the encoded hexadecimal text when the operation succeeds.

Returns: QscStatus.Success on success; otherwise, a QSCABI status value.

QscEncoding.HexDecode [method]

public static byte[] HexDecode(string hex)

Purpose: Decodes hexadecimal text into binary data.

Parameters: hex: The hexadecimal text to decode.

Returns: The decoded binary data.

QscEncoding.TryHexDecode [method]

public static unsafe QscStatus TryHexDecode(string hex, out byte[] output)

Purpose: Attempts to decode hexadecimal text into binary data.

Parameters: hex: The hexadecimal text to decode.; output: Receives the decoded bytes when the operation succeeds.

Returns: QscStatus.Success on success; otherwise, a QSCABI status value.

QscEncoding.Base64Encode [method]

public static string Base64Encode(ReadOnlySpan<byte> data)

Purpose: Encodes binary data as RFC 4648 Base64 text.

Parameters: data: The caller-owned binary data to encode.

Returns: The Base64 representation of .

QscEncoding.TryBase64Encode [method]

public static unsafe QscStatus TryBase64Encode(ReadOnlySpan<byte> data, out string output)

Purpose: Attempts to encode binary data as RFC 4648 Base64 text.

Parameters: data: The caller-owned binary data to encode.; output: Receives the Base64 text when the operation succeeds.

Returns: QscStatus.Success on success; otherwise, a QSCABI status value.

QscEncoding.Base64Decode [method]

public static byte[] Base64Decode(string base64)

Purpose: Decodes RFC 4648 Base64 text into binary data.

Parameters: base64: The Base64 text to decode.

Returns: The decoded binary data.

QscEncoding.TryBase64Decode [method]

public static unsafe QscStatus TryBase64Decode(string base64, out byte[] output)

Purpose: Attempts to decode RFC 4648 Base64 text into binary data.

Parameters: base64: The Base64 text to decode.; output: Receives the decoded bytes when the operation succeeds.

Returns: QscStatus.Success on success; otherwise, a QSCABI status value.

QscEncoding.PemEncode [method]

public static string PemEncode(string label, ReadOnlySpan<byte> data)

Purpose: Encodes binary data as RFC 7468-style PEM armor using the supplied label.

Parameters: label: The PEM label to place in the BEGIN and END delimiters.; data: The caller-owned DER or binary payload to encode.

Returns: The complete PEM-armored text, including delimiters and line breaks.

QscEncoding.TryPemEncode [method]

public static unsafe QscStatus TryPemEncode(string label, ReadOnlySpan<byte> data, out string output)

Purpose: Attempts to encode binary data as RFC 7468-style PEM armor using the supplied label.

Parameters: label: The PEM label to place in the BEGIN and END delimiters.; data: The caller-owned DER or binary payload to encode.; output: Receives the PEM text when the operation succeeds.

Returns: QscStatus.Success on success; otherwise, a QSCABI status value.

QscEncoding.PemDecode [method]

public static byte[] PemDecode(string pem)

Purpose: Decodes RFC 7468-style PEM armor into binary data.

Parameters: pem: The complete PEM text to decode.

Returns: The decoded binary payload.

QscEncoding.TryPemDecode [method]

public static unsafe QscStatus TryPemDecode(string pem, out byte[] output)

Purpose: Attempts to decode RFC 7468-style PEM armor into binary data.

Parameters: pem: The complete PEM text to decode.; output: Receives the decoded payload bytes when the operation succeeds.

Returns: QscStatus.Success on success; otherwise, a QSCABI status value.

25.10 Entropy/QscEntropy.cs (14 members)

Declaration source: QSCCS/Entropy/QscEntropy.cs. A throwing operation and a status-returning Try variant are distinct contracts; private native handles remain implementation details.

QscEntropy.AcpSeedMax [property]

public static int AcpSeedMax

Purpose: Gets the maximum number of bytes accepted by a single ACP request.

QscEntropy.CspSeedMax [property]

public static int CspSeedMax

Purpose: Gets the maximum number of bytes accepted by a single CSP request.

QscEntropy.RdpSeedMax [property]

public static int RdpSeedMax

Purpose: Gets the maximum number of bytes accepted by a single RDP request.

QscEntropy.CjpSeedMax [property]

public static int CjpSeedMax

Purpose: Gets the maximum number of bytes accepted by a single CJP request.

QscEntropy.IsRdpAvailable [property]

public static bool IsRdpAvailable

Purpose: Gets a value indicating whether the hardware random-data provider is available on the current platform.

QscEntropy.IsCjpAvailable [property]

public static bool IsCjpAvailable

Purpose: Gets a value indicating whether the CPU-jitter provider is available on the current platform.

QscEntropy.GenerateAcp [method]

public static byte[] GenerateAcp(int length)

Purpose: Generates entropy using the QSC aggregate provider.

Parameters: length: The positive number of bytes to generate.

Returns: A newly allocated byte array containing generated bytes.

QscEntropy.GenerateCsp [method]

public static byte[] GenerateCsp(int length)

Purpose: Generates entropy using the operating-system cryptographic service provider.

Parameters: length: The positive number of bytes to generate.

Returns: A newly allocated byte array containing generated bytes.

QscEntropy.GenerateRdp [method]

public static byte[] GenerateRdp(int length)

Purpose: Generates entropy using the hardware random-data provider.

Parameters: length: The positive number of bytes to generate.

Returns: A newly allocated byte array containing generated bytes.

QscEntropy.GenerateCjp [method]

public static byte[] GenerateCjp(int length)

Purpose: Generates entropy using the CPU-jitter provider.

Parameters: length: The positive number of bytes to generate.

Returns: A newly allocated byte array containing generated bytes.

QscEntropy.TryGenerateAcp [method]

public static unsafe QscStatus TryGenerateAcp(Span<byte> output)

Purpose: Attempts to fill a caller-owned buffer using the QSC aggregate entropy provider.

Parameters: output: The caller-owned destination buffer. The buffer must contain at least one byte.

Returns: QscStatus.Success on success; otherwise a QSCABI status code describing the failure.

QscEntropy.TryGenerateCsp [method]

public static unsafe QscStatus TryGenerateCsp(Span<byte> output)

Purpose: Attempts to fill a caller-owned buffer using the operating-system cryptographic provider.

Parameters: output: The caller-owned destination buffer. The buffer must contain at least one byte.

Returns: QscStatus.Success on success; otherwise a QSCABI status code describing the failure.

QscEntropy.TryGenerateRdp [method]

public static unsafe QscStatus TryGenerateRdp(Span<byte> output)

Purpose: Attempts to fill a caller-owned buffer using the hardware random-data provider.

Parameters: output: The caller-owned destination buffer. The buffer must contain at least one byte.

Returns: QscStatus.Success on success; otherwise a status such as QscStatus.PlatformUnavailable when RDP is unavailable.

QscEntropy.TryGenerateCjp [method]

public static unsafe QscStatus TryGenerateCjp(Span<byte> output)

Purpose: Attempts to fill a caller-owned buffer using the CPU-jitter provider.

Parameters: output: The caller-owned destination buffer. The buffer must contain at least one byte.

Returns: QscStatus.Success on success; otherwise a status such as QscStatus.PlatformUnavailable when CJP is unavailable.

25.11 Explicit/QscExplicitOperations.cs (15 members)

Declaration source: QSCCS/Explicit/QscExplicitOperations.cs. A throwing operation and a status-returning Try variant are distinct contracts; private native handles remain implementation details.

QscExplicitKeyPair.PublicKey [property]

public byte[] PublicKey

Purpose: Native public key bytes in the algorithm's raw ABI representation.

QscExplicitKeyPair.PrivateKey [property]

public byte[] PrivateKey

Purpose: Native private key bytes; clear them when no longer needed.

QscExplicitKeyPair.Dispose [method]

public void Dispose()

Purpose: Zero the managed secret key; any copies held by the caller must also be erased.

QscExplicitKemOutput.Ciphertext [property]

public byte[] Ciphertext

Purpose: Ciphertext transmitted to the decapsulating party.

QscExplicitKemOutput.SharedSecret [property]

public byte[] SharedSecret

Purpose: Secret output; protect and clear once consumed.

QscExplicitKemOutput.Dispose [method]

public void Dispose()

Purpose: Erase the shared secret in place; caller owns other copies.

QscExplicitKem.unsafe [method]

public static unsafe (int PublicKey, int PrivateKey, int Ciphertext, int SharedSecret) GetSizes(QscKemAlgorithm algorithm)

Purpose: Read the selected parameter set's exact native lengths.

Parameters: algorithm: Native explicit KEM algorithm identifier.

Returns: Public key, private key, ciphertext, and shared-secret byte counts.

QscExplicitKem.Generate [method]

public static unsafe QscExplicitKeyPair Generate(QscKemAlgorithm algorithm)

Purpose: Generate a native entropy-backed keypair with explicit ownership.

Parameters: algorithm: Selected native KEM parameter set.

Returns: Owned raw public and private keys; dispose to clear the latter.

QscExplicitKem.Encapsulate [method]

public static unsafe QscExplicitKemOutput Encapsulate(QscKemAlgorithm algorithm, ReadOnlySpan<byte> publicKey)

Purpose: Encapsulate a fresh native shared secret for an exact-sized public key.

Parameters: algorithm: Explicit KEM selector matching the recipient public key.; publicKey: Borrowed raw recipient public bytes.

Returns: Ciphertext and owned mutable shared secret.

QscExplicitKem.Decapsulate [method]

public static unsafe byte[] Decapsulate(QscKemAlgorithm algorithm, ReadOnlySpan<byte> ciphertext, ReadOnlySpan<byte> privateKey)

Purpose: Recover a KEM shared secret from ciphertext with an exact native private key.

Parameters: algorithm: Recipient's matching native parameter set.; ciphertext: Borrowed ciphertext from encapsulation.; privateKey: Borrowed private key, never retained.

Returns: Caller-owned mutable shared-secret bytes; must be erased after use.

QscExplicitCurve.unsafe [method]

public static unsafe (int PublicKey, int PrivateKey, int Secret, int Seed) GetSizes(QscCurveAlgorithm algorithm)

Purpose: Get public, private, shared-secret and seed lengths for a named curve.

Parameters: algorithm: Native ECDH/EDDH curve selector.

Returns: Exact native lengths in bytes.

QscExplicitCurve.Generate [method]

public static unsafe QscExplicitKeyPair Generate(QscCurveAlgorithm algorithm)

Purpose: Generate native-compatible curve public/private keys.

Parameters: algorithm: Curve used to generate both keys.

Returns: Owned keypair; Dispose clears its private key.

QscExplicitCurve.GenerateSeeded [method]

public static unsafe QscExplicitKeyPair GenerateSeeded(QscCurveAlgorithm algorithm, ReadOnlySpan<byte> seed)

Purpose: Deterministically generate a keypair with an exact native seed.

Parameters: algorithm: Selected native curve.; seed: Exact-length caller-owned deterministic seed; not retained.

Returns: Generated keypair; Dispose zeros private-key material.

QscExplicitCurve.PublicFromPrivate [method]

public static unsafe byte[] PublicFromPrivate(QscCurveAlgorithm algorithm, ReadOnlySpan<byte> privateKey)

Purpose: Derive an unprefixed native public point from a raw private scalar.

Parameters: algorithm: Native curve selector.; privateKey: Exact native private-key bytes.

Returns: New managed public-key array.

QscExplicitCurve.Agree [method]

public static unsafe byte[] Agree(QscCurveAlgorithm algorithm, ReadOnlySpan<byte> peerPublicKey, ReadOnlySpan<byte> privateKey)

Purpose: Derive a raw ECDH/EDDH secret after exact peer/public key length checks.

Parameters: algorithm: Curve that matches both key encodings.; peerPublicKey: Peer's raw point (not PEM/DER or SEC1-prefixed).; privateKey: Local matching raw private scalar.

Returns: Caller-owned secret; erase once consumed.

25.12 Explicit/QscExplicitSignatures.cs (8 members)

Declaration source: QSCCS/Explicit/QscExplicitSignatures.cs. A throwing operation and a status-returning Try variant are distinct contracts; private native handles remain implementation details.

QscExplicitSignature.unsafe [method]

public static unsafe (int PublicKey, int PrivateKey, int SignatureMax, int Seed) GetSizes(QscAttachedSignatureAlgorithm algorithm)

Purpose: Query native key, signature-overhead, and seed byte lengths.

Parameters: algorithm: Named native signature scheme.

Returns: Public, private, maximum signature overhead and seed lengths.

QscExplicitSignature.Generate [method]

public static unsafe QscExplicitKeyPair Generate(QscAttachedSignatureAlgorithm algorithm)

Purpose: Generate a native explicit signing keypair.

Parameters: algorithm: Signing scheme; matches later sign/verify operations.

Returns: New public/private keys, private key erased on Dispose.

QscExplicitSignature.Sign [method]

public static unsafe byte[] Sign(QscAttachedSignatureAlgorithm algorithm, ReadOnlySpan<byte> message, ReadOnlySpan<byte> privateKey)

Purpose: Sign caller message in the native attached signed-message representation.

Parameters: algorithm: Exact signature algorithm used for key generation.; message: Nonempty borrowed message.; privateKey: Raw private key, borrowed until native return.

Returns: New native attached signature/message bytes.

QscExplicitSignature.Verify [method]

public static unsafe byte[] Verify(QscAttachedSignatureAlgorithm algorithm, ReadOnlySpan<byte> signedMessage, ReadOnlySpan<byte> publicKey)

Purpose: Verify an attached signature and recover the authenticated original message.

Parameters: algorithm: Scheme used to sign the original message.; signedMessage: Untrusted native attached-message bytes.; publicKey: Exact native public verification key.

Returns: New managed recovered message bytes on successful native verification.

QscExplicitSlhDsa.unsafe [method]

public static unsafe (int PublicKey, int PrivateKey, int Signature, int Seed) GetSizes(QscExplicitSlhDsaAlgorithm algorithm)

Purpose: Query exact SLH-DSA public/private/signature/seed lengths.

Parameters: algorithm: Native SHAKE SLH-DSA parameter set.

Returns: Exact native size values in bytes.

QscExplicitSlhDsa.Generate [method]

public static unsafe QscExplicitKeyPair Generate(QscExplicitSlhDsaAlgorithm algorithm)

Purpose: Generate a native entropy-backed SLH-DSA keypair.

Parameters: algorithm: Exact FIPS 205 SHAKE parameter set.

Returns: Owned raw keys; Dispose zeroes the private key.

QscExplicitSlhDsa.Sign [method]

public static unsafe byte[] Sign(QscExplicitSlhDsaAlgorithm algorithm, QscSlhDsaPrehash prehash, bool randomized, ReadOnlySpan<byte> message, ReadOnlySpan<byte> context, ReadOnlySpan<byte> privateKey)

Purpose: Sign a message with detached SLH-DSA pure/prehash and optional context.

Parameters: algorithm: SLH-DSA parameter set.; prehash: Pure or prehash identifier.; randomized: Whether to use native CSP signing randomness.; message: Borrowed message bytes.; context: Domain context, at most 255 bytes.; privateKey: Exact native secret key bytes.

Returns: New detached signature byte array.

QscExplicitSlhDsa.Verify [method]

public static unsafe QscStatus Verify(QscExplicitSlhDsaAlgorithm algorithm, QscSlhDsaPrehash prehash, ReadOnlySpan<byte> signature, ReadOnlySpan<byte> message, ReadOnlySpan<byte> context, ReadOnlySpan<byte> publicKey)

Purpose: Verify the exact detached SLH-DSA signature with matching context and prehash.

Parameters: algorithm: SLH-DSA parameter set.; prehash: Pure/prehash mode originally used.; signature: Detached signature bytes.; message: Signed message bytes.; context: Context bound to the signature, at most 255 bytes.; publicKey: Exact native verification public key.

Returns: Success only after native cryptographic verification.

25.13 Hardware/QscHardware.cs (24 members)

Declaration source: QSCCS/Hardware/QscHardware.cs. A throwing operation and a status-returning Try variant are distinct contracts; private native handles remain implementation details.

QscHardwareCallbackSet.Capabilities [property]

public CapabilitiesDelegate Capabilities

Purpose: Required capability reporter.

QscHardwareCallbackSet.ReleaseKey [property]

public ReleaseKeyDelegate ReleaseKey

Purpose: Required transient key release function.

QscHardwareCallbackSet.Open [property]

public OpenDelegate? Open

Purpose: Optional persistent key-open callback.

QscHardwareCallbackSet.Generate [property]

public GenerateDelegate? Generate

Purpose: Optional key-generation callback.

QscHardwareCallbackSet.Sign [property]

public SignDelegate? Sign

Purpose: Optional signing callback.

QscHardwareCallbackSet.Agree [property]

public BytesDelegate? Agree

Purpose: Optional key-agreement callback.

QscHardwareCallbackSet.Random [property]

public RandomDelegate? Random

Purpose: Optional entropy callback.

QscHardwareCallbackSet.Attest [property]

public BytesDelegate? Attest

Purpose: Optional attestation-evidence callback.

QscHardwareCallbackSet.PublicKey [property]

public PublicDelegate? PublicKey

Purpose: Optional public-key extraction callback.

QscHardwareCallbackSet.QscHardwareCallbackSet [constructor]

public QscHardwareCallbackSet(CapabilitiesDelegate capabilities, ReleaseKeyDelegate releaseKey)

Purpose: Initialize the required callback functions.

Parameters: capabilities: Mandatory versioned capability reporter.; releaseKey: Mandatory transient-key release operation.

QscHardwareProvider.IsEnabled [property]

public static bool IsEnabled

Purpose: Query whether the linked QSC build includes hardware callback support.

QscHardwareProvider.QscHardwareProvider [constructor]

public QscHardwareProvider(QscHardwareCallbackSet callbacks, nint context = 0)

Purpose: Open a callback facade on caller-managed backend state.

Parameters: callbacks: Managed callback implementations, retained until Dispose.; context: Borrowed backend pointer; caller keeps it valid while provider is open.

QscHardwareProvider.GetCapabilities [method]

public QscHardwareCapabilities GetCapabilities()

Purpose: Read a versioned capability snapshot from the registered provider.

Returns: Current backend flags, limits, and concurrency requirements.

QscHardwareProvider.OpenKey [method]

public QscHardwareKey OpenKey(string locator, QscHardwarePolicy policy)

Purpose: Open a persistent key locator through the backend.

Parameters: locator: Nonempty UTF-8 key locator, without NUL.; policy: Versioned QSC hardware policy, initialized by caller.

Returns: Owned key reference, released before provider close.

QscHardwareProvider.GenerateKey [method]

public QscHardwareKey GenerateKey(QscHardwarePolicy policy)

Purpose: Generate a native provider-backed key under an explicit custody policy.

Parameters: policy: Requested usage, algorithm and custody restrictions.

Returns: Owned transient handle, not a private-key export.

QscHardwareProvider.Random [method]

public byte[] Random(int length)

Purpose: Generate bytes using the configured backend entropy callback.

Parameters: length: Requested length, at most 64 MiB.

Returns: Owned byte array; hardware entropy certification is not implied.

QscHardwareProvider.Invalidate [method]

public void Invalidate()

Purpose: Invalidate registered provider keys after lifecycle or backend rotation.

QscHardwareProvider.Dispose [method]

public void Dispose()

Purpose: Close the native provider only after all owned keys have been released.

QscHardwareKey.GetInfo [method]

public QscHardwareKeyInfo GetInfo()

Purpose: Read native metadata about algorithm, custody, usage and extractability.

Returns: Versioned key information, not a device-custody attestation.

QscHardwareKey.GetPublicKey [method]

public byte[] GetPublicKey(int capacity = 16384)

Purpose: Copy public-key-only bytes into a fresh managed result.

Parameters: capacity: Maximum public-key bytes permitted by application policy.

Returns: Public key bytes only, never private-key material.

QscHardwareKey.Sign [method]

public byte[] Sign(ReadOnlySpan<byte> message, int capacity = 16384)

Purpose: Sign borrowed message bytes using the backend private key.

Parameters: message: Message input; not retained.; capacity: Maximum signature bytes.

Returns: Backend signature bytes on native success.

QscHardwareKey.Agree [method]

public byte[] Agree(ReadOnlySpan<byte> peerPublicKey, int capacity = 512)

Purpose: Derive key agreement output when the backend policy permits raw secret export.

Parameters: peerPublicKey: Peer raw public-key bytes.; capacity: Maximum resulting secret bytes.

Returns: Caller-owned secret that must be erased after use.

QscHardwareKey.Attest [method]

public byte[] Attest(ReadOnlySpan<byte> challenge, int capacity = 4096)

Purpose: Produce backend attestation evidence in response to a challenge.

Parameters: challenge: Caller-supplied unpredictable challenge.; capacity: Maximum attestation bytes.

Returns: Untrusted evidence to validate against an external attestation policy.

QscHardwareKey.Dispose [method]

public void Dispose()

Purpose: Release transient native key and decrement parent provider's reference count.

25.14 Hash/QscSha2.cs (9 members)

Declaration source: QSCCS/Hash/QscSha2.cs. A throwing operation and a status-returning Try variant are distinct contracts; private native handles remain implementation details.

QscSha2.Digest256Size [property]

public static int Digest256Size

Purpose: Gets the SHA2-256 digest size in bytes.

QscSha2.Digest384Size [property]

public static int Digest384Size

Purpose: Gets the SHA2-384 digest size in bytes.

QscSha2.Digest512Size [property]

public static int Digest512Size

Purpose: Gets the SHA2-512 digest size in bytes.

QscSha2.Compute256 [method]

public static byte[] Compute256(ReadOnlySpan<byte> message)

Purpose: Computes a SHA2-256 digest over a caller-owned message buffer.

Parameters: message: The message bytes to hash. The buffer is read during the call and is not retained.

Returns: A new byte array containing the SHA2-256 digest.

QscSha2.Compute384 [method]

public static byte[] Compute384(ReadOnlySpan<byte> message)

Purpose: Computes a SHA2-384 digest over a caller-owned message buffer.

Parameters: message: The message bytes to hash. The buffer is read during the call and is not retained.

Returns: A new byte array containing the SHA2-384 digest.

QscSha2.Compute512 [method]

public static byte[] Compute512(ReadOnlySpan<byte> message)

Purpose: Computes a SHA2-512 digest over a caller-owned message buffer.

Parameters: message: The message bytes to hash. The buffer is read during the call and is not retained.

Returns: A new byte array containing the SHA2-512 digest.

QscSha2.TryCompute256 [method]

public static unsafe QscStatus TryCompute256(ReadOnlySpan<byte> message, Span<byte> digest)

Purpose: Attempts to compute a SHA2-256 digest into a caller-owned output buffer.

Parameters: message: The message bytes to hash. The buffer is read during the call and is not retained.; digest: The caller-owned output buffer. It must be at least Digest256Size bytes long.

Returns: QscStatus.Success on success, QscStatus.BufferTooSmall for undersized output, or another QSCABI status.

QscSha2.TryCompute384 [method]

public static unsafe QscStatus TryCompute384(ReadOnlySpan<byte> message, Span<byte> digest)

Purpose: Attempts to compute a SHA2-384 digest into a caller-owned output buffer.

Parameters: message: The message bytes to hash. The buffer is read during the call and is not retained.; digest: The caller-owned output buffer. It must be at least Digest384Size bytes long.

Returns: QscStatus.Success on success, QscStatus.BufferTooSmall for undersized output, or another QSCABI status.

QscSha2.TryCompute512 [method]

public static unsafe QscStatus TryCompute512(ReadOnlySpan<byte> message, Span<byte> digest)

Purpose: Attempts to compute a SHA2-512 digest into a caller-owned output buffer.

Parameters: message: The message bytes to hash. The buffer is read during the call and is not retained.; digest: The caller-owned output buffer. It must be at least Digest512Size bytes long.

Returns: QscStatus.Success on success, QscStatus.BufferTooSmall for undersized output, or another QSCABI status.

25.15 Hash/QscSha3.cs (9 members)

Declaration source: QSCCS/Hash/QscSha3.cs. A throwing operation and a status-returning Try variant are distinct contracts; private native handles remain implementation details.

QscSha3.Digest128Size [property]

public static int Digest128Size

Purpose: Gets the SHA3-128 digest size in bytes.

QscSha3.Digest256Size [property]

public static int Digest256Size

Purpose: Gets the SHA3-256 digest size in bytes.

QscSha3.Digest512Size [property]

public static int Digest512Size

Purpose: Gets the SHA3-512 digest size in bytes.

QscSha3.Compute128 [method]

public static byte[] Compute128(ReadOnlySpan<byte> message)

Purpose: Computes a SHA3-128 digest over a caller-owned message buffer.

Parameters: message: The message bytes to hash. The buffer is read during the call and is not retained.

Returns: A new byte array containing the SHA3-128 digest.

QscSha3.Compute256 [method]

public static byte[] Compute256(ReadOnlySpan<byte> message)

Purpose: Computes a SHA3-256 digest over a caller-owned message buffer.

Parameters: message: The message bytes to hash. The buffer is read during the call and is not retained.

Returns: A new byte array containing the SHA3-256 digest.

QscSha3.Compute512 [method]

public static byte[] Compute512(ReadOnlySpan<byte> message)

Purpose: Computes a SHA3-512 digest over a caller-owned message buffer.

Parameters: message: The message bytes to hash. The buffer is read during the call and is not retained.

Returns: A new byte array containing the SHA3-512 digest.

QscSha3.TryCompute128 [method]

public static unsafe QscStatus TryCompute128(ReadOnlySpan<byte> message, Span<byte> digest)

Purpose: Attempts to compute a SHA3-128 digest into a caller-owned output buffer.

Parameters: message: The message bytes to hash. The buffer is read during the call and is not retained.; digest: The caller-owned output buffer. It must be at least Digest128Size bytes long.

Returns: QscStatus.Success on success, QscStatus.BufferTooSmall for undersized output, or another QSCABI status.

QscSha3.TryCompute256 [method]

public static unsafe QscStatus TryCompute256(ReadOnlySpan<byte> message, Span<byte> digest)

Purpose: Attempts to compute a SHA3-256 digest into a caller-owned output buffer.

Parameters: message: The message bytes to hash. The buffer is read during the call and is not retained.; digest: The caller-owned output buffer. It must be at least Digest256Size bytes long.

Returns: QscStatus.Success on success, QscStatus.BufferTooSmall for undersized output, or another QSCABI status.

QscSha3.TryCompute512 [method]

public static unsafe QscStatus TryCompute512(ReadOnlySpan<byte> message, Span<byte> digest)

Purpose: Attempts to compute a SHA3-512 digest into a caller-owned output buffer.

Parameters: message: The message bytes to hash. The buffer is read during the call and is not retained.; digest: The caller-owned output buffer. It must be at least Digest512Size bytes long.

Returns: QscStatus.Success on success, QscStatus.BufferTooSmall for undersized output, or another QSCABI status.

25.16 Kdf/QscHkdf.cs (15 members)

Declaration source: QSCCS/Kdf/QscHkdf.cs. A throwing operation and a status-returning Try variant are distinct contracts; private native handles remain implementation details.

QscHkdf.Sha2256PrkSize [property]

public static int Sha2256PrkSize

Purpose: Gets the Sha2256PrkSize value reported by QSCABI.

QscHkdf.Sha2384PrkSize [property]

public static int Sha2384PrkSize

Purpose: Gets the Sha2384PrkSize value reported by QSCABI.

QscHkdf.Sha2512PrkSize [property]

public static int Sha2512PrkSize

Purpose: Gets the Sha2512PrkSize value reported by QSCABI.

QscHkdf.ExtractSha2256 [method]

public static byte[] ExtractSha2256(ReadOnlySpan<byte> key, ReadOnlySpan<byte> salt)

Purpose: Runs HKDF-SHA2-256 extract and returns the pseudorandom key.

Parameters: key: The cryptographic key material. The buffer is not retained.; salt: The salt bytes used by the derivation or password-based operation.

Returns: A newly allocated pseudorandom key buffer.

QscHkdf.ExtractSha2384 [method]

public static byte[] ExtractSha2384(ReadOnlySpan<byte> key, ReadOnlySpan<byte> salt)

Purpose: Runs HKDF-SHA2-384 extract and returns the pseudorandom key.

Parameters: key: The cryptographic key material. The buffer is not retained.; salt: The salt bytes used by the derivation or password-based operation.

Returns: A newly allocated pseudorandom key buffer.

QscHkdf.ExtractSha2512 [method]

public static byte[] ExtractSha2512(ReadOnlySpan<byte> key, ReadOnlySpan<byte> salt)

Purpose: Runs HKDF-SHA2-512 extract and returns the pseudorandom key.

Parameters: key: The cryptographic key material. The buffer is not retained.; salt: The salt bytes used by the derivation or password-based operation.

Returns: A newly allocated pseudorandom key buffer.

QscHkdf.ExpandSha2256 [method]

public static byte[] ExpandSha2256(ReadOnlySpan<byte> prk, ReadOnlySpan<byte> info, int outputLength)

Purpose: Runs HKDF-SHA2-256 expand and returns output keying material.

Parameters: prk: The pseudorandom key produced by HKDF extract.; info: The optional context or domain-separation information.; outputLength: The operation parameter.

Returns: A newly allocated output-keying-material buffer.

QscHkdf.ExpandSha2384 [method]

public static byte[] ExpandSha2384(ReadOnlySpan<byte> prk, ReadOnlySpan<byte> info, int outputLength)

Purpose: Runs HKDF-SHA2-384 expand and returns output keying material.

Parameters: prk: The pseudorandom key produced by HKDF extract.; info: The optional context or domain-separation information.; outputLength: The operation parameter.

Returns: A newly allocated output-keying-material buffer.

QscHkdf.ExpandSha2512 [method]

public static byte[] ExpandSha2512(ReadOnlySpan<byte> prk, ReadOnlySpan<byte> info, int outputLength)

Purpose: Runs HKDF-SHA2-512 expand and returns output keying material.

Parameters: prk: The pseudorandom key produced by HKDF extract.; info: The optional context or domain-separation information.; outputLength: The operation parameter.

Returns: A newly allocated output-keying-material buffer.

QscHkdf.TryExtractSha2256 [method]

public static unsafe QscStatus TryExtractSha2256(ReadOnlySpan<byte> key, ReadOnlySpan<byte> salt, Span<byte> prk)

Purpose: Attempts to run HKDF-SHA2-256 extract into a caller-provided PRK buffer.

Parameters: key: The cryptographic key material. The buffer is not retained.; salt: The salt bytes used by the derivation or password-based operation.; prk: The pseudorandom key produced by HKDF extract.

Returns: QscStatus.Success on success, or a non-success status when validation, authentication, or native execution fails.

QscHkdf.TryExtractSha2384 [method]

public static unsafe QscStatus TryExtractSha2384(ReadOnlySpan<byte> key, ReadOnlySpan<byte> salt, Span<byte> prk)

Purpose: Attempts to run HKDF-SHA2-384 extract into a caller-provided PRK buffer.

Parameters: key: The cryptographic key material. The buffer is not retained.; salt: The salt bytes used by the derivation or password-based operation.; prk: The pseudorandom key produced by HKDF extract.

Returns: QscStatus.Success on success, or a non-success status when validation, authentication, or native execution fails.

QscHkdf.TryExtractSha2512 [method]

public static unsafe QscStatus TryExtractSha2512(ReadOnlySpan<byte> key, ReadOnlySpan<byte> salt, Span<byte> prk)

Purpose: Attempts to run HKDF-SHA2-512 extract into a caller-provided PRK buffer.

Parameters: key: The cryptographic key material. The buffer is not retained.; salt: The salt bytes used by the derivation or password-based operation.; prk: The pseudorandom key produced by HKDF extract.

Returns: QscStatus.Success on success, or a non-success status when validation, authentication, or native execution fails.

QscHkdf.TryExpandSha2256 [method]

public static unsafe QscStatus TryExpandSha2256(ReadOnlySpan<byte> prk, ReadOnlySpan<byte> info, Span<byte> output)

Purpose: Attempts to run HKDF-SHA2-256 expand into a caller-provided output buffer.

Parameters: prk: The pseudorandom key produced by HKDF extract.; info: The optional context or domain-separation information.; output: The caller-owned output buffer.

Returns: QscStatus.Success on success, or a non-success status when validation, authentication, or native execution fails.

QscHkdf.TryExpandSha2384 [method]

public static unsafe QscStatus TryExpandSha2384(ReadOnlySpan<byte> prk, ReadOnlySpan<byte> info, Span<byte> output)

Purpose: Attempts to run HKDF-SHA2-384 expand into a caller-provided output buffer.

Parameters: prk: The pseudorandom key produced by HKDF extract.; info: The optional context or domain-separation information.; output: The caller-owned output buffer.

Returns: QscStatus.Success on success, or a non-success status when validation, authentication, or native execution fails.

QscHkdf.TryExpandSha2512 [method]

public static unsafe QscStatus TryExpandSha2512(ReadOnlySpan<byte> prk, ReadOnlySpan<byte> info, Span<byte> output)

Purpose: Attempts to run HKDF-SHA2-512 expand into a caller-provided output buffer.

Parameters: prk: The pseudorandom key produced by HKDF extract.; info: The optional context or domain-separation information.; output: The caller-owned output buffer.

Returns: QscStatus.Success on success, or a non-success status when validation, authentication, or native execution fails.

25.17 Kdf/QscPbmac1Kdf.cs (4 members)

Declaration source: QSCCS/Kdf/QscPbmac1Kdf.cs. A throwing operation and a status-returning Try variant are distinct contracts; private native handles remain implementation details.

QscPbmac1Kdf.MaxKeySize [property]

public static int MaxKeySize

Purpose: Gets the MaxKeySize value reported by QSCABI.

QscPbmac1Kdf.MaxIterations [property]

public static ulong MaxIterations

Purpose: Gets the MaxIterations value reported by QSCABI.

QscPbmac1Kdf.DeriveKey [method]

public static byte[] DeriveKey(QscPbmac1Hash hash, ReadOnlySpan<byte> password, ReadOnlySpan<byte> salt, ulong iterations, int keyLength)

Purpose: Derives PBMAC1 key material and returns it in a newly allocated buffer.

Parameters: hash: The PBMAC1 hash profile.; password: The password bytes used by the password-based operation.; salt: The salt bytes used by the derivation or password-based operation.; iterations: The PBKDF/PBMAC1 iteration count.; keyLength: The derived key length, in bytes.

Returns: A newly allocated derived-key buffer.

QscPbmac1Kdf.TryDeriveKey [method]

public static unsafe QscStatus TryDeriveKey(QscPbmac1Hash hash, ReadOnlySpan<byte> password, ReadOnlySpan<byte> salt, ulong iterations, Span<byte> key)

Purpose: Attempts to derive PBMAC1 key material into a caller-provided buffer.

Parameters: hash: The PBMAC1 hash profile.; password: The password bytes used by the password-based operation.; salt: The salt bytes used by the derivation or password-based operation.; iterations: The PBKDF/PBMAC1 iteration count.; key: The cryptographic key material. The buffer is not retained.

Returns: QscStatus.Success on success, or a non-success status when validation, authentication, or native execution fails.

25.18 Mac/QscHmac.cs (9 members)

Declaration source: QSCCS/Mac/QscHmac.cs. A throwing operation and a status-returning Try variant are distinct contracts; private native handles remain implementation details.

QscHmac.Sha2256MacSize [property]

public static int Sha2256MacSize

Purpose: Gets the HMAC-SHA2-256 authentication-code size, in bytes.

QscHmac.Sha2384MacSize [property]

public static int Sha2384MacSize

Purpose: Gets the HMAC-SHA2-384 authentication-code size, in bytes.

QscHmac.Sha2512MacSize [property]

public static int Sha2512MacSize

Purpose: Gets the HMAC-SHA2-512 authentication-code size, in bytes.

QscHmac.ComputeSha2256 [method]

public static byte[] ComputeSha2256(ReadOnlySpan<byte> message, ReadOnlySpan<byte> key)

Purpose: Computes an HMAC-SHA2-256 authentication code over a message.

Parameters: message: The message bytes to authenticate. The buffer is not retained.; key: The HMAC key material. The buffer is not retained.

Returns: A newly allocated authentication-code buffer of Sha2256MacSize bytes.

QscHmac.ComputeSha2384 [method]

public static byte[] ComputeSha2384(ReadOnlySpan<byte> message, ReadOnlySpan<byte> key)

Purpose: Computes an HMAC-SHA2-384 authentication code over a message.

Parameters: message: The message bytes to authenticate. The buffer is not retained.; key: The HMAC key material. The buffer is not retained.

Returns: A newly allocated authentication-code buffer of Sha2384MacSize bytes.

QscHmac.ComputeSha2512 [method]

public static byte[] ComputeSha2512(ReadOnlySpan<byte> message, ReadOnlySpan<byte> key)

Purpose: Computes an HMAC-SHA2-512 authentication code over a message.

Parameters: message: The message bytes to authenticate. The buffer is not retained.; key: The HMAC key material. The buffer is not retained.

Returns: A newly allocated authentication-code buffer of Sha2512MacSize bytes.

QscHmac.TryComputeSha2256 [method]

public static unsafe QscStatus TryComputeSha2256(ReadOnlySpan<byte> message, ReadOnlySpan<byte> key, Span<byte> mac)

Purpose: Attempts to compute an HMAC-SHA2-256 authentication code into a caller-provided buffer.

Parameters: message: The message bytes to authenticate.; key: The HMAC key material.; mac: The caller-owned output buffer. It must be at least Sha2256MacSize bytes.

Returns: QscStatus.Success on success, or a non-success status on validation or native failure.

QscHmac.TryComputeSha2384 [method]

public static unsafe QscStatus TryComputeSha2384(ReadOnlySpan<byte> message, ReadOnlySpan<byte> key, Span<byte> mac)

Purpose: Attempts to compute an HMAC-SHA2-384 authentication code into a caller-provided buffer.

Parameters: message: The message bytes to authenticate.; key: The HMAC key material.; mac: The caller-owned output buffer. It must be at least Sha2384MacSize bytes.

Returns: QscStatus.Success on success, or a non-success status on validation or native failure.

QscHmac.TryComputeSha2512 [method]

public static unsafe QscStatus TryComputeSha2512(ReadOnlySpan<byte> message, ReadOnlySpan<byte> key, Span<byte> mac)

Purpose: Attempts to compute an HMAC-SHA2-512 authentication code into a caller-provided buffer.

Parameters: message: The message bytes to authenticate.; key: The HMAC key material.; mac: The caller-owned output buffer. It must be at least Sha2512MacSize bytes.

Returns: QscStatus.Success on success, or a non-success status on validation or native failure.

25.19 Mac/QscKmac.cs (12 members)

Declaration source: QSCCS/Mac/QscKmac.cs. A throwing operation and a status-returning Try variant are distinct contracts; private native handles remain implementation details.

QscKmac.Compute128 [method]

public static byte[] Compute128(ReadOnlySpan<byte> message, ReadOnlySpan<byte> key, ReadOnlySpan<byte> custom, int outputLength = 32)

Purpose: Computes a fixed-output KMAC-128 code.

Parameters: message: The message bytes to authenticate.; key: The KMAC key material.; custom: Optional customization bytes used for domain separation.; outputLength: The requested output length, in bytes.

Returns: A newly allocated KMAC output buffer.

QscKmac.Compute256 [method]

public static byte[] Compute256(ReadOnlySpan<byte> message, ReadOnlySpan<byte> key, ReadOnlySpan<byte> custom, int outputLength = 64)

Purpose: Computes a fixed-output KMAC-256 code.

Parameters: message: The message bytes to authenticate.; key: The KMAC key material.; custom: Optional customization bytes used for domain separation.; outputLength: The requested output length, in bytes.

Returns: A newly allocated KMAC output buffer.

QscKmac.Compute512 [method]

public static byte[] Compute512(ReadOnlySpan<byte> message, ReadOnlySpan<byte> key, ReadOnlySpan<byte> custom, int outputLength = 64)

Purpose: Computes a fixed-output KMAC-512 code.

Parameters: message: The message bytes to authenticate.; key: The KMAC key material.; custom: Optional customization bytes used for domain separation.; outputLength: The requested output length, in bytes.

Returns: A newly allocated KMAC output buffer.

QscKmac.ComputeXof128 [method]

public static byte[] ComputeXof128(ReadOnlySpan<byte> message, ReadOnlySpan<byte> key, ReadOnlySpan<byte> custom, int outputLength)

Purpose: Computes variable-length KMAC-XOF-128 output.

Parameters: message: The message bytes to authenticate or derive from.; key: The KMAC key material.; custom: Optional customization bytes used for domain separation.; outputLength: The requested output length, in bytes. It must be positive.

Returns: A newly allocated output buffer.

QscKmac.ComputeXof256 [method]

public static byte[] ComputeXof256(ReadOnlySpan<byte> message, ReadOnlySpan<byte> key, ReadOnlySpan<byte> custom, int outputLength)

Purpose: Computes variable-length KMAC-XOF-256 output.

Parameters: message: The message bytes to authenticate or derive from.; key: The KMAC key material.; custom: Optional customization bytes used for domain separation.; outputLength: The requested output length, in bytes. It must be positive.

Returns: A newly allocated output buffer.

QscKmac.ComputeXof512 [method]

public static byte[] ComputeXof512(ReadOnlySpan<byte> message, ReadOnlySpan<byte> key, ReadOnlySpan<byte> custom, int outputLength)

Purpose: Computes variable-length KMAC-XOF-512 output.

Parameters: message: The message bytes to authenticate or derive from.; key: The KMAC key material.; custom: Optional customization bytes used for domain separation.; outputLength: The requested output length, in bytes. It must be positive.

Returns: A newly allocated output buffer.

QscKmac.TryCompute128 [method]

public static unsafe QscStatus TryCompute128(ReadOnlySpan<byte> message, ReadOnlySpan<byte> key, ReadOnlySpan<byte> custom, Span<byte> mac)

Purpose: Attempts to compute fixed-output KMAC-128 into a caller-provided buffer.

Parameters: message: The message bytes.; key: The KMAC key material.; custom: Optional customization bytes.; mac: The caller-owned output buffer.

Returns: QscStatus.Success on success, otherwise a non-success status.

QscKmac.TryCompute256 [method]

public static unsafe QscStatus TryCompute256(ReadOnlySpan<byte> message, ReadOnlySpan<byte> key, ReadOnlySpan<byte> custom, Span<byte> mac)

Purpose: Attempts to compute fixed-output KMAC-256 into a caller-provided buffer.

Parameters: message: The message bytes.; key: The KMAC key material.; custom: Optional customization bytes.; mac: The caller-owned output buffer.

Returns: QscStatus.Success on success, otherwise a non-success status.

QscKmac.TryCompute512 [method]

public static unsafe QscStatus TryCompute512(ReadOnlySpan<byte> message, ReadOnlySpan<byte> key, ReadOnlySpan<byte> custom, Span<byte> mac)

Purpose: Attempts to compute fixed-output KMAC-512 into a caller-provided buffer.

Parameters: message: The message bytes.; key: The KMAC key material.; custom: Optional customization bytes.; mac: The caller-owned output buffer.

Returns: QscStatus.Success on success, otherwise a non-success status.

QscKmac.TryComputeXof128 [method]

public static unsafe QscStatus TryComputeXof128(ReadOnlySpan<byte> message, ReadOnlySpan<byte> key, ReadOnlySpan<byte> custom, Span<byte> output)

Purpose: Attempts to compute KMAC-XOF-128 output into a caller-provided buffer.

Parameters: message: The message bytes.; key: The KMAC key material.; custom: Optional customization bytes.; output: The caller-owned output buffer. It must not be empty.

Returns: QscStatus.Success on success, otherwise a non-success status.

QscKmac.TryComputeXof256 [method]

public static unsafe QscStatus TryComputeXof256(ReadOnlySpan<byte> message, ReadOnlySpan<byte> key, ReadOnlySpan<byte> custom, Span<byte> output)

Purpose: Attempts to compute KMAC-XOF-256 output into a caller-provided buffer.

Parameters: message: The message bytes.; key: The KMAC key material.; custom: Optional customization bytes.; output: The caller-owned output buffer. It must not be empty.

Returns: QscStatus.Success on success, otherwise a non-success status.

QscKmac.TryComputeXof512 [method]

public static unsafe QscStatus TryComputeXof512(ReadOnlySpan<byte> message, ReadOnlySpan<byte> key, ReadOnlySpan<byte> custom, Span<byte> output)

Purpose: Attempts to compute KMAC-XOF-512 output into a caller-provided buffer.

Parameters: message: The message bytes.; key: The KMAC key material.; custom: Optional customization bytes.; output: The caller-owned output buffer. It must not be empty.

Returns: QscStatus.Success on success, otherwise a non-success status.

25.20 Mac/QscPbmac1.cs (5 members)

Declaration source: QSCCS/Mac/QscPbmac1.cs. A throwing operation and a status-returning Try variant are distinct contracts; private native handles remain implementation details.

QscPbmac1.MacSize [method]

public static int MacSize(QscPbmac1Hash hash)

Purpose: Returns the PBMAC1 MAC size for the selected hash profile.

Parameters: hash: The PBMAC1 hash profile.

Returns: The requested size in bytes.

QscPbmac1.Compute [method]

public static byte[] Compute(QscPbmac1Hash hash, ReadOnlySpan<byte> password, ReadOnlySpan<byte> salt, ulong iterations, int keyLength, ReadOnlySpan<byte> message)

Purpose: Computes a PBMAC1 authentication code and returns a newly allocated MAC buffer.

Parameters: hash: The PBMAC1 hash profile.; password: The password bytes used by the password-based operation.; salt: The salt bytes used by the derivation or password-based operation.; iterations: The PBKDF/PBMAC1 iteration count.; keyLength: The derived key length, in bytes.; message: The message bytes to process. The buffer is not retained.

Returns: A newly allocated buffer containing the computed value.

QscPbmac1.TryCompute [method]

public static unsafe QscStatus TryCompute(QscPbmac1Hash hash, ReadOnlySpan<byte> password, ReadOnlySpan<byte> salt, ulong iterations, int keyLength, ReadOnlySpan<byte> message, Span<byte> mac)

Purpose: Attempts to compute a PBMAC1 authentication code into a caller-provided buffer.

Parameters: hash: The PBMAC1 hash profile.; password: The password bytes used by the password-based operation.; salt: The salt bytes used by the derivation or password-based operation.; iterations: The PBKDF/PBMAC1 iteration count.; keyLength: The derived key length, in bytes.; message: The message bytes to process. The buffer is not retained.; mac: The caller-owned MAC output buffer.

Returns: QscStatus.Success on success, or a non-success status when validation, authentication, or native execution fails.

QscPbmac1.Verify [method]

public static bool Verify(ReadOnlySpan<byte> code, QscPbmac1Hash hash, ReadOnlySpan<byte> password, ReadOnlySpan<byte> salt, ulong iterations, int keyLength, ReadOnlySpan<byte> message)

Purpose: Verifies a PBMAC1 authentication code against the supplied message and parameters.

Parameters: code: The authentication code to verify.; hash: The PBMAC1 hash profile.; password: The password bytes used by the password-based operation.; salt: The salt bytes used by the derivation or password-based operation.; iterations: The PBKDF/PBMAC1 iteration count.; keyLength: The derived key length, in bytes.; message: The message bytes to process. The buffer is not retained.

Returns: when verification succeeds; otherwise, .

QscPbmac1.TryVerify [method]

public static unsafe QscStatus TryVerify(ReadOnlySpan<byte> code, QscPbmac1Hash hash, ReadOnlySpan<byte> password, ReadOnlySpan<byte> salt, ulong iterations, int keyLength, ReadOnlySpan<byte> message)

Purpose: Attempts to verify a PBMAC1 authentication code using status-code based error handling.

Parameters: code: The authentication code to verify.; hash: The PBMAC1 hash profile.; password: The password bytes used by the password-based operation.; salt: The salt bytes used by the derivation or password-based operation.; iterations: The PBKDF/PBMAC1 iteration count.; keyLength: The derived key length, in bytes.; message: The message bytes to process. The buffer is not retained.

Returns: QscStatus.Success on success, or a non-success status when validation, authentication, or native execution fails.

25.21 Mac/QscPoly1305.cs (6 members)

Declaration source: QSCCS/Mac/QscPoly1305.cs. A throwing operation and a status-returning Try variant are distinct contracts; private native handles remain implementation details.

QscPoly1305.KeySize [property]

public static int KeySize

Purpose: Gets the KeySize value reported by QSCABI.

QscPoly1305.MacSize [property]

public static int MacSize

Purpose: Gets the MacSize value reported by QSCABI.

QscPoly1305.Compute [method]

public static byte[] Compute(ReadOnlySpan<byte> message, ReadOnlySpan<byte> key)

Purpose: Computes a Poly1305 authentication code and returns it in a newly allocated buffer.

Parameters: message: The message bytes to process. The buffer is not retained.; key: The cryptographic key material. The buffer is not retained.

Returns: A newly allocated buffer containing the computed value.

QscPoly1305.TryCompute [method]

public static unsafe QscStatus TryCompute(ReadOnlySpan<byte> message, ReadOnlySpan<byte> key, Span<byte> mac)

Purpose: Attempts to compute a Poly1305 authentication code into a caller-provided buffer.

Parameters: message: The message bytes to process. The buffer is not retained.; key: The cryptographic key material. The buffer is not retained.; mac: The caller-owned MAC output buffer.

Returns: QscStatus.Success on success, or a non-success status when validation, authentication, or native execution fails.

QscPoly1305.Verify [method]

public static bool Verify(ReadOnlySpan<byte> code, ReadOnlySpan<byte> message, ReadOnlySpan<byte> key)

Purpose: Verifies a Poly1305 authentication code against the supplied message and key.

Parameters: code: The authentication code to verify.; message: The message bytes to process. The buffer is not retained.; key: The cryptographic key material. The buffer is not retained.

Returns: when verification succeeds; otherwise, .

QscPoly1305.TryVerify [method]

public static unsafe QscStatus TryVerify(ReadOnlySpan<byte> code, ReadOnlySpan<byte> message, ReadOnlySpan<byte> key)

Purpose: Attempts to verify a Poly1305 authentication code using status-code based error handling.

Parameters: code: The authentication code to verify.; message: The message bytes to process. The buffer is not retained.; key: The cryptographic key material. The buffer is not retained.

Returns: QscStatus.Success on success, or a non-success status when validation, authentication, or native execution fails.

25.22 Mac/QscQmac.cs (5 members)

Declaration source: QSCCS/Mac/QscQmac.cs. A throwing operation and a status-returning Try variant are distinct contracts; private native handles remain implementation details.

QscQmac.KeySize [property]

public static int KeySize

Purpose: Gets the KeySize value reported by QSCABI.

QscQmac.NonceSize [property]

public static int NonceSize

Purpose: Gets the NonceSize value reported by QSCABI.

QscQmac.MacSize [property]

public static int MacSize

Purpose: Gets the MacSize value reported by QSCABI.

QscQmac.Compute [method]

public static byte[] Compute(ReadOnlySpan<byte> message, ReadOnlySpan<byte> key, ReadOnlySpan<byte> nonce, ReadOnlySpan<byte> info)

Purpose: Computes a QMAC authentication code and returns it in a newly allocated buffer.

Parameters: message: The message bytes to process. The buffer is not retained.; key: The cryptographic key material. The buffer is not retained.; nonce: The nonce value. Nonces must satisfy the uniqueness requirements of the selected primitive.; info: The optional context or domain-separation information.

Returns: A newly allocated buffer containing the computed value.

QscQmac.TryCompute [method]

public static unsafe QscStatus TryCompute(ReadOnlySpan<byte> message, ReadOnlySpan<byte> key, ReadOnlySpan<byte> nonce, ReadOnlySpan<byte> info, Span<byte> mac)

Purpose: Attempts to compute a QMAC authentication code into a caller-provided buffer.

Parameters: message: The message bytes to process. The buffer is not retained.; key: The cryptographic key material. The buffer is not retained.; nonce: The nonce value. Nonces must satisfy the uniqueness requirements of the selected primitive.; info: The optional context or domain-separation information.; mac: The caller-owned MAC output buffer.

Returns: QscStatus.Success on success, or a non-success status when validation, authentication, or native execution fails.

25.23 Pqc/QscFalcon.cs (10 members)

Declaration source: QSCCS/Pqc/QscFalcon.cs. A throwing operation and a status-returning Try variant are distinct contracts; private native handles remain implementation details.

QscFalcon.PublicKeySize [property]

public static int PublicKeySize

Purpose: Gets the public-key size, in bytes, required by the native QSCABI algorithm profile.

QscFalcon.PrivateKeySize [property]

public static int PrivateKeySize

Purpose: Gets the private-key size, in bytes, required by the native QSCABI algorithm profile.

QscFalcon.SignatureSize [property]

public static int SignatureSize

Purpose: Gets the detached signature size, in bytes, for the native signature profile.

QscFalcon.SignedMessageSize [method]

public static int SignedMessageSize(int messageLength)

Purpose: Calculates the signed-message buffer size required for a message of the supplied length.

Parameters: messageLength: The non-negative message length, in bytes.

Returns: The required signed-message buffer size, in bytes.

QscFalcon.GenerateKeyPair [method]

public static QscPqcKeyPair GenerateKeyPair()

Purpose: Generates a new key pair using QSCABI randomness.

Returns: A managed key-pair object containing newly allocated public and private key arrays.

QscFalcon.Sign [method]

public static byte[] Sign(ReadOnlySpan<byte> message, ReadOnlySpan<byte> privateKey)

Purpose: Signs a message using the supplied private key.

Parameters: message: The message bytes to sign. The buffer is read during the call and is not retained.; privateKey: The private signing key. The length must equal PrivateKeySize.

Returns: A signed-message buffer containing the signature and recoverable message representation produced by QSCABI.

QscFalcon.Verify [method]

public static byte[] Verify(ReadOnlySpan<byte> signedMessage, ReadOnlySpan<byte> publicKey)

Purpose: Verifies a signed message and recovers the authenticated message bytes.

Parameters: signedMessage: The signed-message buffer returned by Sign(ReadOnlySpan{byte}, ReadOnlySpan{byte}).; publicKey: The public verification key. The length must equal PublicKeySize.

Returns: The recovered message bytes when verification succeeds.

QscFalcon.TryGenerateKeyPair [method]

public static unsafe QscStatus TryGenerateKeyPair(Span<byte> publicKey, Span<byte> privateKey)

Purpose: Attempts to generate a key pair into caller-owned buffers.

Parameters: publicKey: The destination buffer for the public key. The buffer must be at least PublicKeySize bytes.; privateKey: The destination buffer for the private key. The buffer must be at least PrivateKeySize bytes.

Returns: QscStatus.Success on success; otherwise a QSCABI status code describing invalid sizes, insufficient output space, verification failure, or platform/algorithm unavailability.

QscFalcon.TrySign [method]

public static unsafe QscStatus TrySign(ReadOnlySpan<byte> message, ReadOnlySpan<byte> privateKey, Span<byte> signedMessage, out int written)

Purpose: Attempts to sign a message into a caller-owned signed-message buffer.

Parameters: message: The message bytes to sign.; privateKey: The private signing key. The length must equal PrivateKeySize.; signedMessage: The destination buffer for the signed-message representation.; written: Receives the number of bytes written to .

Returns: QscStatus.Success on success; otherwise a QSCABI status code describing invalid sizes, insufficient output space, verification failure, or platform/algorithm unavailability.

QscFalcon.TryVerify [method]

public static unsafe QscStatus TryVerify(ReadOnlySpan<byte> signedMessage, ReadOnlySpan<byte> publicKey, Span<byte> message, out int written)

Purpose: Attempts to verify a signed message and recover the authenticated message bytes.

Parameters: signedMessage: The signed-message representation to verify.; publicKey: The public verification key. The length must equal PublicKeySize.; message: The destination buffer for the recovered message.; written: Receives the number of bytes written to .

Returns: QscStatus.Success on success; otherwise a QSCABI status code describing invalid sizes, insufficient output space, verification failure, or platform/algorithm unavailability.

25.24 Pqc/QscHqc.cs (10 members)

Declaration source: QSCCS/Pqc/QscHqc.cs. A throwing operation and a status-returning Try variant are distinct contracts; private native handles remain implementation details.

QscHqc.PublicKeySize [property]

public static int PublicKeySize

Purpose: Gets the public-key size, in bytes, required by the native QSCABI algorithm profile.

QscHqc.PrivateKeySize [property]

public static int PrivateKeySize

Purpose: Gets the private-key size, in bytes, required by the native QSCABI algorithm profile.

QscHqc.CiphertextSize [property]

public static int CiphertextSize

Purpose: Gets the KEM ciphertext size, in bytes, produced by encapsulation and consumed by decapsulation.

QscHqc.SharedSecretSize [property]

public static int SharedSecretSize

Purpose: Gets the shared-secret size, in bytes, produced by encapsulation and decapsulation.

QscHqc.GenerateKeyPair [method]

public static QscPqcKeyPair GenerateKeyPair()

Purpose: Generates a new key pair using QSCABI randomness.

Returns: A managed key-pair object containing newly allocated public and private key arrays.

QscHqc.Encapsulate [method]

public static QscKemEncapsulation Encapsulate(ReadOnlySpan<byte> publicKey)

Purpose: Encapsulates a shared secret to a recipient public key.

Parameters: publicKey: The recipient public key. The length must equal PublicKeySize.

Returns: A managed encapsulation containing the ciphertext and shared secret.

QscHqc.Decapsulate [method]

public static byte[] Decapsulate(ReadOnlySpan<byte> ciphertext, ReadOnlySpan<byte> privateKey)

Purpose: Decapsulates a shared secret from a KEM ciphertext.

Parameters: ciphertext: The ciphertext produced by encapsulation. The length must equal CiphertextSize.; privateKey: The recipient private key. The length must equal PrivateKeySize.

Returns: The recovered shared-secret bytes.

QscHqc.TryGenerateKeyPair [method]

public static unsafe QscStatus TryGenerateKeyPair(Span<byte> publicKey, Span<byte> privateKey)

Purpose: Attempts to generate a key pair into caller-owned buffers.

Parameters: publicKey: The destination buffer for the public key. The buffer must be at least PublicKeySize bytes.; privateKey: The destination buffer for the private key. The buffer must be at least PrivateKeySize bytes.

Returns: QscStatus.Success on success; otherwise a QSCABI status code describing invalid sizes, insufficient output space, verification failure, or platform/algorithm unavailability.

QscHqc.TryEncapsulate [method]

public static unsafe QscStatus TryEncapsulate(ReadOnlySpan<byte> publicKey, Span<byte> ciphertext, Span<byte> secret)

Purpose: Attempts to encapsulate a shared secret into caller-owned output buffers.

Parameters: publicKey: The recipient public key. The length must equal PublicKeySize.; ciphertext: The destination buffer for the KEM ciphertext. The buffer must be at least CiphertextSize bytes.; secret: The destination buffer for the shared secret. The buffer must be at least SharedSecretSize bytes.

Returns: QscStatus.Success on success; otherwise a QSCABI status code describing invalid sizes, insufficient output space, verification failure, or platform/algorithm unavailability.

QscHqc.TryDecapsulate [method]

public static unsafe QscStatus TryDecapsulate(ReadOnlySpan<byte> ciphertext, ReadOnlySpan<byte> privateKey, Span<byte> secret)

Purpose: Attempts to decapsulate a shared secret into a caller-owned output buffer.

Parameters: ciphertext: The KEM ciphertext. The length must equal CiphertextSize.; privateKey: The recipient private key. The length must equal PrivateKeySize.; secret: The destination buffer for the shared secret. The buffer must be at least SharedSecretSize bytes.

Returns: QscStatus.Success on success; otherwise a QSCABI status code describing invalid sizes, insufficient output space, verification failure, or platform/algorithm unavailability.

25.25 Pqc/QscMcEliece.cs (10 members)

Declaration source: QSCCS/Pqc/QscMcEliece.cs. A throwing operation and a status-returning Try variant are distinct contracts; private native handles remain implementation details.

QscMcEliece.PublicKeySize [property]

public static int PublicKeySize

Purpose: Gets the public-key size, in bytes, required by the native QSCABI algorithm profile.

QscMcEliece.PrivateKeySize [property]

public static int PrivateKeySize

Purpose: Gets the private-key size, in bytes, required by the native QSCABI algorithm profile.

QscMcEliece.CiphertextSize [property]

public static int CiphertextSize

Purpose: Gets the KEM ciphertext size, in bytes, produced by encapsulation and consumed by decapsulation.

QscMcEliece.SharedSecretSize [property]

public static int SharedSecretSize

Purpose: Gets the shared-secret size, in bytes, produced by encapsulation and decapsulation.

QscMcEliece.GenerateKeyPair [method]

public static QscPqcKeyPair GenerateKeyPair()

Purpose: Generates a new key pair using QSCABI randomness.

Returns: A managed key-pair object containing newly allocated public and private key arrays.

QscMcEliece.Encapsulate [method]

public static QscKemEncapsulation Encapsulate(ReadOnlySpan<byte> publicKey)

Purpose: Encapsulates a shared secret to a recipient public key.

Parameters: publicKey: The recipient public key. The length must equal PublicKeySize.

Returns: A managed encapsulation containing the ciphertext and shared secret.

QscMcEliece.Decapsulate [method]

public static byte[] Decapsulate(ReadOnlySpan<byte> ciphertext, ReadOnlySpan<byte> privateKey)

Purpose: Decapsulates a shared secret from a KEM ciphertext.

Parameters: ciphertext: The ciphertext produced by encapsulation. The length must equal CiphertextSize.; privateKey: The recipient private key. The length must equal PrivateKeySize.

Returns: The recovered shared-secret bytes.

QscMcEliece.TryGenerateKeyPair [method]

public static unsafe QscStatus TryGenerateKeyPair(Span<byte> publicKey, Span<byte> privateKey)

Purpose: Attempts to generate a key pair into caller-owned buffers.

Parameters: publicKey: The destination buffer for the public key. The buffer must be at least PublicKeySize bytes.; privateKey: The destination buffer for the private key. The buffer must be at least PrivateKeySize bytes.

Returns: QscStatus.Success on success; otherwise a QSCABI status code describing invalid sizes, insufficient output space, verification failure, or platform/algorithm unavailability.

QscMcEliece.TryEncapsulate [method]

public static unsafe QscStatus TryEncapsulate(ReadOnlySpan<byte> publicKey, Span<byte> ciphertext, Span<byte> secret)

Purpose: Attempts to encapsulate a shared secret into caller-owned output buffers.

Parameters: publicKey: The recipient public key. The length must equal PublicKeySize.; ciphertext: The destination buffer for the KEM ciphertext. The buffer must be at least CiphertextSize bytes.; secret: The destination buffer for the shared secret. The buffer must be at least SharedSecretSize bytes.

Returns: QscStatus.Success on success; otherwise a QSCABI status code describing invalid sizes, insufficient output space, verification failure, or platform/algorithm unavailability.

QscMcEliece.TryDecapsulate [method]

public static unsafe QscStatus TryDecapsulate(ReadOnlySpan<byte> ciphertext, ReadOnlySpan<byte> privateKey, Span<byte> secret)

Purpose: Attempts to decapsulate a shared secret into a caller-owned output buffer.

Parameters: ciphertext: The KEM ciphertext. The length must equal CiphertextSize.; privateKey: The recipient private key. The length must equal PrivateKeySize.; secret: The destination buffer for the shared secret. The buffer must be at least SharedSecretSize bytes.

Returns: QscStatus.Success on success; otherwise a QSCABI status code describing invalid sizes, insufficient output space, verification failure, or platform/algorithm unavailability.

25.26 Pqc/QscMlDsa.cs (10 members)

Declaration source: QSCCS/Pqc/QscMlDsa.cs. A throwing operation and a status-returning Try variant are distinct contracts; private native handles remain implementation details.

QscMlDsa.PublicKeySize [property]

public static int PublicKeySize

Purpose: Gets the public-key size, in bytes, required by the native QSCABI algorithm profile.

QscMlDsa.PrivateKeySize [property]

public static int PrivateKeySize

Purpose: Gets the private-key size, in bytes, required by the native QSCABI algorithm profile.

QscMlDsa.SignatureSize [property]

public static int SignatureSize

Purpose: Gets the detached signature size, in bytes, for the native signature profile.

QscMlDsa.SignedMessageSize [method]

public static int SignedMessageSize(int messageLength)

Purpose: Calculates the signed-message buffer size required for a message of the supplied length.

Parameters: messageLength: The non-negative message length, in bytes.

Returns: The required signed-message buffer size, in bytes.

QscMlDsa.GenerateKeyPair [method]

public static QscPqcKeyPair GenerateKeyPair()

Purpose: Generates a new key pair using QSCABI randomness.

Returns: A managed key-pair object containing newly allocated public and private key arrays.

QscMlDsa.Sign [method]

public static byte[] Sign(ReadOnlySpan<byte> message, ReadOnlySpan<byte> privateKey)

Purpose: Signs a message using the supplied private key.

Parameters: message: The message bytes to sign. The buffer is read during the call and is not retained.; privateKey: The private signing key. The length must equal PrivateKeySize.

Returns: A signed-message buffer containing the signature and recoverable message representation produced by QSCABI.

QscMlDsa.Verify [method]

public static byte[] Verify(ReadOnlySpan<byte> signedMessage, ReadOnlySpan<byte> publicKey)

Purpose: Verifies a signed message and recovers the authenticated message bytes.

Parameters: signedMessage: The signed-message buffer returned by Sign(ReadOnlySpan{byte}, ReadOnlySpan{byte}).; publicKey: The public verification key. The length must equal PublicKeySize.

Returns: The recovered message bytes when verification succeeds.

QscMlDsa.TryGenerateKeyPair [method]

public static unsafe QscStatus TryGenerateKeyPair(Span<byte> publicKey, Span<byte> privateKey)

Purpose: Attempts to generate a key pair into caller-owned buffers.

Parameters: publicKey: The destination buffer for the public key. The buffer must be at least PublicKeySize bytes.; privateKey: The destination buffer for the private key. The buffer must be at least PrivateKeySize bytes.

Returns: QscStatus.Success on success; otherwise a QSCABI status code describing invalid sizes, insufficient output space, verification failure, or platform/algorithm unavailability.

QscMlDsa.TrySign [method]

public static unsafe QscStatus TrySign(ReadOnlySpan<byte> message, ReadOnlySpan<byte> privateKey, Span<byte> signedMessage, out int written)

Purpose: Attempts to sign a message into a caller-owned signed-message buffer.

Parameters: message: The message bytes to sign.; privateKey: The private signing key. The length must equal PrivateKeySize.; signedMessage: The destination buffer for the signed-message representation.; written: Receives the number of bytes written to .

Returns: QscStatus.Success on success; otherwise a QSCABI status code describing invalid sizes, insufficient output space, verification failure, or platform/algorithm unavailability.

QscMlDsa.TryVerify [method]

public static unsafe QscStatus TryVerify(ReadOnlySpan<byte> signedMessage, ReadOnlySpan<byte> publicKey, Span<byte> message, out int written)

Purpose: Attempts to verify a signed message and recover the authenticated message bytes.

Parameters: signedMessage: The signed-message representation to verify.; publicKey: The public verification key. The length must equal PublicKeySize.; message: The destination buffer for the recovered message.; written: Receives the number of bytes written to .

Returns: QscStatus.Success on success; otherwise a QSCABI status code describing invalid sizes, insufficient output space, verification failure, or platform/algorithm unavailability.

25.27 Pqc/QscMlKem.cs (10 members)

Declaration source: QSCCS/Pqc/QscMlKem.cs. A throwing operation and a status-returning Try variant are distinct contracts; private native handles remain implementation details.

QscMlKem.PublicKeySize [property]

public static int PublicKeySize

Purpose: Gets the public-key size, in bytes, required by the native QSCABI algorithm profile.

QscMlKem.PrivateKeySize [property]

public static int PrivateKeySize

Purpose: Gets the private-key size, in bytes, required by the native QSCABI algorithm profile.

QscMlKem.CiphertextSize [property]

public static int CiphertextSize

Purpose: Gets the KEM ciphertext size, in bytes, produced by encapsulation and consumed by decapsulation.

QscMlKem.SharedSecretSize [property]

public static int SharedSecretSize

Purpose: Gets the shared-secret size, in bytes, produced by encapsulation and decapsulation.

QscMlKem.GenerateKeyPair [method]

public static QscPqcKeyPair GenerateKeyPair()

Purpose: Generates a new key pair using QSCABI randomness.

Returns: A managed key-pair object containing newly allocated public and private key arrays.

QscMlKem.Encapsulate [method]

public static QscKemEncapsulation Encapsulate(ReadOnlySpan<byte> publicKey)

Purpose: Encapsulates a shared secret to a recipient public key.

Parameters: publicKey: The recipient public key. The length must equal PublicKeySize.

Returns: A managed encapsulation containing the ciphertext and shared secret.

QscMlKem.Decapsulate [method]

public static byte[] Decapsulate(ReadOnlySpan<byte> ciphertext, ReadOnlySpan<byte> privateKey)

Purpose: Decapsulates a shared secret from a KEM ciphertext.

Parameters: ciphertext: The ciphertext produced by encapsulation. The length must equal CiphertextSize.; privateKey: The recipient private key. The length must equal PrivateKeySize.

Returns: The recovered shared-secret bytes.

QscMlKem.TryGenerateKeyPair [method]

public static unsafe QscStatus TryGenerateKeyPair(Span<byte> publicKey, Span<byte> privateKey)

Purpose: Attempts to generate a key pair into caller-owned buffers.

Parameters: publicKey: The destination buffer for the public key. The buffer must be at least PublicKeySize bytes.; privateKey: The destination buffer for the private key. The buffer must be at least PrivateKeySize bytes.

Returns: QscStatus.Success on success; otherwise a QSCABI status code describing invalid sizes, insufficient output space, verification failure, or platform/algorithm unavailability.

QscMlKem.TryEncapsulate [method]

public static unsafe QscStatus TryEncapsulate(ReadOnlySpan<byte> publicKey, Span<byte> ciphertext, Span<byte> secret)

Purpose: Attempts to encapsulate a shared secret into caller-owned output buffers.

Parameters: publicKey: The recipient public key. The length must equal PublicKeySize.; ciphertext: The destination buffer for the KEM ciphertext. The buffer must be at least CiphertextSize bytes.; secret: The destination buffer for the shared secret. The buffer must be at least SharedSecretSize bytes.

Returns: QscStatus.Success on success; otherwise a QSCABI status code describing invalid sizes, insufficient output space, verification failure, or platform/algorithm unavailability.

QscMlKem.TryDecapsulate [method]

public static unsafe QscStatus TryDecapsulate(ReadOnlySpan<byte> ciphertext, ReadOnlySpan<byte> privateKey, Span<byte> secret)

Purpose: Attempts to decapsulate a shared secret into a caller-owned output buffer.

Parameters: ciphertext: The KEM ciphertext. The length must equal CiphertextSize.; privateKey: The recipient private key. The length must equal PrivateKeySize.; secret: The destination buffer for the shared secret. The buffer must be at least SharedSecretSize bytes.

Returns: QscStatus.Success on success; otherwise a QSCABI status code describing invalid sizes, insufficient output space, verification failure, or platform/algorithm unavailability.

25.28 Pqc/QscPqcTypes.cs (2 members)

Declaration source: QSCCS/Pqc/QscPqcTypes.cs. A throwing operation and a status-returning Try variant are distinct contracts; private native handles remain implementation details.

QscPqcKeyPair.Dispose [method]

public void Dispose()

Purpose: Zeros the owned private-key array without modifying the public key.

Lifetime: Observe native owner validity and deterministic disposal; no use after close.

QscKemEncapsulation.Dispose [method]

public void Dispose()

Purpose: Zeros the owned shared-secret array without modifying the ciphertext.

Lifetime: Observe native owner validity and deterministic disposal; no use after close.

25.29 Pqc/QscSlhDsa.cs (10 members)

Declaration source: QSCCS/Pqc/QscSlhDsa.cs. A throwing operation and a status-returning Try variant are distinct contracts; private native handles remain implementation details.

QscSlhDsa.PublicKeySize [property]

public static int PublicKeySize

Purpose: Gets the public-key size, in bytes, required by the native QSCABI algorithm profile.

QscSlhDsa.PrivateKeySize [property]

public static int PrivateKeySize

Purpose: Gets the private-key size, in bytes, required by the native QSCABI algorithm profile.

QscSlhDsa.SignatureSize [property]

public static int SignatureSize

Purpose: Gets the detached signature size, in bytes, for the native signature profile.

QscSlhDsa.SignedMessageSize [method]

public static int SignedMessageSize(int messageLength)

Purpose: Calculates the signed-message buffer size required for a message of the supplied length.

Parameters: messageLength: The non-negative message length, in bytes.

Returns: The required signed-message buffer size, in bytes.

QscSlhDsa.GenerateKeyPair [method]

public static QscPqcKeyPair GenerateKeyPair()

Purpose: Generates a new key pair using QSCABI randomness.

Returns: A managed key-pair object containing newly allocated public and private key arrays.

QscSlhDsa.Sign [method]

public static byte[] Sign(ReadOnlySpan<byte> message, ReadOnlySpan<byte> privateKey)

Purpose: Signs a message using the supplied private key.

Parameters: message: The message bytes to sign. The buffer is read during the call and is not retained.; privateKey: The private signing key. The length must equal PrivateKeySize.

Returns: A signed-message buffer containing the signature and recoverable message representation produced by QSCABI.

QscSlhDsa.Verify [method]

public static byte[] Verify(ReadOnlySpan<byte> signedMessage, ReadOnlySpan<byte> publicKey)

Purpose: Verifies a signed message and recovers the authenticated message bytes.

Parameters: signedMessage: The signed-message buffer returned by Sign(ReadOnlySpan{byte}, ReadOnlySpan{byte}).; publicKey: The public verification key. The length must equal PublicKeySize.

Returns: The recovered message bytes when verification succeeds.

QscSlhDsa.TryGenerateKeyPair [method]

public static unsafe QscStatus TryGenerateKeyPair(Span<byte> publicKey, Span<byte> privateKey)

Purpose: Attempts to generate a key pair into caller-owned buffers.

Parameters: publicKey: The destination buffer for the public key. The buffer must be at least PublicKeySize bytes.; privateKey: The destination buffer for the private key. The buffer must be at least PrivateKeySize bytes.

Returns: QscStatus.Success on success; otherwise a QSCABI status code describing invalid sizes, insufficient output space, verification failure, or platform/algorithm unavailability.

QscSlhDsa.TrySign [method]

public static unsafe QscStatus TrySign(ReadOnlySpan<byte> message, ReadOnlySpan<byte> privateKey, Span<byte> signedMessage, out int written)

Purpose: Attempts to sign a message into a caller-owned signed-message buffer.

Parameters: message: The message bytes to sign.; privateKey: The private signing key. The length must equal PrivateKeySize.; signedMessage: The destination buffer for the signed-message representation.; written: Receives the number of bytes written to .

Returns: QscStatus.Success on success; otherwise a QSCABI status code describing invalid sizes, insufficient output space, verification failure, or platform/algorithm unavailability.

QscSlhDsa.TryVerify [method]

public static unsafe QscStatus TryVerify(ReadOnlySpan<byte> signedMessage, ReadOnlySpan<byte> publicKey, Span<byte> message, out int written)

Purpose: Attempts to verify a signed message and recover the authenticated message bytes.

Parameters: signedMessage: The signed-message representation to verify.; publicKey: The public verification key. The length must equal PublicKeySize.; message: The destination buffer for the recovered message.; written: Receives the number of bytes written to .

Returns: QscStatus.Success on success; otherwise a QSCABI status code describing invalid sizes, insufficient output space, verification failure, or platform/algorithm unavailability.

25.30 Streaming/QscStreaming.cs (17 members)

Declaration source: QSCCS/Streaming/QscStreaming.cs. A throwing operation and a status-returning Try variant are distinct contracts; private native handles remain implementation details.

QscIncrementalDigest.QscIncrementalDigest [constructor]

public unsafe QscIncrementalDigest(QscDigestAlgorithm algorithm)

Purpose: Create a digest state using the native algorithm selector.

Parameters: algorithm: SHA-2 or SHA-3 fixed digest algorithm.

QscIncrementalDigest.Update [method]

public unsafe void Update(ReadOnlySpan<byte> data)

Purpose: Append borrowed bytes to the active native digest state.

Parameters: data: Message chunk; zero length is permitted.

QscIncrementalDigest.FinalizeDigest [method]

public unsafe byte[] FinalizeDigest()

Purpose: Finalize the digest exactly once.

Returns: Caller-owned fixed-length digest array.

QscIncrementalDigest.Dispose [method]

public void Dispose()

Purpose: Dispose of the native digest state; idempotent.

Lifetime: Observe native owner validity and deterministic disposal; no use after close.

QscIncrementalXof.QscIncrementalXof [constructor]

public unsafe QscIncrementalXof(QscStreamingXofAlgorithm algorithm)

Purpose: Create a native SHAKE-128/256/512 sponge.

Parameters: algorithm: Exact native XOF selector.

QscIncrementalXof.Absorb [method]

public unsafe void Absorb(ReadOnlySpan<byte> data)

Purpose: Absorb borrowed message bytes before finalization.

Parameters: data: Next message chunk.

QscIncrementalXof.FinalizeAbsorb [method]

public void FinalizeAbsorb()

Purpose: Finish the absorb phase once; subsequently only squeeze.

QscIncrementalXof.Squeeze [method]

public unsafe byte[] Squeeze(int length)

Purpose: Squeeze the next output bytes, retaining the native sponge position.

Parameters: length: Requested output size, in bytes.

Returns: Owned XOF output.

QscIncrementalXof.Dispose [method]

public void Dispose()

Purpose: Dispose of the native SHAKE sponge.

Lifetime: Observe native owner validity and deterministic disposal; no use after close.

QscIncrementalMac.QscIncrementalMac [constructor]

public unsafe QscIncrementalMac(QscStreamingMacAlgorithm algorithm, ReadOnlySpan<byte> key, ReadOnlySpan<byte> customization = default)

Purpose: Create a native HMAC or KMAC state.

Parameters: algorithm: Exact keyed MAC algorithm ID.; key: Nonempty borrowed secret key.; customization: KMAC customization; HMAC requires empty customization.

QscIncrementalMac.Update [method]

public unsafe void Update(ReadOnlySpan<byte> message)

Purpose: Append a message chunk while MAC state is active.

Parameters: message: Borrowed message bytes.

QscIncrementalMac.FinalizeMac [method]

public unsafe byte[] FinalizeMac(int outputLength)

Purpose: Produce one fixed-length HMAC or chosen-length KMAC tag.

Parameters: outputLength: HMAC digest length (32/48/64) or nonzero KMAC output length.

Returns: Caller-owned MAC bytes.

QscIncrementalMac.Dispose [method]

public void Dispose()

Purpose: Destroy native keyed state, including its copied secret key.

QscCtrStream.QscCtrStream [constructor]

public unsafe QscCtrStream(QscStreamingCtrAlgorithm algorithm, ReadOnlySpan<byte> key, ReadOnlySpan<byte> nonce)

Purpose: Initialize AES CTR with exact key length and fresh 16-byte initial counter.

Parameters: algorithm: AES-128/256 and little/big-endian counter mode.; key: Caller-owned secret, borrowed during create.; nonce: Fresh 16-byte initial counter for this key.

QscCtrStream.Transform [method]

public unsafe byte[] Transform(ReadOnlySpan<byte> input)

Purpose: Transform bytes into a separate caller-owned buffer; native state advances.

Parameters: input: Borrowed plaintext/ciphertext bytes.

Returns: New ciphertext/plaintext bytes, without authentication.

QscCtrStream.Dispose [method]

public void Dispose()

Purpose: Erase native AES key and counter state.

QscCustomizedShake.Compute [method]

public static unsafe byte[] Compute(QscStreamingXofAlgorithm algorithm, ReadOnlySpan<byte> key, ReadOnlySpan<byte> name, ReadOnlySpan<byte> customization, int outputLength)

Purpose: Calculate one customized cSHAKE digest with independent call state.

Parameters: algorithm: Native SHAKE strength.; key: Caller-owned absorbed data.; name: Optional function-name domain separator.; customization: Optional cSHAKE personalization.; outputLength: Nonzero requested output bytes.

Returns: Fresh caller-owned digest bytes.

25.31 Symmetric/QscAes.cs (16 members)

Declaration source: QSCCS/Symmetric/QscAes.cs. A throwing operation and a status-returning Try variant are distinct contracts; private native handles remain implementation details.

QscAes.BlockSize [property]

public static int BlockSize

Purpose: Gets the AES block size, in bytes.

QscAes.Key128Size [property]

public static int Key128Size

Purpose: Gets the AES-128 key size, in bytes.

QscAes.Key256Size [property]

public static int Key256Size

Purpose: Gets the AES-256 key size, in bytes.

QscAes.GcmNonceSize [property]

public static int GcmNonceSize

Purpose: Gets the preferred AES-GCM nonce size, in bytes.

QscAes.GcmMaxNonceSize [property]

public static int GcmMaxNonceSize

Purpose: Gets the maximum AES-GCM nonce size accepted by QSCABI, in bytes.

QscAes.GcmTagSize [property]

public static int GcmTagSize

Purpose: Gets the AES-GCM authentication-tag size, in bytes.

QscAes.TransformCtr256 [method]

public static byte[] TransformCtr256(ReadOnlySpan<byte> input, ReadOnlySpan<byte> key, ReadOnlySpan<byte> nonce, QscAesCtrCounter counterMode = QscAesCtrCounter.BigEndian)

Purpose: Transforms data using AES-256 in CTR mode.

Parameters: input: The input bytes to transform.; key: The cryptographic key material. The buffer is not retained.; nonce: The nonce value. Nonces must satisfy the uniqueness requirements of the selected primitive.; counterMode: The byte order used when incrementing the AES-CTR counter block.

Returns: The operation result.

QscAes.TryTransformCtr256 [method]

public static unsafe QscStatus TryTransformCtr256(ReadOnlySpan<byte> input, ReadOnlySpan<byte> key, ReadOnlySpan<byte> nonce, Span<byte> output, QscAesCtrCounter counterMode = QscAesCtrCounter.BigEndian)

Purpose: Attempts to transform data using AES-256 in CTR mode into a caller-provided buffer.

Parameters: input: The input bytes to transform.; key: The cryptographic key material. The buffer is not retained.; nonce: The nonce value. Nonces must satisfy the uniqueness requirements of the selected primitive.; output: The caller-owned output buffer.; counterMode: The byte order used when incrementing the AES-CTR counter block.

Returns: QscStatus.Success on success, or a non-success status when validation, authentication, or native execution fails.

QscAes.EncryptGcm128 [method]

public static byte[] EncryptGcm128(ReadOnlySpan<byte> plaintext, ReadOnlySpan<byte> key, ReadOnlySpan<byte> nonce, ReadOnlySpan<byte> aad = default)

Purpose: Encrypts plaintext using AES-128-GCM and returns ciphertext with the authentication tag appended.

Parameters: plaintext: The plaintext bytes to encrypt.; key: The cryptographic key material. The buffer is not retained.; nonce: The nonce value. Nonces must satisfy the uniqueness requirements of the selected primitive.; aad: Associated data to authenticate but not encrypt.

Returns: A newly allocated ciphertext buffer containing the encrypted data and authentication tag when the mode is AEAD.

QscAes.DecryptGcm128 [method]

public static byte[] DecryptGcm128(ReadOnlySpan<byte> ciphertext, ReadOnlySpan<byte> key, ReadOnlySpan<byte> nonce, ReadOnlySpan<byte> aad = default)

Purpose: Decrypts and authenticates AES-128-GCM ciphertext.

Parameters: ciphertext: The ciphertext buffer, including the authentication tag for AEAD modes.; key: The cryptographic key material. The buffer is not retained.; nonce: The nonce value. Nonces must satisfy the uniqueness requirements of the selected primitive.; aad: Associated data to authenticate but not encrypt.

Returns: A newly allocated plaintext buffer. Authentication failures cause a non-success status or exception depending on the API variant.

QscAes.EncryptGcm256 [method]

public static byte[] EncryptGcm256(ReadOnlySpan<byte> plaintext, ReadOnlySpan<byte> key, ReadOnlySpan<byte> nonce, ReadOnlySpan<byte> aad = default)

Purpose: Encrypts plaintext using AES-256-GCM and returns ciphertext with the authentication tag appended.

Parameters: plaintext: The plaintext bytes to encrypt.; key: The cryptographic key material. The buffer is not retained.; nonce: The nonce value. Nonces must satisfy the uniqueness requirements of the selected primitive.; aad: Associated data to authenticate but not encrypt.

Returns: A newly allocated ciphertext buffer containing the encrypted data and authentication tag when the mode is AEAD.

QscAes.DecryptGcm256 [method]

public static byte[] DecryptGcm256(ReadOnlySpan<byte> ciphertext, ReadOnlySpan<byte> key, ReadOnlySpan<byte> nonce, ReadOnlySpan<byte> aad = default)

Purpose: Decrypts and authenticates AES-256-GCM ciphertext.

Parameters: ciphertext: The ciphertext buffer, including the authentication tag for AEAD modes.; key: The cryptographic key material. The buffer is not retained.; nonce: The nonce value. Nonces must satisfy the uniqueness requirements of the selected primitive.; aad: Associated data to authenticate but not encrypt.

Returns: A newly allocated plaintext buffer. Authentication failures cause a non-success status or exception depending on the API variant.

QscAes.TryEncryptGcm128 [method]

public static unsafe QscStatus TryEncryptGcm128(ReadOnlySpan<byte> plaintext, ReadOnlySpan<byte> key, ReadOnlySpan<byte> nonce, ReadOnlySpan<byte> aad, Span<byte> ciphertext, out int written)

Purpose: Attempts to encrypt plaintext using AES-128-GCM into a caller-provided buffer.

Parameters: plaintext: The plaintext bytes to encrypt.; key: The cryptographic key material. The buffer is not retained.; nonce: The nonce value. Nonces must satisfy the uniqueness requirements of the selected primitive.; aad: Associated data to authenticate but not encrypt.; ciphertext: The ciphertext buffer, including the authentication tag for AEAD modes.; written: Receives the number of bytes written to the output buffer.

Returns: QscStatus.Success on success, or a non-success status when validation, authentication, or native execution fails.

QscAes.TryDecryptGcm128 [method]

public static unsafe QscStatus TryDecryptGcm128(ReadOnlySpan<byte> ciphertext, ReadOnlySpan<byte> key, ReadOnlySpan<byte> nonce, ReadOnlySpan<byte> aad, Span<byte> plaintext, out int written)

Purpose: Attempts to decrypt and authenticate AES-128-GCM ciphertext into a caller-provided buffer.

Parameters: ciphertext: The ciphertext buffer, including the authentication tag for AEAD modes.; key: The cryptographic key material. The buffer is not retained.; nonce: The nonce value. Nonces must satisfy the uniqueness requirements of the selected primitive.; aad: Associated data to authenticate but not encrypt.; plaintext: The plaintext bytes to encrypt.; written: Receives the number of bytes written to the output buffer.

Returns: QscStatus.Success on success, or a non-success status when validation, authentication, or native execution fails.

QscAes.TryEncryptGcm256 [method]

public static unsafe QscStatus TryEncryptGcm256(ReadOnlySpan<byte> plaintext, ReadOnlySpan<byte> key, ReadOnlySpan<byte> nonce, ReadOnlySpan<byte> aad, Span<byte> ciphertext, out int written)

Purpose: Attempts to encrypt plaintext using AES-256-GCM into a caller-provided buffer.

Parameters: plaintext: The plaintext bytes to encrypt.; key: The cryptographic key material. The buffer is not retained.; nonce: The nonce value. Nonces must satisfy the uniqueness requirements of the selected primitive.; aad: Associated data to authenticate but not encrypt.; ciphertext: The ciphertext buffer, including the authentication tag for AEAD modes.; written: Receives the number of bytes written to the output buffer.

Returns: QscStatus.Success on success, or a non-success status when validation, authentication, or native execution fails.

QscAes.TryDecryptGcm256 [method]

public static unsafe QscStatus TryDecryptGcm256(ReadOnlySpan<byte> ciphertext, ReadOnlySpan<byte> key, ReadOnlySpan<byte> nonce, ReadOnlySpan<byte> aad, Span<byte> plaintext, out int written)

Purpose: Attempts to decrypt and authenticate AES-256-GCM ciphertext into a caller-provided buffer.

Parameters: ciphertext: The ciphertext buffer, including the authentication tag for AEAD modes.; key: The cryptographic key material. The buffer is not retained.; nonce: The nonce value. Nonces must satisfy the uniqueness requirements of the selected primitive.; aad: Associated data to authenticate but not encrypt.; plaintext: The plaintext bytes to encrypt.; written: Receives the number of bytes written to the output buffer.

Returns: QscStatus.Success on success, or a non-success status when validation, authentication, or native execution fails.

25.32 Symmetric/QscChaCha.cs (4 members)

Declaration source: QSCCS/Symmetric/QscChaCha.cs. A throwing operation and a status-returning Try variant are distinct contracts; private native handles remain implementation details.

QscChaCha.KeySize [property]

public static int KeySize

Purpose: Gets the KeySize value reported by QSCABI.

QscChaCha.NonceSize [property]

public static int NonceSize

Purpose: Gets the NonceSize value reported by QSCABI.

QscChaCha.Transform [method]

public static byte[] Transform(ReadOnlySpan<byte> input, ReadOnlySpan<byte> key, ReadOnlySpan<byte> nonce)

Purpose: Transforms data using ChaCha20.

Parameters: input: The input bytes to transform.; key: The cryptographic key material. The buffer is not retained.; nonce: The nonce value. Nonces must satisfy the uniqueness requirements of the selected primitive.

Returns: The operation result.

QscChaCha.TryTransform [method]

public static unsafe QscStatus TryTransform(ReadOnlySpan<byte> input, ReadOnlySpan<byte> key, ReadOnlySpan<byte> nonce, Span<byte> output)

Purpose: Attempts to transform data using ChaCha20 into a caller-provided output buffer.

Parameters: input: The input bytes to transform.; key: The cryptographic key material. The buffer is not retained.; nonce: The nonce value. Nonces must satisfy the uniqueness requirements of the selected primitive.; output: The caller-owned output buffer.

Returns: QscStatus.Success on success, or a non-success status when validation, authentication, or native execution fails.

25.33 Symmetric/QscChaChaPoly1305.cs (7 members)

Declaration source: QSCCS/Symmetric/QscChaChaPoly1305.cs. A throwing operation and a status-returning Try variant are distinct contracts; private native handles remain implementation details.

QscChaChaPoly1305.KeySize [property]

public static int KeySize

Purpose: Gets the KeySize value reported by QSCABI.

QscChaChaPoly1305.NonceSize [property]

public static int NonceSize

Purpose: Gets the NonceSize value reported by QSCABI.

QscChaChaPoly1305.TagSize [property]

public static int TagSize

Purpose: Gets the TagSize value reported by QSCABI.

QscChaChaPoly1305.Encrypt [method]

public static byte[] Encrypt(ReadOnlySpan<byte> plaintext, ReadOnlySpan<byte> key, ReadOnlySpan<byte> nonce, ReadOnlySpan<byte> aad = default)

Purpose: Encrypts plaintext using ChaCha20-Poly1305 and returns ciphertext with the tag appended.

Parameters: plaintext: The plaintext bytes to encrypt.; key: The cryptographic key material. The buffer is not retained.; nonce: The nonce value. Nonces must satisfy the uniqueness requirements of the selected primitive.; aad: Associated data to authenticate but not encrypt.

Returns: A newly allocated ciphertext buffer containing the encrypted data and authentication tag when the mode is AEAD.

QscChaChaPoly1305.Decrypt [method]

public static byte[] Decrypt(ReadOnlySpan<byte> ciphertext, ReadOnlySpan<byte> key, ReadOnlySpan<byte> nonce, ReadOnlySpan<byte> aad = default)

Purpose: Decrypts and authenticates ChaCha20-Poly1305 ciphertext.

Parameters: ciphertext: The ciphertext buffer, including the authentication tag for AEAD modes.; key: The cryptographic key material. The buffer is not retained.; nonce: The nonce value. Nonces must satisfy the uniqueness requirements of the selected primitive.; aad: Associated data to authenticate but not encrypt.

Returns: A newly allocated plaintext buffer. Authentication failures cause a non-success status or exception depending on the API variant.

QscChaChaPoly1305.TryEncrypt [method]

public static unsafe QscStatus TryEncrypt(ReadOnlySpan<byte> plaintext, ReadOnlySpan<byte> key, ReadOnlySpan<byte> nonce, ReadOnlySpan<byte> aad, Span<byte> ciphertext, out int written)

Purpose: Attempts to encrypt plaintext using ChaCha20-Poly1305 into a caller-provided buffer.

Parameters: plaintext: The plaintext bytes to encrypt.; key: The cryptographic key material. The buffer is not retained.; nonce: The nonce value. Nonces must satisfy the uniqueness requirements of the selected primitive.; aad: Associated data to authenticate but not encrypt.; ciphertext: The ciphertext buffer, including the authentication tag for AEAD modes.; written: Receives the number of bytes written to the output buffer.

Returns: QscStatus.Success on success, or a non-success status when validation, authentication, or native execution fails.

QscChaChaPoly1305.TryDecrypt [method]

public static unsafe QscStatus TryDecrypt(ReadOnlySpan<byte> ciphertext, ReadOnlySpan<byte> key, ReadOnlySpan<byte> nonce, ReadOnlySpan<byte> aad, Span<byte> plaintext, out int written)

Purpose: Attempts to decrypt and authenticate ChaCha20-Poly1305 ciphertext into a caller-provided buffer.

Parameters: ciphertext: The ciphertext buffer, including the authentication tag for AEAD modes.; key: The cryptographic key material. The buffer is not retained.; nonce: The nonce value. Nonces must satisfy the uniqueness requirements of the selected primitive.; aad: Associated data to authenticate but not encrypt.; plaintext: The plaintext bytes to encrypt.; written: Receives the number of bytes written to the output buffer.

Returns: QscStatus.Success on success, or a non-success status when validation, authentication, or native execution fails.

25.34 Symmetric/QscCsx.cs (7 members)

Declaration source: QSCCS/Symmetric/QscCsx.cs. A throwing operation and a status-returning Try variant are distinct contracts; private native handles remain implementation details.

QscCsx.KeySize [property]

public static int KeySize

Purpose: Gets the KeySize value reported by QSCABI.

QscCsx.NonceSize [property]

public static int NonceSize

Purpose: Gets the NonceSize value reported by QSCABI.

QscCsx.TagSize [property]

public static int TagSize

Purpose: Gets the TagSize value reported by QSCABI.

QscCsx.Encrypt [method]

public static byte[] Encrypt(ReadOnlySpan<byte> plaintext, ReadOnlySpan<byte> key, ReadOnlySpan<byte> nonce, ReadOnlySpan<byte> info = default, ReadOnlySpan<byte> aad = default)

Purpose: Encrypts plaintext using CSX and returns ciphertext with the authentication tag appended.

Parameters: plaintext: The plaintext bytes to encrypt.; key: The cryptographic key material. The buffer is not retained.; nonce: The nonce value. Nonces must satisfy the uniqueness requirements of the selected primitive.; info: Optional context or domain-separation information.; aad: Associated data to authenticate but not encrypt.

Returns: A newly allocated ciphertext buffer containing the encrypted data and authentication tag when the mode is AEAD.

QscCsx.Decrypt [method]

public static byte[] Decrypt(ReadOnlySpan<byte> ciphertext, ReadOnlySpan<byte> key, ReadOnlySpan<byte> nonce, ReadOnlySpan<byte> info = default, ReadOnlySpan<byte> aad = default)

Purpose: Decrypts and authenticates CSX ciphertext.

Parameters: ciphertext: The ciphertext buffer, including the authentication tag for AEAD modes.; key: The cryptographic key material. The buffer is not retained.; nonce: The nonce value. Nonces must satisfy the uniqueness requirements of the selected primitive.; info: Optional context or domain-separation information.; aad: Associated data to authenticate but not encrypt.

Returns: A newly allocated plaintext buffer. Authentication failures cause a non-success status or exception depending on the API variant.

QscCsx.TryEncrypt [method]

public static unsafe QscStatus TryEncrypt(ReadOnlySpan<byte> plaintext, ReadOnlySpan<byte> key, ReadOnlySpan<byte> nonce, ReadOnlySpan<byte> info, ReadOnlySpan<byte> aad, Span<byte> ciphertext, out int written)

Purpose: Attempts to encrypt plaintext using CSX into a caller-provided buffer.

Parameters: plaintext: The plaintext bytes to encrypt.; key: The cryptographic key material. The buffer is not retained.; nonce: The nonce value. Nonces must satisfy the uniqueness requirements of the selected primitive.; info: The optional context or domain-separation information.; aad: Associated data to authenticate but not encrypt.; ciphertext: The ciphertext buffer, including the authentication tag for AEAD modes.; written: Receives the number of bytes written to the output buffer.

Returns: QscStatus.Success on success, or a non-success status when validation, authentication, or native execution fails.

QscCsx.TryDecrypt [method]

public static unsafe QscStatus TryDecrypt(ReadOnlySpan<byte> ciphertext, ReadOnlySpan<byte> key, ReadOnlySpan<byte> nonce, ReadOnlySpan<byte> info, ReadOnlySpan<byte> aad, Span<byte> plaintext, out int written)

Purpose: Attempts to decrypt and authenticate CSX ciphertext into a caller-provided buffer.

Parameters: ciphertext: The ciphertext buffer, including the authentication tag for AEAD modes.; key: The cryptographic key material. The buffer is not retained.; nonce: The nonce value. Nonces must satisfy the uniqueness requirements of the selected primitive.; info: The optional context or domain-separation information.; aad: Associated data to authenticate but not encrypt.; plaintext: The plaintext bytes to encrypt.; written: Receives the number of bytes written to the output buffer.

Returns: QscStatus.Success on success, or a non-success status when validation, authentication, or native execution fails.

25.35 Symmetric/QscRcs.cs (7 members)

Declaration source: QSCCS/Symmetric/QscRcs.cs. A throwing operation and a status-returning Try variant are distinct contracts; private native handles remain implementation details.

QscRcs.NonceSize [property]

public static int NonceSize

Purpose: Gets the NonceSize value reported by QSCABI.

QscRcs.KeySize [method]

public static int KeySize(QscRcsMode mode)

Purpose: Returns the key size required by the selected RCS profile.

Parameters: mode: The RCS profile selection.

Returns: The requested size in bytes.

QscRcs.TagSize [method]

public static int TagSize(QscRcsMode mode)

Purpose: Returns the authentication-tag size produced by the selected RCS profile.

Parameters: mode: The RCS profile selection.

Returns: The requested size in bytes.

QscRcs.Encrypt [method]

public static byte[] Encrypt(QscRcsMode mode, ReadOnlySpan<byte> plaintext, ReadOnlySpan<byte> key, ReadOnlySpan<byte> nonce, ReadOnlySpan<byte> info = default, ReadOnlySpan<byte> aad = default)

Purpose: Encrypts plaintext using the selected RCS AEAD profile and returns ciphertext with the tag appended.

Parameters: mode: The RCS profile selection.; plaintext: The plaintext bytes to encrypt.; key: The cryptographic key material. The buffer is not retained.; nonce: The nonce value. Nonces must satisfy the uniqueness requirements of the selected primitive.; info: Optional context or domain-separation information.; aad: Associated data to authenticate but not encrypt.

Returns: A newly allocated ciphertext buffer containing the encrypted data and authentication tag when the mode is AEAD.

QscRcs.Decrypt [method]

public static byte[] Decrypt(QscRcsMode mode, ReadOnlySpan<byte> ciphertext, ReadOnlySpan<byte> key, ReadOnlySpan<byte> nonce, ReadOnlySpan<byte> info = default, ReadOnlySpan<byte> aad = default)

Purpose: Decrypts and authenticates RCS ciphertext using the selected AEAD profile.

Parameters: mode: The RCS profile selection.; ciphertext: The ciphertext buffer, including the authentication tag for AEAD modes.; key: The cryptographic key material. The buffer is not retained.; nonce: The nonce value. Nonces must satisfy the uniqueness requirements of the selected primitive.; info: Optional context or domain-separation information.; aad: Associated data to authenticate but not encrypt.

Returns: A newly allocated plaintext buffer. Authentication failures cause a non-success status or exception depending on the API variant.

QscRcs.TryEncrypt [method]

public static unsafe QscStatus TryEncrypt(QscRcsMode mode, ReadOnlySpan<byte> plaintext, ReadOnlySpan<byte> key, ReadOnlySpan<byte> nonce, ReadOnlySpan<byte> info, ReadOnlySpan<byte> aad, Span<byte> ciphertext, out int written)

Purpose: Attempts to encrypt plaintext using the selected RCS AEAD profile into a caller-provided buffer.

Parameters: mode: The RCS profile selection.; plaintext: The plaintext bytes to encrypt.; key: The cryptographic key material. The buffer is not retained.; nonce: The nonce value. Nonces must satisfy the

uniqueness requirements of the selected primitive.; info: The optional context or domain-separation information.; aad: Associated data to authenticate but not encrypt.; ciphertext: The ciphertext buffer, including the authentication tag for AEAD modes.; written: Receives the number of bytes written to the output buffer.

Returns: QscStatus.Success on success, or a non-success status when validation, authentication, or native execution fails.

QscRcs.TryDecrypt [method]

public static unsafe QscStatus TryDecrypt(QscRcsMode mode, ReadOnlySpan<byte> ciphertext, ReadOnlySpan<byte> key, ReadOnlySpan<byte> nonce, ReadOnlySpan<byte> info, ReadOnlySpan<byte> aad, Span<byte> plaintext, out int written)

Purpose: Attempts to decrypt and authenticate RCS ciphertext into a caller-provided buffer.

Parameters: mode: The RCS profile selection.; ciphertext: The ciphertext buffer, including the authentication tag for AEAD modes.; key: The cryptographic key material. The buffer is not retained.; nonce: The nonce value. Nonces must satisfy the uniqueness requirements of the selected primitive.; info: The optional context or domain-separation information.; aad: Associated data to authenticate but not encrypt.; plaintext: The plaintext bytes to encrypt.; written: Receives the number of bytes written to the output buffer.

Returns: QscStatus.Success on success, or a non-success status when validation, authentication, or native execution fails.

25.36 Symmetric/QscSymmetricConstants.cs (15 members)

Declaration source: QSCCS/Symmetric/QscSymmetricConstants.cs. A throwing operation and a status-returning Try variant are distinct contracts; private native handles remain implementation details.

QscSymmetricConstants.AesBlockSize [property]

public static int AesBlockSize

Purpose: Gets the AesBlockSize value reported by QSCABI.

QscSymmetricConstants.Aes128KeySize [property]

public static int Aes128KeySize

Purpose: Gets the Aes128KeySize value reported by QSCABI.

QscSymmetricConstants.Aes256KeySize [property]

public static int Aes256KeySize

Purpose: Gets the Aes256KeySize value reported by QSCABI.

QscSymmetricConstants.AesGcmNonceSize [property]

public static int AesGcmNonceSize

Purpose: Gets the AesGcmNonceSize value reported by QSCABI.

QscSymmetricConstants.AesGcmMaxNonceSize [property]

public static int AesGcmMaxNonceSize

Purpose: Gets the AesGcmMaxNonceSize value reported by QSCABI.

QscSymmetricConstants.AesGcmTagSize [property]

public static int AesGcmTagSize

Purpose: Gets the AesGcmTagSize value reported by QSCABI.

QscSymmetricConstants.ChaCha20KeySize [property]

public static int ChaCha20KeySize

Purpose: Gets the ChaCha20KeySize value reported by QSCABI.

QscSymmetricConstants.ChaCha20NonceSize [property]

public static int ChaCha20NonceSize

Purpose: Gets the ChaCha20NonceSize value reported by QSCABI.

QscSymmetricConstants.ChaCha20Poly1305TagSize [property]

public static int ChaCha20Poly1305TagSize

Purpose: Gets the ChaCha20Poly1305TagSize value reported by QSCABI.

QscSymmetricConstants.CsxKeySize [property]

public static int CsxKeySize

Purpose: Gets the CsxKeySize value reported by QSCABI.

QscSymmetricConstants.CsxNonceSize [property]

public static int CsxNonceSize

Purpose: Gets the CsxNonceSize value reported by QSCABI.

QscSymmetricConstants.CsxTagSize [property]

public static int CsxTagSize

Purpose: Gets the CsxTagSize value reported by QSCABI.

QscSymmetricConstants.RcsNonceSize [property]

public static int RcsNonceSize

Purpose: Gets the RcsNonceSize value reported by QSCABI.

QscSymmetricConstants.RcsKeySize [method]

public static int RcsKeySize(QscRcsMode mode)

Purpose: Returns the size associated with RcsKeySize.

Parameters: mode: The RCS profile selection.

Returns: The requested size in bytes.

QscSymmetricConstants.RcsTagSize [method]

public static int RcsTagSize(QscRcsMode mode)

Purpose: Returns the size associated with RcsTagSize.

Parameters: mode: The RCS profile selection.

Returns: The requested size in bytes.

25.37 Tls/QscTlsConnection.cs (23 members)

Declaration source: QSCCS/Tls/QscTlsConnection.cs. A throwing operation and a status-returning Try variant are distinct contracts; private native handles remain implementation details.

QscTlsConnection.IsConnected [property]

public bool IsConnected

Purpose: Gets a value indicating whether the native connection is currently connected.

QscTlsConnection.IsHandshakeComplete [property]

public bool IsHandshakeComplete

Purpose: Gets a value indicating whether the TLS handshake is complete.

QscTlsConnection.Create [method]

public static QscTlsConnection Create()

Purpose: Creates an idle TLS connection handle and throws on native failure.

Returns: A managed TLS connection that owns the native handle.

Lifetime: Observe native owner validity and deterministic disposal; no use after close.

QscTlsConnection.TryCreate [method]

public static unsafe QscStatus TryCreate(out QscTlsConnection? connection)

Purpose: Attempts to create an idle TLS connection handle.

Parameters: connection: Receives the created connection on success, or on failure.

Returns: QscStatus.Success on success; otherwise, the native QSCABI status.

Lifetime: Observe native owner validity and deterministic disposal; no use after close.

QscTlsConnection.ConnectHost [method]

public void ConnectHost(QscTlsContext context, string hostname, string service)

Purpose: Connects this connection to a remote host using a configured TLS context.

Parameters: context: The TLS context containing client policy and trust material.; hostname: The DNS host name or address to connect to.; service: The service name or port string accepted by the native resolver.

Lifetime: Observe native owner validity and deterministic disposal; no use after close.

QscTlsConnection.TryConnectHost [method]

public unsafe QscStatus TryConnectHost(QscTlsContext context, string hostname, string service)

Purpose: Attempts to connect this connection to a remote host using a configured TLS context.

Parameters: context: The TLS context containing client policy and trust material.; hostname: The DNS host name or address to connect to.; service: The service name or port string accepted by the native resolver.

Returns: QscStatus.Success on success; otherwise, the native QSCABI status.

QscTlsConnection.Send [method]

public int Send(ReadOnlySpan<byte> input)

Purpose: Sends application data over the TLS connection and throws on native failure.

Parameters: input: The caller-owned plaintext bytes to send.

Returns: The number of bytes accepted by the native TLS send operation.

Lifetime: Observe native owner validity and deterministic disposal; no use after close.

QscTlsConnection.TrySend [method]

public unsafe QscStatus TrySend(ReadOnlySpan<byte> input, out nuint written)

Purpose: Attempts to send application data over the TLS connection.

Parameters: input: The caller-owned plaintext bytes to send.; written: Receives the number of bytes accepted by the native TLS send operation.

Returns: QscStatus.Success on success; otherwise, the native QSCABI status.

QscTlsConnection.Receive [method]

public int Receive(Span<byte> output)

Purpose: Receives application data from the TLS connection and throws on native failure.

Parameters: output: The caller-owned output buffer that receives plaintext bytes.

Returns: The number of bytes written to .

Lifetime: Observe native owner validity and deterministic disposal; no use after close.

QscTlsConnection.TryReceive [method]

public unsafe QscStatus TryReceive(Span<byte> output, out nuint read)

Purpose: Attempts to receive application data from the TLS connection.

Parameters: output: The caller-owned output buffer that receives plaintext bytes.; read: Receives the number of bytes written to .

Returns: QscStatus.Success on success; otherwise, the native QSCABI status.

QscTlsConnection.SendFrame [method]

public void SendFrame(ReadOnlySpan<byte> input)

Purpose: Sends one application frame over the TLS connection and throws on native failure.

Parameters: input: The caller-owned frame payload to send.

Lifetime: Observe native owner validity and deterministic disposal; no use after close.

QscTlsConnection.TrySendFrame [method]

public unsafe QscStatus TrySendFrame(ReadOnlySpan<byte> input)

Purpose: Attempts to send one application frame over the TLS connection.

Parameters: input: The caller-owned frame payload to send.

Returns: QscStatus.Success on success; otherwise, the native QSCABI status.

QscTlsConnection.ReceiveFrame [method]

public int ReceiveFrame(Span<byte> output)

Purpose: Receives one application frame from the TLS connection and throws on native failure.

Parameters: output: The caller-owned buffer that receives the frame payload.

Returns: The number of bytes written to .

Lifetime: Observe native owner validity and deterministic disposal; no use after close.

QscTlsConnection.TryReceiveFrame [method]

public unsafe QscStatus TryReceiveFrame(Span<byte> output, out nuint read)

Purpose: Attempts to receive one application frame from the TLS connection.

Parameters: output: The caller-owned buffer that receives the frame payload.; read: Receives the number of bytes written to .

Returns: QscStatus.Success on success; otherwise, the native QSCABI status.

QscTlsConnection.Shutdown [method]

public void Shutdown()

Purpose: Performs a graceful TLS shutdown and throws on native failure.

Lifetime: Observe native owner validity and deterministic disposal; no use after close.

QscTlsConnection.TryShutdown [method]

public QscStatus TryShutdown()

Purpose: Attempts to perform a graceful TLS shutdown.

Returns: QscStatus.Success on success; otherwise, the native QSCABI status.

QscTlsConnection.Cancel [method]

public void Cancel()

Purpose: Cancels the connection and throws on native failure.

Lifetime: Observe native owner validity and deterministic disposal; no use after close.

QscTlsConnection.TryCancel [method]

public QscStatus TryCancel()

Purpose: Attempts to cancel the connection.

Returns: QscStatus.Success on success; otherwise, the native QSCABI status.

QscTlsConnection.GetSelectedAlpn [method]

public string GetSelectedAlpn()

Purpose: Gets the selected ALPN protocol and throws on native failure.

Returns: The selected ALPN protocol, or an empty string if no protocol is available.

QscTlsConnection.TryGetSelectedAlpn [method]

public unsafe QscStatus TryGetSelectedAlpn(out string selectedAlpn)

Purpose: Attempts to get the selected ALPN protocol.

Parameters: selectedAlpn: Receives the selected ALPN protocol, or an empty string when unavailable.

Returns: QscStatus.Success on success; otherwise, the native QSCABI status.

QscTlsConnection.GetPeerInfo [method]

public QscTlsPeerInfo GetPeerInfo()

Purpose: Gets peer and negotiated-session information and throws on native failure.

Returns: A managed peer-information snapshot.

QscTlsConnection.TryGetPeerInfo [method]

public unsafe QscStatus TryGetPeerInfo(out QscTlsPeerInfo? peerInfo)

Purpose: Attempts to get peer and negotiated-session information.

Parameters: peerInfo: Receives a managed peer-information snapshot on success, or on failure.

Returns: QscStatus.Success on success; otherwise, the native QSCABI status.

QscTlsConnection.Dispose [method]

public void Dispose()

Purpose: Releases the native TLS connection handle owned by this instance.

Lifetime: Observe native owner validity and deterministic disposal; no use after close.

25.38 Tls/QscTlsContext.cs (30 members)

Declaration source: QSCCS/Tls/QscTlsContext.cs. A throwing operation and a status-returning Try variant are distinct contracts; private native handles remain implementation details.

QscTlsContext.Create [method]

public static QscTlsContext Create()

Purpose: Creates a new TLS context and throws if the native context cannot be allocated.

Returns: A managed TLS context that owns the newly allocated native handle.

Lifetime: Observe native owner validity and deterministic disposal; no use after close.

QscTlsContext.TryCreate [method]

public static unsafe QscStatus TryCreate(out QscTlsContext? context)

Purpose: Attempts to create a new TLS context without throwing on QSCABI failure.

Parameters: context: Receives the created context on success, or on failure.

Returns: QscStatus.Success on success; otherwise, the native QSCABI status.

Lifetime: Observe native owner validity and deterministic disposal; no use after close.

QscTlsContext.SetPolicy [method]

public void SetPolicy(QscTlsPolicy policy)

Purpose: Applies a predefined QSCABI TLS policy to this context.

Parameters: policy: The policy selector to apply.

QscTlsContext.TrySetPolicy [method]

public QscStatus TrySetPolicy(QscTlsPolicy policy)

Purpose: Attempts to apply a predefined QSCABI TLS policy to this context.

Parameters: policy: The policy selector to apply.

Returns: QscStatus.Success on success; otherwise, the native QSCABI status.

QscTlsContext.SetSocketOptions [method]

public unsafe void SetSocketOptions(QscTlsOptions options)

Purpose: Applies socket options to this context and throws on native failure.

Parameters: options: The managed socket-options object to copy into the native context.

QscTlsContext.TrySetSocketOptions [method]

public unsafe QscStatus TrySetSocketOptions(QscTlsOptions options)

Purpose: Attempts to apply socket options to this context without throwing on QSCABI failure.

Parameters: options: The managed socket-options object to copy into the native context.

Returns: QscStatus.Success on success; otherwise, the native QSCABI status.

QscTlsContext.SetAlpnProtocols [method]

public void SetAlpnProtocols(IReadOnlyList<string> protocols, bool required)

Purpose: Configures the ALPN protocols offered or accepted by this context.

Parameters: protocols: The ordered ALPN protocol identifiers. Each value must be non-empty and non-whitespace.; required: A value indicating whether ALPN selection is mandatory for the handshake.

QscTlsContext.TrySetAlpnProtocols [method]

public unsafe QscStatus TrySetAlpnProtocols(IReadOnlyList<string> protocols, bool required)

Purpose: Attempts to configure ALPN protocols without throwing on QSCABI failure.

Parameters: protocols: The ordered ALPN protocol identifiers. Each value must be non-empty and non-whitespace.; required: A value indicating whether ALPN selection is mandatory for the handshake.

Returns: QscStatus.Success on success; QscStatus.InvalidArgument for empty managed input; otherwise, the native QSCABI status.

QscTlsContext.ClearAlpnProtocols [method]

public void ClearAlpnProtocols()

Purpose: Clears all ALPN protocol configuration from this context and throws on native failure.

Lifetime: Observe native owner validity and deterministic disposal; no use after close.

QscTlsContext.TryClearAlpnProtocols [method]

public QscStatus TryClearAlpnProtocols()

Purpose: Attempts to clear all ALPN protocol configuration from this context.

Returns: QscStatus.Success on success; otherwise, the native QSCABI status.

QscTlsContext.SetSniRequired [method]

public void SetSniRequired(bool required)

Purpose: Configures whether Server Name Indication is required for handshakes using this context.

Parameters: required: to require SNI; otherwise, .

QscTlsContext.TrySetSniRequired [method]

public QscStatus TrySetSniRequired(bool required)

Purpose: Attempts to configure whether Server Name Indication is required.

Parameters: required: to require SNI; otherwise, .

Returns: QscStatus.Success on success; otherwise, the native QSCABI status.

QscTlsContext.SetClientAuthentication [method]

public void SetClientAuthentication(bool requestClientAuthentication, bool requireClientAuthentication)

Purpose: Configures whether this context requests and requires peer client certificates.

Parameters: requestClientAuthentication: to request a client certificate during handshake.; requireClientAuthentication: to fail the handshake if the client does not authenticate.

QscTlsContext.TrySetClientAuthentication [method]

public QscStatus TrySetClientAuthentication(bool requestClientAuthentication, bool requireClientAuthentication)

Purpose: Attempts to configure client-authentication behavior for this context.

Parameters: requestClientAuthentication: to request a client certificate during handshake.; requireClientAuthentication: to fail the handshake if the client does not authenticate.

Returns: QscStatus.Success on success; otherwise, the native QSCABI status.

QscTlsContext.SetTicketPolicy [method]

public unsafe void SetTicketPolicy(QscTlsTicketPolicy policy)

Purpose: Applies a TLS session-ticket policy to this context.

Parameters: policy: The managed ticket policy to copy into the native context.

QscTlsContext.TrySetTicketPolicy [method]

public unsafe QscStatus TrySetTicketPolicy(QscTlsTicketPolicy policy)

Purpose: Attempts to apply a TLS session-ticket policy to this context.

Parameters: policy: The managed ticket policy to copy into the native context.

Returns: QscStatus.Success on success; otherwise, the native QSCABI status.

QscTlsContext.LoadTrustAnchorPem [method]

public void LoadTrustAnchorPem(string pem, bool selfSigned)

Purpose: Loads a PEM-encoded trust anchor into this TLS context and throws on native failure.

Parameters: pem: The PEM-encoded certificate to use as a trust anchor.; selfSigned: Indicates whether the trust anchor is expected to be self-signed.

Lifetime: Observe native owner validity and deterministic disposal; no use after close.

QscTlsContext.TryLoadTrustAnchorPem [method]

public QscStatus TryLoadTrustAnchorPem(string pem, bool selfSigned)

Purpose: Attempts to load a PEM-encoded trust anchor into this TLS context.

Parameters: pem: The PEM-encoded certificate to use as a trust anchor.; selfSigned: Indicates whether the trust anchor is expected to be self-signed.

Returns: QscStatus.Unsupported for ABI 0.10.

Lifetime: Observe native owner validity and deterministic disposal; no use after close.

QscTlsContext.LoadServerIdentityPem [method]

public void LoadServerIdentityPem(string chainPem, string privateKeyPem, QscTlsSignatureScheme signatureScheme)

Purpose: Loads a PEM-encoded server certificate chain and private key into this context.

Parameters: chainPem: The PEM-encoded server certificate chain.; privateKeyPem: The PEM-encoded private key corresponding to the server certificate.; signatureScheme: The signature scheme associated with the supplied identity material.

Lifetime: Observe native owner validity and deterministic disposal; no use after close.

QscTlsContext.TryLoadServerIdentityPem [method]

public QscStatus TryLoadServerIdentityPem(string chainPem, string privateKeyPem, QscTlsSignatureScheme signatureScheme)

Purpose: Attempts to load a PEM-encoded server certificate chain and private key into this context.

Parameters: chainPem: The PEM-encoded server certificate chain.; privateKeyPem: The PEM-encoded private key corresponding to the server certificate.; signatureScheme: The signature scheme associated with the supplied identity material.

Returns: QscStatus.Unsupported for ABI 0.10, without copying key material.

Lifetime: Observe native owner validity and deterministic disposal; no use after close.

QscTlsContext.SetCipherSuites [method]

public unsafe void SetCipherSuites(ReadOnlySpan<ushort> suites)

Purpose: Releases the native TLS context handle owned by this instance.

Parameters: suites: One or more unique TLS wire IDs such as 0x1301 and 0x1302.

Lifetime: Observe native owner validity and deterministic disposal; no use after close.

QscTlsContext.SetNamedGroups [method]

public unsafe void SetNamedGroups(ReadOnlySpan<ushort> groups)

Purpose: Set ordered TLS named-group wire identifiers.

Parameters: groups: Unique groups, such as secp256r1 (0x0017).

QscTlsContext.SetSignatureIds [method]

public unsafe void SetSignatureIds(ReadOnlySpan<ushort> schemes)

Purpose: Set ordered TLS CertificateVerify signature-algorithm wire IDs.

Parameters: schemes: Unique TLS wire IDs, such as ECDSA P-256 SHA-256 (0x0403).

QscTlsContext.LoadTrustAnchorFile [method]

public unsafe void LoadTrustAnchorFile(string path, bool selfSigned = true)

Purpose: Load a trusted PEM certificate from a caller-authorized file.

Parameters: path: UTF-8 file path containing a certificate.; selfSigned: Whether to allow a self-signed trust anchor.

QscTlsContext.LoadTrustAnchorBundleFile [method]

public unsafe void LoadTrustAnchorBundleFile(string path, bool selfSigned = true)

Purpose: Load a PEM trust-anchor bundle with native certificate parsing.

Parameters: path: Authorized PEM bundle path.; selfSigned: Whether self-signed root anchors are permitted.

QscTlsContext.LoadCrlFile [method]

public unsafe void LoadCrlFile(string path)

Purpose: Load CRL evidence into the native TLS policy from a file.

Parameters: path: Authorized CRL file path.

QscTlsContext.LoadServerIdentityFiles [method]

public unsafe void LoadServerIdentityFiles(string chainPath, string keyPath, QscTlsSignatureScheme scheme)

Purpose: Load a server certificate-chain file and corresponding native private-key file.

Parameters: chainPath: Certificate chain file path.; keyPath: Protected matching private-key file path.; scheme: Native certificate signature scheme selector.

QscTlsContext.LoadClientIdentityFiles [method]

public unsafe void LoadClientIdentityFiles(string chainPath, string keyPath, QscTlsSignatureScheme scheme)

Purpose: Load a client certificate-chain file and its protected private-key file.

Parameters: chainPath: Client chain file path.; keyPath: Protected matching client private-key file path.; scheme: Native signature scheme selector matching the certificate.

QscTlsContext.AddServerIdentityFiles [method]

public unsafe void AddServerIdentityFiles(string hostname, string chainPath, string keyPath, QscTlsSignatureScheme scheme)

Purpose: Register a hostname-specific SNI server chain and key from native files.

Parameters: hostname: Nonempty ASCII/UTF-8 DNS name, without NUL.; chainPath: Certificate chain file path.; keyPath: Matching protected private key path.; scheme: Native signature scheme for the provisioned key.

QscTlsContext.Dispose [method]

public void Dispose()

Purpose: Source-declared public method; consult implementation for its precise contract.

25.39 Tls/QscTlsListener.cs (10 members)

Declaration source: QSCCS/Tls/QscTlsListener.cs. A throwing operation and a status-returning Try variant are distinct contracts; private native handles remain implementation details.

QscTlsListener.Create [method]

public static QscTlsListener Create()

Purpose: Creates a TLS listener handle and throws on native failure.

Returns: A managed TLS listener that owns the native handle.

Lifetime: Observe native owner validity and deterministic disposal; no use after close.

QscTlsListener.TryCreate [method]

public static unsafe QscStatus TryCreate(out QscTlsListener? listener)

Purpose: Attempts to create a TLS listener handle without throwing on QSCABI failure.

Parameters: listener: Receives the created listener on success, or on failure.

Returns: QscStatus.Success on success; otherwise, the native QSCABI status.

Lifetime: Observe native owner validity and deterministic disposal; no use after close.

QscTlsListener.SetSocketOptions [method]

public unsafe void SetSocketOptions(QscTlsOptions options)

Purpose: Applies socket options to this listener and throws on native failure.

Parameters: options: The managed socket-options object to copy to the native listener.

QscTlsListener.TrySetSocketOptions [method]

public unsafe QscStatus TrySetSocketOptions(QscTlsOptions options)

Purpose: Attempts to apply socket options to this listener.

Parameters: options: The managed socket-options object to copy to the native listener.

Returns: QscStatus.Success on success; otherwise, the native QSCABI status.

QscTlsListener.Bind [method]

public void Bind(QscTlsContext context, string address, ushort port, QscTlsAddressFamily family)

Purpose: Binds this listener to a local address and port using the supplied TLS context.

Parameters: context: The configured server TLS context.; address: The local address to bind, such as 127.0.0.1 or ::1.; port: The TCP port to bind.; family: The address family to use for the native socket.

Lifetime: Observe native owner validity and deterministic disposal; no use after close.

QscTlsListener.TryBind [method]

public unsafe QscStatus TryBind(QscTlsContext context, string address, ushort port, QscTlsAddressFamily family)

Purpose: Attempts to bind this listener to a local address and port using the supplied TLS context.

Parameters: context: The configured server TLS context.; address: The local address to bind, such as 127.0.0.1 or ::1.; port: The TCP port to bind.; family: The address family to use for the native socket.

Returns: QscStatus.Success on success; otherwise, the native QSCABI status.

QscTlsListener.Accept [method]

public QscTlsConnection Accept()

Purpose: Accepts an inbound TLS connection and throws on native failure.

Returns: A managed TLS connection containing the accepted native session state.

Lifetime: Observe native owner validity and deterministic disposal; no use after close.

QscTlsListener.TryAccept [method]

public QscStatus TryAccept(out QscTlsConnection? connection)

Purpose: Attempts to accept an inbound TLS connection.

Parameters: connection: Receives the accepted connection on success, or on failure.

Returns: QscStatus.Success on success; otherwise, the native QSCABI status.

QscTlsListener.Close [method]

public void Close()

Purpose: Requests native listener closure without disposing the managed wrapper.

Lifetime: Observe native owner validity and deterministic disposal; no use after close.

QscTlsListener.Dispose [method]

public void Dispose()

Purpose: Closes the native listener and releases the native handle.

Lifetime: Observe native owner validity and deterministic disposal; no use after close.

25.40 Tls/QscTlsOptions.cs (13 members)

Declaration source: QSCCS/Tls/QscTlsOptions.cs. A throwing operation and a status-returning Try variant are distinct contracts; private native handles remain implementation details.

QscTlsOptions.ConnectTimeoutMs [property]

public uint ConnectTimeoutMs

Purpose: Gets or sets the socket connection timeout, in milliseconds.

QscTlsOptions.HandshakeTimeoutMs [property]

public uint HandshakeTimeoutMs

Purpose: Gets or sets the TLS handshake timeout, in milliseconds.

QscTlsOptions.ReceiveTimeoutMs [property]

public uint ReceiveTimeoutMs

Purpose: Gets or sets the receive timeout, in milliseconds.

QscTlsOptions.SendTimeoutMs [property]

public uint SendTimeoutMs

Purpose: Gets or sets the send timeout, in milliseconds.

QscTlsOptions.IdleTimeoutMs [property]

public uint IdleTimeoutMs

Purpose: Gets or sets the idle timeout, in milliseconds.

QscTlsOptions.ReceiveBufferSize [property]

public nuint ReceiveBufferSize

Purpose: Gets or sets the native socket receive-buffer size.

QscTlsOptions.SendBufferSize [property]

public nuint SendBufferSize

Purpose: Gets or sets the native socket send-buffer size.

QscTlsOptions.ReuseAddress [property]

public bool ReuseAddress

Purpose: Gets or sets a value indicating whether the native socket may reuse a local address.

QscTlsOptions.NoDelay [property]

public bool NoDelay

Purpose: Gets or sets a value indicating whether Nagle's algorithm should be disabled.

QscTlsOptions.KeepAlive [property]

public bool KeepAlive

Purpose: Gets or sets a value indicating whether TCP keepalive should be enabled.

QscTlsOptions.DualStack [property]

public bool DualStack

Purpose: Gets or sets a value indicating whether dual-stack IPv4 and IPv6 behavior should be requested.

QscTlsOptions.Blocking [property]

public bool Blocking

Purpose: Gets or sets a value indicating whether sockets should use blocking mode.

QscTlsOptions.Default [property]

public static unsafe QscTlsOptions Default

Purpose: Gets the native default TLS socket options.

25.41 Tls/QscTlsPeerInfo.cs (15 members)

Declaration source: QSCCS/Tls/QscTlsPeerInfo.cs. A throwing operation and a status-returning Try variant are distinct contracts; private native handles remain implementation details.

QscTlsPeerInfo.Subject [property]

public string Subject

Purpose: Gets the peer certificate subject string reported by the native TLS layer.

QscTlsPeerInfo.Issuer [property]

public string Issuer

Purpose: Gets the peer certificate issuer string reported by the native TLS layer.

QscTlsPeerInfo.CommonName [property]

public string CommonName

Purpose: Gets the peer certificate common name reported by the native TLS layer.

QscTlsPeerInfo.DnsName [property]

public string DnsName

Purpose: Gets a peer DNS name reported by the native TLS layer.

QscTlsPeerInfo.SelectedAlpn [property]

public string SelectedAlpn

Purpose: Gets the selected ALPN protocol.

QscTlsPeerInfo.CipherSuite [property]

public uint CipherSuite

Purpose: Gets the numeric TLS cipher-suite identifier reported by QSCABI.

QscTlsPeerInfo.NamedGroup [property]

public uint NamedGroup

Purpose: Gets the numeric named-group identifier reported by QSCABI.

QscTlsPeerInfo.SignatureScheme [property]

public uint SignatureScheme

Purpose: Gets the numeric signature-scheme identifier reported by QSCABI.

QscTlsPeerInfo.Authenticated [property]

public bool Authenticated

Purpose: Gets a value indicating whether the peer was authenticated by the native TLS policy.

QscTlsPeerInfo.HostnameMatched [property]

public bool HostnameMatched

Purpose: Gets a value indicating whether the peer hostname matched the configured hostname policy.

QscTlsPeerInfo.HostnameChecked [property]

public bool HostnameChecked

Purpose: Gets a value indicating whether hostname verification was actually performed.

QscTlsPeerInfo.ChainValid [property]

public bool ChainValid

Purpose: Gets a value indicating whether the peer certificate chain was valid under configured trust.

QscTlsPeerInfo.PskAccepted [property]

public bool PskAccepted

Purpose: Gets a value indicating whether a pre-shared-key path was accepted by the native session.

QscTlsPeerInfo.EarlyDataAccepted [property]

public bool EarlyDataAccepted

Purpose: Gets a value indicating whether early data was accepted by the native session.

QscTlsPeerInfo.AlpnSelected [property]

public bool AlpnSelected

Purpose: Gets a value indicating whether ALPN negotiation selected a protocol.

25.42 Tls/QscTlsServer.cs (12 members)

Declaration source: QSCCS/Tls/QscTlsServer.cs. A throwing operation and a status-returning Try variant are distinct contracts; private native handles remain implementation details.

QscTlsServer.Create [method]

public static QscTlsServer Create()

Purpose: Creates a native TLS server handle and throws on native failure.

Returns: A managed server wrapper that owns the native handle.

Lifetime: Observe native owner validity and deterministic disposal; no use after close.

QscTlsServer.TryCreate [method]

public static unsafe QscStatus TryCreate(out QscTlsServer? server)

Purpose: Attempts to create a native TLS server handle.

Parameters: server: Receives the created server on success, or on failure.

Returns: QscStatus.Success on success; otherwise, the native QSCABI status.

Lifetime: Observe native owner validity and deterministic disposal; no use after close.

QscTlsServer.Configure [method]

public void Configure(QscTlsContext context, string address, ushort port, QscTlsAddressFamily family)

Purpose: Configures this server with a TLS context, local address, port, and address family.

Parameters: context: The configured server TLS context.; address: The local address to bind.; port: The TCP port to bind.; family: The address family to use for the native server socket.

Lifetime: Observe native owner validity and deterministic disposal; no use after close.

QscTlsServer.TryConfigure [method]

public unsafe QscStatus TryConfigure(QscTlsContext context, string address, ushort port, QscTlsAddressFamily family)

Purpose: Attempts to configure this server with a TLS context, local address, port, and address family.

Parameters: context: The configured server TLS context.; address: The local address to bind.; port: The TCP port to bind.; family: The address family to use for the native server socket.

Returns: QscStatus.Success on success; otherwise, the native QSCABI status.

QscTlsServer.SetMaxClients [method]

public void SetMaxClients(nuint maxClients)

Purpose: Sets the maximum number of clients accepted by the native server.

Parameters: maxClients: The maximum number of concurrent or queued clients permitted by native policy.

QscTlsServer.TrySetMaxClients [method]

public QscStatus TrySetMaxClients(nuint maxClients)

Purpose: Attempts to set the maximum number of clients accepted by the native server.

Parameters: maxClients: The maximum number of concurrent or queued clients permitted by native policy.

Returns: QscStatus.Success on success; otherwise, the native QSCABI status.

QscTlsServer.Start [method]

public void Start()

Purpose: Starts the native TLS server using the configured server state.

Lifetime: Observe native owner validity and deterministic disposal; no use after close.

QscTlsServer.TryStart [method]

public QscStatus TryStart()

Purpose: Attempts to start the native TLS server using the configured server state.

Returns: QscStatus.Success on success; otherwise, the native QSCABI status.

QscTlsServer.StartConcurrent [method]

public void StartConcurrent()

Purpose: Starts the native TLS server in concurrent mode.

Lifetime: Observe native owner validity and deterministic disposal; no use after close.

QscTlsServer.TryStartConcurrent [method]

public QscStatus TryStartConcurrent()

Purpose: Attempts to start the native TLS server in concurrent mode.

Returns: QscStatus.Success on success; otherwise, the native QSCABI status.

QscTlsServer.Stop [method]

public void Stop()

Purpose: Requests that the native server stop accepting and processing clients.

Lifetime: Observe native owner validity and deterministic disposal; no use after close.

QscTlsServer.Dispose [method]

public void Dispose()

Purpose: Stops the native TLS server and releases its native handle.

Lifetime: Observe native owner validity and deterministic disposal; no use after close.

25.43 Tls/QscTlsTicketPolicy.cs (6 members)

Declaration source: QSCCS/Tls/QscTlsTicketPolicy.cs. A throwing operation and a status-returning Try variant are distinct contracts; private native handles remain implementation details.

QscTlsTicketPolicy.Enabled [property]

public bool Enabled

Purpose: Gets or sets a value indicating whether session tickets are enabled.

QscTlsTicketPolicy.AllowEarlyData [property]

public bool AllowEarlyData

Purpose: Gets or sets a value indicating whether early data may be accepted with resumed sessions.

QscTlsTicketPolicy.AutoSendServerTicket [property]

public bool AutoSendServerTicket

Purpose: Gets or sets a value indicating whether a server should automatically send a ticket after handshake.

QscTlsTicketPolicy.LifetimeSeconds [property]

public uint LifetimeSeconds

Purpose: Gets or sets the ticket lifetime, in seconds.

QscTlsTicketPolicy.RenewalIntervalSeconds [property]

public uint RenewalIntervalSeconds

Purpose: Gets or sets the ticket-renewal interval, in seconds.

QscTlsTicketPolicy.Default [property]

public static unsafe QscTlsTicketPolicy Default

Purpose: Gets the native default TLS session-ticket policy.

25.44 X509/QscX509Certificate.cs (18 members)

Declaration source: QSCCS/X509/QscX509Certificate.cs. A throwing operation and a status-returning Try variant are distinct contracts; private native handles remain implementation details.

QscX509Certificate.Subject [property]

public string Subject

Purpose: Gets the certificate subject distinguished-name string.

QscX509Certificate.Issuer [property]

public string Issuer

Purpose: Gets the certificate issuer distinguished-name string.

QscX509Certificate.SubjectCommonName [property]

public string SubjectCommonName

Purpose: Gets the certificate subject common-name value.

QscX509Certificate.DnsNameCount [property]

public nuint DnsNameCount

Purpose: Gets the number of DNS subject-alternative-name entries in the certificate.

QscX509Certificate.LoadMemory [method]

public static QscX509Certificate LoadMemory(ReadOnlySpan<byte> data)

Purpose: Loads an X.509 certificate from a caller-supplied memory buffer.

Parameters: data: The encoded certificate bytes. The buffer may contain a DER certificate, a PEM certificate, or another certificate memory encoding accepted by QSCABI.

Returns: A managed certificate object that owns the native certificate handle.

Lifetime: Observe native owner validity and deterministic disposal; no use after close.

QscX509Certificate.TryLoadMemory [method]

public static unsafe QscStatus TryLoadMemory(ReadOnlySpan<byte> data, out QscX509Certificate? certificate)

Purpose: Attempts to load an X.509 certificate from a caller-supplied memory buffer.

Parameters: data: The encoded certificate bytes. The buffer is read during the call and is not retained.; certificate: On success, receives a managed certificate object that owns the native handle. On failure, receives .

Returns: QscStatus.Success on success; otherwise, a QSCABI status indicating why the certificate could not be created or parsed.

Lifetime: Observe native owner validity and deterministic disposal; no use after close.

QscX509Certificate.LoadPem [method]

public static QscX509Certificate LoadPem(string pem)

Purpose: Loads an X.509 certificate from PEM text.

Parameters: pem: The PEM-armored certificate text, including the BEGIN and END certificate markers.

Returns: A managed certificate object that owns the native certificate handle.

Lifetime: Observe native owner validity and deterministic disposal; no use after close.

QscX509Certificate.TryLoadPem [method]

public static unsafe QscStatus TryLoadPem(string pem, out QscX509Certificate? certificate)

Purpose: Attempts to load an X.509 certificate from PEM text.

Parameters: pem: The PEM-armored certificate text, including the BEGIN and END certificate markers.; certificate: On success, receives a managed certificate object that owns the native handle. On failure, receives .

Returns: QscStatus.Success on success; otherwise, a QSCABI parse or allocation status.

Lifetime: Observe native owner validity and deterministic disposal; no use after close.

QscX509Certificate.LoadDer [method]

public static QscX509Certificate LoadDer(ReadOnlySpan<byte> der)

Purpose: Loads an X.509 certificate from DER bytes.

Parameters: der: The Distinguished Encoding Rules certificate bytes.

Returns: A managed certificate object that owns the native certificate handle.

Lifetime: Observe native owner validity and deterministic disposal; no use after close.

QscX509Certificate.TryLoadDer [method]

public static unsafe QscStatus TryLoadDer(ReadOnlySpan<byte> der, out QscX509Certificate? certificate)

Purpose: Attempts to load an X.509 certificate from DER bytes.

Parameters: der: The Distinguished Encoding Rules certificate bytes. The buffer is read during the call and is not retained.; certificate: On success, receives a managed certificate object that owns the native handle. On failure, receives .

Returns: QscStatus.Success on success; otherwise, a QSCABI parse or allocation status.

Lifetime: Observe native owner validity and deterministic disposal; no use after close.

QscX509Certificate.ExportDer [method]

public byte[] ExportDer()

Purpose: Exports this certificate as DER bytes.

Returns: A managed byte array containing the DER-encoded certificate.

QscX509Certificate.ExportPem [method]

public string ExportPem()

Purpose: Exports this certificate as PEM text.

Returns: A UTF-8 decoded PEM string containing the certificate armor and base64 body.

QscX509Certificate.GetDnsName [method]

public unsafe string GetDnsName(nuint index)

Purpose: Gets a DNS subject-alternative-name entry by index.

Parameters: index: The zero-based DNS SAN index. Valid values are less than DnsNameCount.

Returns: The DNS SAN entry at .

QscX509Certificate.MatchesHostname [method]

public bool MatchesHostname(string hostname)

Purpose: Checks whether this certificate matches the supplied host name.

Parameters: hostname: The DNS host name or address string to validate against the certificate identity.

Returns: if QSCABI accepts the hostname for this certificate; otherwise, .

QscX509Certificate.TryMatchHostname [method]

public unsafe QscStatus TryMatchHostname(string hostname)

Purpose: Attempts to match this certificate against the supplied host name.

Parameters: hostname: The DNS host name or address string to validate against the certificate identity.

Returns: QscStatus.Success when the hostname is accepted; otherwise, a QSCABI status describing the validation failure.

QscX509Certificate.CheckRole [method]

public bool CheckRole(QscX509CertificateRole role, string? hostname = null)

Purpose: Checks whether this certificate satisfies the requested role.

Parameters: role: The certificate role expected by the caller.; hostname: Optional hostname used by role checks that include server identity validation.

Returns: if the certificate satisfies the requested role; otherwise, .

QscX509Certificate.TryCheckRole [method]

public unsafe QscStatus TryCheckRole(QscX509CertificateRole role, string? hostname = null)

Purpose: Attempts to check whether this certificate satisfies the requested role.

Parameters: role: The certificate role expected by the caller.; hostname: Optional hostname used by role checks that include server identity validation.

Returns: QscStatus.Success when the role is accepted; otherwise, a QSCABI status describing the validation failure.

QscX509Certificate.Dispose [method]

public void Dispose()

Purpose: Releases the native certificate handle owned by this instance.

Lifetime: Observe native owner validity and deterministic disposal; no use after close.

25.45 X509/QscX509Chain.cs (7 members)

Declaration source: QSCCS/X509/QscX509Chain.cs. A throwing operation and a status-returning Try variant are distinct contracts; private native handles remain implementation details.

QscX509Chain.Count [property]

public nuint Count

Purpose: Gets the number of certificates currently loaded into the chain.

QscX509Chain.Create [method]

public static QscX509Chain Create(nuint certificateCapacity)

Purpose: Creates an empty certificate chain with the requested certificate capacity.

Parameters: certificateCapacity: Maximum number of certificates the native chain should be prepared to hold.

Returns: A managed certificate-chain object that owns the native handle.

Lifetime: Observe native owner validity and deterministic disposal; no use after close.

QscX509Chain.TryCreate [method]

public static unsafe QscStatus TryCreate(nuint certificateCapacity, out QscX509Chain? chain)

Purpose: Attempts to create an empty certificate chain with the requested certificate capacity.

Parameters: certificateCapacity: Maximum number of certificates the native chain should be prepared to hold.; chain: On success, receives a managed chain object that owns the native handle. On failure, receives .

Returns: QscStatus.Success on success; otherwise, a QSCABI allocation or argument status.

Lifetime: Observe native owner validity and deterministic disposal; no use after close.

QscX509Chain.LoadMemory [method]

public static QscX509Chain LoadMemory(ReadOnlySpan<byte> data, nuint certificateCapacity)

Purpose: Loads a certificate chain from a caller-supplied memory buffer.

Parameters: data: The encoded certificate-chain bytes. The buffer may contain concatenated PEM certificates or another chain memory encoding accepted by QSCABI.; certificateCapacity: Maximum number of certificates expected in the chain.

Returns: A managed certificate-chain object that owns the native handle.

Lifetime: Observe native owner validity and deterministic disposal; no use after close.

QscX509Chain.LoadPem [method]

public static QscX509Chain LoadPem(string pem, nuint certificateCapacity)

Purpose: Loads a certificate chain from PEM text.

Parameters: pem: One or more PEM-armored certificates, commonly ordered from leaf toward issuer.; certificateCapacity: Maximum number of certificates expected in the chain.

Returns: A managed certificate-chain object that owns the native handle.

Lifetime: Observe native owner validity and deterministic disposal; no use after close.

QscX509Chain.TryLoadMemory [method]

public static unsafe QscStatus TryLoadMemory(ReadOnlySpan<byte> data, nuint certificateCapacity, out QscX509Chain? chain)

Purpose: Attempts to load a certificate chain from a caller-supplied memory buffer.

Parameters: data: The encoded certificate-chain bytes. The buffer is read during the call and is not retained.; certificateCapacity: Maximum number of certificates expected in the chain.; chain: On success, receives a managed chain object that owns the native handle. On failure, receives .

Returns: QscStatus.Success on success; otherwise, a QSCABI parse, allocation, or argument status.

Lifetime: Observe native owner validity and deterministic disposal; no use after close.

QscX509Chain.Dispose [method]

public void Dispose()

Purpose: Releases the native certificate-chain handle owned by this instance.

Lifetime: Observe native owner validity and deterministic disposal; no use after close.

25.46 X509/QscX509Issuance.cs (16 members)

Declaration source: QSCCS/X509/QscX509Issuance.cs. A throwing operation and a status-returning Try variant are distinct contracts; private native handles remain implementation details.

QscP256DerSigner.QscP256DerSigner [constructor]

public QscP256DerSigner(Func<byte[], byte[]> sign)

Purpose: Bind an authorized external signer returning ASN.1 DER-encoded ECDSA-P256-SHA256.

Parameters: sign: Callback receives a caller-owned TBS copy and returns DER ECDSA signature bytes.

QscP256DerSigner.FromEcdsa [method]

public static QscP256DerSigner FromEcdsa(ECDsa privateKey)

Purpose: Construct a signer from an existing authorized P-256 ECDsa object.

Parameters: privateKey: Caller-owned key; must remain live until all signing calls complete.

Returns: Non-owning callback adapter; does not dispose the input ECDsa.

QscX509Csr.QscX509Csr [constructor]

public QscX509Csr()

Purpose: Create an empty native CSR handle with checked QSCABI ownership.

QscX509Csr.InitializeP256 [method]

public void InitializeP256(string commonName, ReadOnlySpan<byte> publicKey)

Purpose: Initialize a P-256 subject using exactly 64 raw x||y public-key bytes.

Parameters: commonName: Nonempty common name.; publicKey: Borrowed raw unprefixed 64-byte P-256 point.

QscX509Csr.AddDns [method]

public void AddDns(string dnsName)

Purpose: Add a DNS subject alternative name to this CSR.

Parameters: dnsName: Validated nonempty DNS name.

QscX509Csr.LoadDer [method]

public void LoadDer(ReadOnlySpan<byte> der)

Purpose: Decode a signed PKCS#10 request from borrowed DER bytes.

Parameters: der: Nonempty CSR DER, never retained by managed code.

QscX509Csr.Verify [method]

public QscStatus Verify()

Purpose: Verify the CSR signature against its embedded public key.

Returns: Native cryptographic verification status; success does not establish domain trust.

QscX509Csr.SignDer [method]

public byte[] SignDer(QscP256DerSigner signer, int capacity = 16384)

Purpose: Sign a prepared CSR through an externally owned synchronous ECDSA-P256 signer.

Parameters: signer: Live authorized signer that returns DER ECDSA signature bytes.; capacity: DER output buffer capacity; at most 1 MiB.

Returns: New managed CSR DER bytes.

QscX509Csr.IssueTlsServerP256 [method]

public byte[] IssueTlsServerP256(QscX509Certificate issuer, ReadOnlySpan<byte> serial, DateTimeOffset notBefore, DateTimeOffset notAfter, QscP256DerSigner signer, int capacity = 16384)

Purpose: Issue a signed P-256 TLS server certificate after caller authorization of CSR and issuer.

Parameters: issuer: Authorized live issuer certificate; private issuer key is not exposed.; serial: Unique, positive issuer-specific serial bytes.; notBefore: Inclusive UTC validity beginning.; notAfter: Exclusive UTC validity end.; signer: Issuer's externally owned P-256 private-key DER signer.; capacity: DER output maximum in bytes.

Returns: Encoded new leaf certificate; issuance alone is not trust validation.

QscX509Csr.Dispose [method]

public void Dispose()

Purpose: Destroy native CSR; repeated disposal is safe.

QscX509CrlBuilder.QscX509CrlBuilder [constructor]

public QscX509CrlBuilder(QscX509Certificate issuer, DateTimeOffset thisUpdate, DateTimeOffset nextUpdate)

Purpose: Create an issuer-bound native CRL with ordered UTC update times.

Parameters: issuer: Live CA certificate with CRL signing authorization.; thisUpdate: Effective CRL publication time.; nextUpdate: Expiration of published revocation information.

QscX509CrlBuilder.SetNumber [method]

public void SetNumber(ReadOnlySpan<byte> number)

Purpose: Set a positive and unique CRL number.

Parameters: number: Positive big-endian CRLNumber extension value.

QscX509CrlBuilder.AddRevoked [method]

public void AddRevoked(ReadOnlySpan<byte> serial, DateTimeOffset revokedAt)

Purpose: Append a revoked certificate serial and UTC revocation time.

Parameters: serial: Positive big-endian certificate serial.; revokedAt: UTC revocation instant.

QscX509CrlBuilder.SignDer [method]

public byte[] SignDer(QscP256DerSigner signer, int capacity = 16384)

Purpose: DER-sign a CRL with the issuer's authorized private-key callback.

Parameters: signer: Externally owned P-256 issuer signer.; capacity: Max DER bytes, limited to 1 MiB.

Returns: Caller-owned DER CRL with native signature.

QscX509CrlBuilder.VerifyDer [method]

public static QscStatus VerifyDer(ReadOnlySpan<byte> der, QscX509Certificate issuer, DateTimeOffset at)

Purpose: Verify externally supplied signed DER CRL against an issuer and UTC time.

Parameters: der: Untrusted DER CRL bytes.; issuer: Approved issuer CA certificate.; at: Verification UTC instant.

Returns: Native status; success does not by itself establish issuer chain trust.

QscX509CrlBuilder.Dispose [method]

public void Dispose()

Purpose: Release the native CRL and its issuer SafeHandle reference together.

25.47 X509/QscX509TrustStore.cs (10 members)

Declaration source: QSCCS/X509/QscX509TrustStore.cs. A throwing operation and a status-returning Try variant are distinct contracts; private native handles remain implementation details.

QscX509TrustStore.Count [property]

public nuint Count

Purpose: Gets the number of trust anchors loaded into the store.

QscX509TrustStore.Create [method]

public static QscX509TrustStore Create(nuint anchorCapacity)

Purpose: Creates an empty trust store with the requested anchor capacity.

Parameters: anchorCapacity: Maximum number of trust anchors the native store should be prepared to hold.

Returns: A managed trust-store object that owns the native handle.

Lifetime: Observe native owner validity and deterministic disposal; no use after close.

QscX509TrustStore.TryCreate [method]

public static unsafe QscStatus TryCreate(nuint anchorCapacity, out QscX509TrustStore? store)

Purpose: Attempts to create an empty trust store with the requested anchor capacity.

Parameters: anchorCapacity: Maximum number of trust anchors the native store should be prepared to hold.; store: On success, receives a managed trust-store object that owns the native handle. On failure, receives .

Returns: QscStatus.Success on success; otherwise, a QSCABI allocation or argument status.

Lifetime: Observe native owner validity and deterministic disposal; no use after close.

QscX509TrustStore.AddAnchorMemory [method]

public void AddAnchorMemory(ReadOnlySpan<byte> data, bool selfSigned)

Purpose: Adds a trust anchor from a caller-supplied memory buffer.

Parameters: data: The encoded trust-anchor certificate bytes. The buffer may contain PEM, DER, or another certificate memory encoding accepted by QSCABI.; selfSigned: when the anchor is expected to be self-signed; otherwise, .

QscX509TrustStore.AddAnchorPem [method]

public void AddAnchorPem(string pem, bool selfSigned)

Purpose: Adds a trust anchor from PEM text.

Parameters: pem: The PEM-armored trust-anchor certificate text.; selfSigned: when the anchor is expected to be self-signed; otherwise, .

QscX509TrustStore.TryAddAnchorMemory [method]

public unsafe QscStatus TryAddAnchorMemory(ReadOnlySpan<byte> data, bool selfSigned)

Purpose: Attempts to add a trust anchor from a caller-supplied memory buffer.

Parameters: data: The encoded trust-anchor certificate bytes. The buffer is read during the call and is not retained.; selfSigned: when the anchor is expected to be self-signed; otherwise, .

Returns: QscStatus.Success on success; otherwise, a QSCABI parse, validation, capacity, or allocation status.

QscX509TrustStore.VerifyPeerCertificates [method]

public bool VerifyPeerCertificates(QscX509Chain chain, QscX509ProfilePreset preset, string? hostname = null)

Purpose: Verifies a peer certificate chain against this trust store.

Parameters: chain: The peer certificate chain to verify.; preset: The native verification profile to apply.; hostname: Optional hostname to validate for profiles that require peer identity checks.

Returns: if the chain verifies successfully; otherwise, .

QscX509TrustStore.TryVerifyPeerCertificates [method]

public unsafe QscStatus TryVerifyPeerCertificates(QscX509Chain chain, QscX509ProfilePreset preset, string? hostname = null)

Purpose: Attempts to verify a peer certificate chain against this trust store.

Parameters: chain: The peer certificate chain to verify.; preset: The native verification profile to apply.; hostname: Optional hostname to validate for profiles that require peer identity checks.

Returns: QscStatus.Success when verification succeeds; otherwise, a QSCABI status describing the verification failure.

QscX509TrustStore.AddCrlMemory [method]

public unsafe void AddCrlMemory(ReadOnlySpan<byte> der)

Purpose: Releases the native trust-store handle owned by this instance.

Parameters: der: Nonempty borrowed signed CRL DER.

Lifetime: Observe native owner validity and deterministic disposal; no use after close.

QscX509TrustStore.Dispose [method]

public void Dispose()

Purpose: Source-declared public method; consult implementation for its precise contract.

25.48 Xof/QscShake.cs (6 members)

Declaration source: QSCCS/Xof/QscShake.cs. A throwing operation and a status-returning Try variant are distinct contracts; private native handles remain implementation details.

QscShake.Compute128 [method]

public static byte[] Compute128(ReadOnlySpan<byte> input, int outputLength)

Purpose: Computes SHAKE-128 output over a caller-owned input buffer.

Parameters: input: The input bytes to absorb. The buffer is read during the call and is not retained.; outputLength: The number of output bytes to produce. The value must be greater than zero.

Returns: A new byte array containing exactly bytes.

QscShake.Compute256 [method]

public static byte[] Compute256(ReadOnlySpan<byte> input, int outputLength)

Purpose: Computes SHAKE-256 output over a caller-owned input buffer.

Parameters: input: The input bytes to absorb. The buffer is read during the call and is not retained.; outputLength: The number of output bytes to produce. The value must be greater than zero.

Returns: A new byte array containing exactly bytes.

QscShake.Compute512 [method]

public static byte[] Compute512(ReadOnlySpan<byte> input, int outputLength)

Purpose: Computes SHAKE-512 output over a caller-owned input buffer.

Parameters: input: The input bytes to absorb. The buffer is read during the call and is not retained.; outputLength: The number of output bytes to produce. The value must be greater than zero.

Returns: A new byte array containing exactly bytes.

QscShake.TryCompute128 [method]

public static unsafe QscStatus TryCompute128(ReadOnlySpan<byte> input, Span<byte> output)

Purpose: Attempts to compute SHAKE-128 output into a caller-owned buffer.

Parameters: input: The input bytes to absorb. The buffer is read during the call and is not retained.; output: The caller-owned output buffer. Its length determines the XOF output length.

Returns: QscStatus.Success on success, QscStatus.InvalidSize for empty output, or another QSCABI status.

QscShake.TryCompute256 [method]

public static unsafe QscStatus TryCompute256(ReadOnlySpan<byte> input, Span<byte> output)

Purpose: Attempts to compute SHAKE-256 output into a caller-owned buffer.

Parameters: input: The input bytes to absorb. The buffer is read during the call and is not retained.; output: The caller-owned output buffer. Its length determines the XOF output length.

Returns: QscStatus.Success on success, QscStatus.InvalidSize for empty output, or another QSCABI status.

QscShake.TryCompute512 [method]

public static unsafe QscStatus TryCompute512(ReadOnlySpan<byte> input, Span<byte> output)

Purpose: Attempts to compute SHAKE-512 output into a caller-owned buffer.

Parameters: input: The input bytes to absorb. The buffer is read during the call and is not retained.; output: The caller-owned output buffer. Its length determines the XOF output length.

Returns: QscStatus.Success on success, QscStatus.InvalidSize for empty output, or another QSCABI status.

26: QSCABI Native LibraryImport Reference

The source declares exactly 329 distinct [LibraryImport] functions across Interop/*.cs. Each row gives the native C exported symbol and its managed method signature. These are implementation-level imports, generally not public C# consumer APIs. Complete named export presence is only an import inventory check; the native QSCABI binary must still satisfy exact parameter widths, struct layout, calling conventions and functional tests.

26.1 Interop/QscAbi010ExplicitNative.cs (17 imports)

Native entry point	Managed P/Invoke declaration
qscabi_curve_agree	QscStatus CurveAgree(int algorithm, byte* secret, nuint secret_length, byte* peer_public_key, nuint peer_length, byte* private_key, nuint private_length)
qscabi_curve_generate	QscStatus CurveGenerate(int algorithm, byte* public_key, nuint public_length, byte* private_key, nuint private_length)
qscabi_curve_generate_seeded	QscStatus CurveGenerateSeeded(int algorithm, byte* public_key, nuint public_length, byte* private_key, nuint private_length, byte* seed, nuint seed_length)
qscabi_curve_get_sizes	QscStatus CurveGetSizes(int algorithm, QscCurveSizes* sizes)
qscabi_curve_public_from_private	QscStatus CurvePublicFromPrivate(int algorithm, byte* public_key, nuint public_length, byte* private_key, nuint private_length)
qscabi_kem_decapsulate	QscStatus KemDecapsulate(int algorithm, byte* secret, nuint secret_length, byte* ciphertext, nuint ciphertext_length, byte* private_key, nuint private_length)
qscabi_kem_encapsulate	QscStatus KemEncapsulate(int algorithm, byte* ciphertext, nuint ciphertext_length, byte* secret, nuint secret_length, byte* public_key, nuint public_length)
qscabi_kem_generate	QscStatus KemGenerate(int algorithm, byte* public_key, nuint public_length, byte* private_key, nuint private_length)
qscabi_kem_get_sizes	QscStatus KemGetSizes(int algorithm, QscKemSizes* sizes)
qscabi_signature_generate	QscStatus SignatureGenerate(int algorithm, byte* public_key, nuint public_length, byte* private_key, nuint private_length)
qscabi_signature_get_sizes	QscStatus SignatureGetSizes(int algorithm, QscSignatureSizes* sizes)
qscabi_signature_sign	QscStatus SignatureSign(int algorithm, byte* signed_message, nuint capacity, nuint* written, byte* message, nuint message_length, byte* private_key, nuint private_length)
qscabi_signature_verify	QscStatus SignatureVerify(int algorithm, byte* message, nuint capacity, nuint* written, byte* signed_message, nuint signed_length, byte* public_key, nuint public_length)
qscabi_slhdsa_explicit_generate	QscStatus SlhdsaExplicitGenerate(int algorithm, byte* public_key, nuint public_length, byte* private_key, nuint private_length)
qscabi_slhdsa_explicit_sign	QscStatus SlhdsaExplicitSign(int algorithm, int prehash, byte randomized, byte* signature, nuint signature_length, byte* message, nuint message_length, byte* context, nuint context_length, byte* private_key, nuint private_length)

Native entry point	Managed P/Invoke declaration
qscabi_slhdsa_explicit_verify	QscStatus SlhdsaExplicitVerify(int algorithm, int prehash, byte* signature, nuint signature_length, byte* message, nuint message_length, byte* context, nuint context_length, byte* public_key, nuint public_length)
qscabi_slhdsa_get_sizes	QscStatus SlhdsaGetSizes(int algorithm, QscSlhDsaSizes* sizes)

Table 26.1 Native declarations in Interop/QscAbi010ExplicitNative.cs

26.2 Interop/QscAbi010HardwareNative.cs (14 imports)

Native entry point	Managed P/Invoke declaration
qscabi_hw_is_enabled	int HwIsEnabled()
qscabi_hw_key_agree	QscStatus HwKeyAgree(nint key, byte* peer, nuint peer_length, byte* secret, nuint* secret_length)
qscabi_hw_key_attest	QscStatus HwKeyAttest(nint key, byte* challenge, nuint challenge_length, byte* evidence, nuint* evidence_length)
qscabi_hw_key_generate	QscStatus HwKeyGenerate(nint* key, nint provider, QscHardwarePolicy* policy)
qscabi_hw_key_get_info	QscStatus HwKeyGetInfo(nint key, QscHardwareKeyInfo* info)
qscabi_hw_key_open	QscStatus HwKeyOpen(nint* key, nint provider, byte* locator, QscHardwarePolicy* policy)
qscabi_hw_key_public	QscStatus HwKeyPublic(nint key, byte* output, nuint* output_length)
qscabi_hw_key_release	QscStatus HwKeyRelease(nint* key)
qscabi_hw_key_sign	QscStatus HwKeySign(nint key, byte* message, nuint message_length, byte* signature, nuint* signature_length)
qscabi_hw_provider_capabilities	QscStatus HwProviderCapabilities(nint provider, QscHardwareCapabilities* capabilities)
qscabi_hw_provider_close	QscStatus HwProviderClose(nint* provider)
qscabi_hw_provider_invalidate	QscStatus HwProviderInvalidate(nint provider)
qscabi_hw_provider_open	QscStatus HwProviderOpen(nint* provider, QscHardwareBackendOps* operations, nint context)
qscabi_hw_provider_random	QscStatus HwProviderRandom(nint provider, byte* output, nuint length)

Table 26.2 Native declarations in Interop/QscAbi010HardwareNative.cs

26.3 Interop/QscAbi010MetadataNative.cs (2 imports)

Native entry point	Managed P/Invoke declaration
qscabi_build_backend	byte* BuildBackend()
qscabi_qsc_build_profile	byte* QscBuildProfile()

Table 26.3 Native declarations in Interop/QscAbi010MetadataNative.cs

26.4 Interop/QscAbi010StreamingNative.cs (18 imports)

Native entry point	Managed P/Invoke declaration
qscabi_cshake_compute	QscStatus CshakeCompute(int algorithm, byte* key, nuint key_length, byte* name, nuint name_length, byte* custom, nuint custom_length, byte* output, nuint output_length)

Native entry point	Managed P/Invoke declaration
qscabi_ctr_create	QscStatus CtrCreate(int algorithm, byte* key, nuint key_length, byte* nonce, nuint nonce_length, nint* state)
qscabi_ctr_destroy	void CtrDestroy(nint state)
qscabi_ctr_transform	QscStatus CtrTransform(nint state, byte* input, nuint length, byte* output, nuint output_length)
qscabi_digest_create	QscStatus DigestCreate(int algorithm, nint* state)
qscabi_digest_destroy	void DigestDestroy(nint state)
qscabi_digest_finalize	QscStatus DigestFinalize(nint state, byte* digest, nuint digest_length)
qscabi_digest_size	nuint DigestSize(int algorithm)
qscabi_digest_update	QscStatus DigestUpdate(nint state, byte* data, nuint length)
qscabi_stream_mac_create	QscStatus StreamMacCreate(int algorithm, byte* key, nuint key_length, byte* custom, nuint custom_length, nint* state)
qscabi_stream_mac_destroy	void StreamMacDestroy(nint state)
qscabi_stream_mac_finalize	QscStatus StreamMacFinalize(nint state, byte* output, nuint length)
qscabi_stream_mac_update	QscStatus StreamMacUpdate(nint state, byte* data, nuint length)
qscabi_xof_absorb	QscStatus XofAbsorb(nint state, byte* data, nuint length)
qscabi_xof_create	QscStatus XofCreate(int algorithm, nint* state)
qscabi_xof_destroy	void XofDestroy(nint state)
qscabi_xof_finalize	QscStatus XofFinalize(nint state)
qscabi_xof_squeeze	QscStatus XofSqueeze(nint state, byte* output, nuint length)

Table 26.4 Native declarations in Interop/QscAbi010StreamingNative.cs

26.5 Interop/QscAbi010TlsNative.cs (9 imports)

Native entry point	Managed P/Invoke declaration
qscabi_tls_context_add_server_identity_files	QscStatus TlsContextAddServerIdentityFiles(nint context, byte* hostname, byte* chain_path, byte* key_path, int scheme)
qscabi_tls_context_load_client_identity_files	QscStatus TlsContextLoadClientIdentityFiles(nint context, byte* chain_path, byte* key_path, int scheme)
qscabi_tls_context_load_crl_file	QscStatus TlsContextLoadCrlFile(nint context, byte* path)
qscabi_tls_context_load_server_identity_files	QscStatus TlsContextLoadServerIdentityFiles(nint context, byte* chain_path, byte* key_path, int scheme)
qscabi_tls_context_load_trust_anchor_bundle_file	QscStatus TlsContextLoadTrustAnchorBundleFile(nint context, byte* path, int self_signed)
qscabi_tls_context_load_trust_anchor_file	QscStatus TlsContextLoadTrustAnchorFile(nint context, byte* path, int self_signed)
qscabi_tls_context_set_cipher_suites	QscStatus TlsContextSetCipherSuites(nint context, ushort* suites, nuint count)
qscabi_tls_context_set_named_groups	QscStatus TlsContextSetNamedGroups(nint context, ushort* groups, nuint count)
qscabi_tls_context_set_signature_ids	QscStatus TlsContextSetSignatureIds(nint context, ushort* schemes, nuint count)

Table 26.5 Native declarations in Interop/QscAbi010TlsNative.cs

26.6 Interop/QscAbi010X509Native.cs (15 imports)

Native entry point	Managed P/Invoke declaration
qscabi_x509_crl_add_revoked	QscStatus X509CrlAddRevoked(nint builder, byte* serial, nuint serial_length, QscX509Time* revoked_at)
qscabi_x509_crl_builder_create	QscStatus X509CrlBuilderCreate(nint* builder, nint issuer, QscX509Time* this_update, QscX509Time* next_update)
qscabi_x509_crl_builder_destroy	void X509CrlBuilderDestroy(nint builder)
qscabi_x509_crl_set_number	QscStatus X509CrlSetNumber(nint builder, byte* value, nuint length)
qscabi_x509_crl_sign_der	QscStatus X509CrlSignDer(nint builder, nint issuer, nint signer, nint context, byte* output, nuint* output_length)
qscabi_x509_crl_verify_der	QscStatus X509CrlVerifyDer(byte* der, nuint length, nint issuer, QscX509Time* verification_time)
qscabi_x509_csr_add_dns	QscStatus X509CsrAddDns(nint csr, byte* dns)
qscabi_x509_csr_create	QscStatus X509CsrCreate(nint* csr)
qscabi_x509_csr_destroy	void X509CsrDestroy(nint csr)
qscabi_x509_csr_init_p256	QscStatus X509CsrInitP256(nint csr, byte* common_name, byte* public_key, nuint public_key_length)
qscabi_x509_csr_load	QscStatus X509CsrLoad(nint csr, byte* data, nuint length)
qscabi_x509_csr_sign_der	QscStatus X509CsrSignDer(nint csr, nint signer, nint context, byte* output, nuint* output_length)
qscabi_x509_csr_verify	QscStatus X509CsrVerify(nint csr)
qscabi_x509_issue_tls_server_p256	QscStatus X509IssueTlsServerP256(nint csr, nint issuer, byte* serial, nuint serial_length, QscX509Time* not_before, QscX509Time* not_after, nint signer, nint context, byte* output, nuint* output_length)
qscabi_x509_store_add_crl_memory	QscStatus X509StoreAddCrlMemory(nint store, byte* der, nuint length)

Table 26.6 Native declarations in Interop/QscAbi010X509Native.cs

26.7 Interop/QscAbiAsymmetricNative.cs (36 imports)

Native entry point	Managed P/Invoke declaration
qscabi_ecdh_public_key_size	nuint EcdhPublicKeySize()
qscabi_ecdh_private_key_size	nuint EcdhPrivateKeySize()
qscabi_ecdh_shared_secret_size	nuint EcdhSharedSecretSize()
qscabi_ecdh_seed_size	nuint EcdhSeedSize()
qscabi_ecdh_generate_keypair	QscStatus EcdhGenerateKeypair(byte* publicKey, nuint publicKeyLength, byte* privateKey, nuint privateKeyLength)
qscabi_ecdh_generate_seeded_keypair	QscStatus EcdhGenerateSeededKeypair(byte* publicKey, nuint publicKeyLength, byte* privateKey, nuint privateKeyLength, byte* seed, nuint seedLength)
qscabi_ecdh_public_from_private	QscStatus EcdhPublicFromPrivate(byte* publicKey, nuint publicKeyLength, byte* privateKey, nuint privateKeyLength)
qscabi_ecdh_key_exchange	QscStatus EcdhKeyExchange(byte* secret, nuint secretLength, byte* privateKey, nuint privateKeyLength, byte* publicKey, nuint publicKeyLength)
qscabi_eddh_public_key_size	nuint EddhPublicKeySize()

Native entry point	Managed P/Invoke declaration
qscabi_eddh_private_key_size	nuint EddhPrivateKeySize()
qscabi_eddh_shared_secret_size	nuint EddhSharedSecretSize()
qscabi_eddh_seed_size	nuint EddhSeedSize()
qscabi_eddh_generate_keypair	QscStatus EddhGenerateKeypair(byte* publicKey, nuint publicKeyLength, byte* privateKey, nuint privateKeyLength)
qscabi_eddh_generate_seeded_keypair	QscStatus EddhGenerateSeededKeypair(byte* publicKey, nuint publicKeyLength, byte* privateKey, nuint privateKeyLength, byte* seed, nuint seedLength)
qscabi_eddh_public_from_private	QscStatus EddhPublicFromPrivate(byte* publicKey, nuint publicKeyLength, byte* privateKey, nuint privateKeyLength)
qscabi_eddh_key_exchange	QscStatus EddhKeyExchange(byte* secret, nuint secretLength, byte* privateKey, nuint privateKeyLength, byte* publicKey, nuint publicKeyLength)
qscabi_ecdsa_public_key_size	nuint EcdsaPublicKeySize()
qscabi_ecdsa_private_key_size	nuint EcdsaPrivateKeySize()
qscabi_ecdsa_signature_size	nuint EcdsaSignatureSize()
qscabi_ecdsa_seed_size	nuint EcdsaSeedSize()
qscabi_ecdsa_signed_message_size	nuint EcdsaSignedMessageSize(nuint messageLength)
qscabi_ecdsa_generate_keypair	QscStatus EcdsaGenerateKeypair(byte* publicKey, nuint publicKeyLength, byte* privateKey, nuint privateKeyLength)
qscabi_ecdsa_generate_seeded_keypair	QscStatus EcdsaGenerateSeededKeypair(byte* publicKey, nuint publicKeyLength, byte* privateKey, nuint privateKeyLength, byte* seed, nuint seedLength)
qscabi_ecdsa_public_from_private	QscStatus EcdsaPublicFromPrivate(byte* publicKey, nuint publicKeyLength, byte* privateKey, nuint privateKeyLength)
qscabi_ecdsa_sign	QscStatus EcdsaSign(byte* signedMessage, nuint signedMessageLength, nuint* written, byte* message, nuint messageLength, byte* privateKey, nuint privateKeyLength)
qscabi_ecdsa_verify	QscStatus EcdsaVerify(byte* message, nuint messageLength, nuint* written, byte* signedMessage, nuint signedMessageLength, byte* publicKey, nuint publicKeyLength)
qscabi_eddsa_public_key_size	nuint EddsaPublicKeySize()
qscabi_eddsa_private_key_size	nuint EddsaPrivateKeySize()
qscabi_eddsa_signature_size	nuint EddsaSignatureSize()
qscabi_eddsa_seed_size	nuint EddsaSeedSize()
qscabi_eddsa_signed_message_size	nuint EddsaSignedMessageSize(nuint messageLength)
qscabi_eddsa_generate_keypair	QscStatus EddsaGenerateKeypair(byte* publicKey, nuint publicKeyLength, byte* privateKey, nuint privateKeyLength)
qscabi_eddsa_generate_seeded_keypair	QscStatus EddsaGenerateSeededKeypair(byte* publicKey, nuint publicKeyLength, byte* privateKey, nuint privateKeyLength, byte* seed, nuint seedLength)
qscabi_eddsa_public_from_private	QscStatus EddsaPublicFromPrivate(byte* publicKey, nuint publicKeyLength, byte* privateKey, nuint privateKeyLength)
qscabi_eddsa_sign	QscStatus EddsaSign(byte* signedMessage, nuint signedMessageLength, nuint* written, byte* message, nuint messageLength, byte* privateKey, nuint privateKeyLength)

Native entry point	Managed P/Invoke declaration
qscabi_eddsa_verify	QscStatus EddsaVerify(byte* message, nuint messageLength, nuint* written, byte* signedMessage, nuint signedMessageLength, byte* publicKey, nuint publicKeyLength)

Table 26.7 Native declarations in Interop/QscAbiAsymmetricNative.cs

26.8 Interop/QscAbiEncodingNative.cs (12 imports)

Native entry point	Managed P/Invoke declaration
qscabi_hex_encoded_size	nuint HexEncodedSize(nuint inputLength)
qscabi_hex_decoded_size	QscStatus HexDecodedSize(nuint* outputLength, byte* input, nuint inputLength)
qscabi_hex_encode	QscStatus HexEncode(byte* output, nuint outputLength, nuint* written, byte* input, nuint inputLength)
qscabi_hex_decode	QscStatus HexDecode(byte* output, nuint outputLength, nuint* written, byte* input, nuint inputLength)
qscabi_base64_encoded_size	nuint Base64EncodedSize(nuint inputLength)
qscabi_base64_decoded_size	QscStatus Base64DecodedSize(nuint* outputLength, byte* input, nuint inputLength)
qscabi_base64_encode	QscStatus Base64Encode(byte* output, nuint outputLength, nuint* written, byte* input, nuint inputLength)
qscabi_base64_decode	QscStatus Base64Decode(byte* output, nuint outputLength, nuint* written, byte* input, nuint inputLength)
qscabi_pem_encoded_size	QscStatus PemEncodedSize(nuint* outputLength, byte* label, nuint dataLength)
qscabi_pem_decoded_size	QscStatus PemDecodedSize(nuint* outputLength, byte* input, nuint inputLength)
qscabi_pem_encode	QscStatus PemEncode(byte* output, nuint outputLength, nuint* written, byte* label, byte* data, nuint dataLength)
qscabi_pem_decode	QscStatus PemDecode(byte* output, nuint outputLength, nuint* written, byte* input, nuint inputLength)

Table 26.8 Native declarations in Interop/QscAbiEncodingNative.cs

26.9 Interop/QscAbiEntropyNative.cs (10 imports)

Native entry point	Managed P/Invoke declaration
qscabi_entropy_acp_seed_max	nuint EntropyAcpSeedMax()
qscabi_entropy_csp_seed_max	nuint EntropyCspSeedMax()
qscabi_entropy_rdp_seed_max	nuint EntropyRdpSeedMax()
qscabi_entropy_cjp_seed_max	nuint EntropyCjpSeedMax()
qscabi_entropy_rdp_available	int EntropyRdpAvailable()
qscabi_entropy_cjp_available	int EntropyCjpAvailable()
qscabi_entropy_acp_generate	QscStatus EntropyAcpGenerate(byte* output, nuint outputLength)
qscabi_entropy_csp_generate	QscStatus EntropyCspGenerate(byte* output, nuint outputLength)
qscabi_entropy_rdp_generate	QscStatus EntropyRdpGenerate(byte* output, nuint outputLength)
qscabi_entropy_cjp_generate	QscStatus EntropyCjpGenerate(byte* output, nuint outputLength)

Table 26.9 Native declarations in Interop/QscAbiEntropyNative.cs

26.10 Interop/QscAbiHashNative.cs (12 imports)

Native entry point	Managed P/Invoke declaration
qscabi_sha2_256_digest_size	nuint Sha2256DigestSize()
qscabi_sha2_384_digest_size	nuint Sha2384DigestSize()
qscabi_sha2_512_digest_size	nuint Sha2512DigestSize()
qscabi_sha3_128_digest_size	nuint Sha3128DigestSize()
qscabi_sha3_256_digest_size	nuint Sha3256DigestSize()
qscabi_sha3_512_digest_size	nuint Sha3512DigestSize()
qscabi_sha2_256_compute	QscStatus Sha2256Compute(byte* output, nuint outputLength, byte* message, nuint messageLength)
qscabi_sha2_384_compute	QscStatus Sha2384Compute(byte* output, nuint outputLength, byte* message, nuint messageLength)
qscabi_sha2_512_compute	QscStatus Sha2512Compute(byte* output, nuint outputLength, byte* message, nuint messageLength)
qscabi_sha3_128_compute	QscStatus Sha3128Compute(byte* output, nuint outputLength, byte* message, nuint messageLength)
qscabi_sha3_256_compute	QscStatus Sha3256Compute(byte* output, nuint outputLength, byte* message, nuint messageLength)
qscabi_sha3_512_compute	QscStatus Sha3512Compute(byte* output, nuint outputLength, byte* message, nuint messageLength)

Table 26.10 Native declarations in Interop/QscAbiHashNative.cs

26.11 Interop/QscAbiKdfNative.cs (12 imports)

Native entry point	Managed P/Invoke declaration
qscabi_hkdf_sha2_256_prk_size	nuint HkdfSha2256PrkSize()
qscabi_hkdf_sha2_384_prk_size	nuint HkdfSha2384PrkSize()
qscabi_hkdf_sha2_512_prk_size	nuint HkdfSha2512PrkSize()
qscabi_pbmac1_max_key_size	nuint Pbmac1MaxKeySize()
qscabi_pbmac1_max_iterations	ulong Pbmac1MaxIterations()
qscabi_hkdf_sha2_256_extract	QscStatus HkdfSha2256Extract(byte* output, nuint outputLength, byte* key, nuint keyLength, byte* salt, nuint saltLength)
qscabi_hkdf_sha2_256_expand	QscStatus HkdfSha2256Expand(byte* output, nuint outputLength, byte* key, nuint keyLength, byte* info, nuint infoLength)
qscabi_hkdf_sha2_384_extract	QscStatus HkdfSha2384Extract(byte* output, nuint outputLength, byte* key, nuint keyLength, byte* salt, nuint saltLength)
qscabi_hkdf_sha2_384_expand	QscStatus HkdfSha2384Expand(byte* output, nuint outputLength, byte* key, nuint keyLength, byte* info, nuint infoLength)
qscabi_hkdf_sha2_512_extract	QscStatus HkdfSha2512Extract(byte* output, nuint outputLength, byte* key, nuint keyLength, byte* salt, nuint saltLength)
qscabi_hkdf_sha2_512_expand	QscStatus HkdfSha2512Expand(byte* output, nuint outputLength, byte* key, nuint keyLength, byte* info, nuint infoLength)
qscabi_pbmac1_derive_key	QscStatus Pbmac1DeriveKey(byte* output, nuint outputLength, QscPbmac1Hash hash, byte* password, nuint passwordLength, byte* salt, nuint saltLength, ulong iterations, nuint keyLength)

Table 26.11 Native declarations in Interop/QscAbiKdfNative.cs

26.12 Interop/QscAbiMacNative.cs (23 imports)

Native entry point	Managed P/Invoke declaration
qscabi_hmac_sha2_256_mac_size	nuint HmacSha2256MacSize()
qscabi_hmac_sha2_384_mac_size	nuint HmacSha2384MacSize()
qscabi_hmac_sha2_512_mac_size	nuint HmacSha2512MacSize()
qscabi_poly1305_mac_size	nuint Poly1305MacSize()
qscabi_poly1305_key_size	nuint Poly1305KeySize()
qscabi_qmac_mac_size	nuint QmacMacSize()
qscabi_qmac_key_size	nuint QmacKeySize()
qscabi_qmac_nonce_size	nuint QmacNonceSize()
qscabi_pbmac1_mac_size	nuint Pbmac1MacSize(QscPbmac1Hash hash)
qscabi_hmac_sha2_256_compute	QscStatus HmacSha2256Compute(byte* output, nuint outputLength, byte* message, nuint messageLength, byte* key, nuint keyLength)
qscabi_hmac_sha2_384_compute	QscStatus HmacSha2384Compute(byte* output, nuint outputLength, byte* message, nuint messageLength, byte* key, nuint keyLength)
qscabi_hmac_sha2_512_compute	QscStatus HmacSha2512Compute(byte* output, nuint outputLength, byte* message, nuint messageLength, byte* key, nuint keyLength)
qscabi_kmac_128_compute	QscStatus Kmac128Compute(byte* output, nuint outputLength, byte* message, nuint messageLength, byte* key, nuint keyLength, byte* custom, nuint customLength)
qscabi_kmac_256_compute	QscStatus Kmac256Compute(byte* output, nuint outputLength, byte* message, nuint messageLength, byte* key, nuint keyLength, byte* custom, nuint customLength)
qscabi_kmac_512_compute	QscStatus Kmac512Compute(byte* output, nuint outputLength, byte* message, nuint messageLength, byte* key, nuint keyLength, byte* custom, nuint customLength)
qscabi_kmac_xof_128_compute	QscStatus KmacXof128Compute(byte* output, nuint outputLength, byte* message, nuint messageLength, byte* key, nuint keyLength, byte* custom, nuint customLength)
qscabi_kmac_xof_256_compute	QscStatus KmacXof256Compute(byte* output, nuint outputLength, byte* message, nuint messageLength, byte* key, nuint keyLength, byte* custom, nuint customLength)
qscabi_kmac_xof_512_compute	QscStatus KmacXof512Compute(byte* output, nuint outputLength, byte* message, nuint messageLength, byte* key, nuint keyLength, byte* custom, nuint customLength)
qscabi_poly1305_compute	QscStatus Poly1305Compute(byte* output, nuint outputLength, byte* message, nuint messageLength, byte* key, nuint keyLength)
qscabi_poly1305_verify	QscStatus Poly1305Verify(byte* code, nuint codeLength, byte* message, nuint messageLength, byte* key, nuint keyLength)
qscabi_qmac_compute	QscStatus QmacCompute(byte* output, nuint outputLength, byte* message, nuint messageLength, byte* key, nuint keyLength, byte* nonce, nuint nonceLength, byte* info, nuint infoLength)
qscabi_pbmac1_compute	QscStatus Pbmac1Compute(byte* output, nuint outputLength, QscPbmac1Hash hash, byte* password, nuint passwordLength, byte* salt, nuint saltLength, ulong iterations, nuint keyLength, byte* message, nuint messageLength)

Native entry point	Managed P/Invoke declaration
qscabi_pbmac1_verify	QscStatus Pbmac1Verify(byte* code, nuint codeLength, QscPbmac1Hash hash, byte* password, nuint passwordLength, byte* salt, nuint saltLength, ulong iterations, nuint keyLength, byte* message, nuint messageLength)

Table 26.12 Native declarations in Interop/QscAbiMacNative.cs

26.13 Interop/QscAbiNative.cs (12 imports)

Native entry point	Managed P/Invoke declaration
qscabi_version_major	uint VersionMajor()
qscabi_version_minor	uint VersionMinor()
qscabi_version_patch	uint VersionPatch()
qscabi_abi_is_compatible	uint AbiIsCompatible(uint major, uint minimumMinor)
qscabi_capability_flags	ulong CapabilityFlags()
qscabi_version_string	nint VersionString()
qscabi_qsc_version_string	nint QscVersionString()
qscabi_build_platform	nint BuildPlatform()
qscabi_build_compiler	nint BuildCompiler()
qscabi_enabled_features	nint EnabledFeatures()
qscabi_memory_clear	void MemoryClear(nint data, nuint length)
qscabi_memory_are_equal	int MemoryAreEqual(nint left, nint right, nuint length)

Table 26.13 Native declarations in Interop/QscAbiNative.cs

26.14 Interop/QscAbiPqcNative.cs (42 imports)

Native entry point	Managed P/Invoke declaration
qscabi_mlkem_public_key_size	nuint MlKemPublicKeySize()
qscabi_mlkem_private_key_size	nuint MlKemPrivateKeySize()
qscabi_mlkem_ciphertext_size	nuint MlKemCiphertextSize()
qscabi_mlkem_shared_secret_size	nuint MlKemSharedSecretSize()
qscabi_mlkem_generate_keypair	QscStatus MlKemGenerateKeypair(byte* publicKey, nuint publicKeyLength, byte* privateKey, nuint privateKeyLength)
qscabi_mlkem_encapsulate	QscStatus MlKemEncapsulate(byte* ciphertext, nuint ciphertextLength, byte* secret, nuint secretLength, byte* publicKey, nuint publicKeyLength)
qscabi_mlkem_decapsulate	QscStatus MlKemDecapsulate(byte* secret, nuint secretLength, byte* ciphertext, nuint ciphertextLength, byte* privateKey, nuint privateKeyLength)
qscabi_hqc_public_key_size	nuint HqcPublicKeySize()
qscabi_hqc_private_key_size	nuint HqcPrivateKeySize()
qscabi_hqc_ciphertext_size	nuint HqcCiphertextSize()
qscabi_hqc_shared_secret_size	nuint HqcSharedSecretSize()
qscabi_hqc_generate_keypair	QscStatus HqcGenerateKeypair(byte* publicKey, nuint publicKeyLength, byte* privateKey, nuint privateKeyLength)
qscabi_hqc_encapsulate	QscStatus HqcEncapsulate(byte* ciphertext, nuint ciphertextLength, byte* secret, nuint secretLength, byte* publicKey, nuint publicKeyLength)

Native entry point	Managed P/Invoke declaration
qscabi_hqc_decapsulate	QscStatus HqcDecapsulate(byte* secret, nuint secretLength, byte* ciphertext, nuint ciphertextLength, byte* privateKey, nuint privateKeyLength)
qscabi_mceliece_public_key_size	nuint McEliecePublicKeySize()
qscabi_mceliece_private_key_size	nuint McEliecePrivateKeySize()
qscabi_mceliece_ciphertext_size	nuint McElieceCiphertextSize()
qscabi_mceliece_shared_secret_size	nuint McElieceSharedSecretSize()
qscabi_mceliece_generate_keypair	QscStatus McElieceGenerateKeypair(byte* publicKey, nuint publicKeyLength, byte* privateKey, nuint privateKeyLength)
qscabi_mceliece_encapsulate	QscStatus McElieceEncapsulate(byte* ciphertext, nuint ciphertextLength, byte* secret, nuint secretLength, byte* publicKey, nuint publicKeyLength)
qscabi_mceliece_decapsulate	QscStatus McElieceDecapsulate(byte* secret, nuint secretLength, byte* ciphertext, nuint ciphertextLength, byte* privateKey, nuint privateKeyLength)
qscabi_mldsa_public_key_size	nuint MlDsaPublicKeySize()
qscabi_mldsa_private_key_size	nuint MlDsaPrivateKeySize()
qscabi_mldsa_signature_size	nuint MlDsaSignatureSize()
qscabi_mldsa_signed_message_size	nuint MlDsaSignedMessageSize(nuint messageLength)
qscabi_mldsa_generate_keypair	QscStatus MlDsaGenerateKeypair(byte* publicKey, nuint publicKeyLength, byte* privateKey, nuint privateKeyLength)
qscabi_mldsa_sign	QscStatus MlDsaSign(byte* signedMessage, nuint signedMessageLength, nuint* written, byte* message, nuint messageLength, byte* privateKey, nuint privateKeyLength)
qscabi_mldsa_verify	QscStatus MlDsaVerify(byte* message, nuint messageLength, nuint* written, byte* signedMessage, nuint signedMessageLength, byte* publicKey, nuint publicKeyLength)
qscabi_falcon_public_key_size	nuint FalconPublicKeySize()
qscabi_falcon_private_key_size	nuint FalconPrivateKeySize()
qscabi_falcon_signature_size	nuint FalconSignatureSize()
qscabi_falcon_signed_message_size	nuint FalconSignedMessageSize(nuint messageLength)
qscabi_falcon_generate_keypair	QscStatus FalconGenerateKeypair(byte* publicKey, nuint publicKeyLength, byte* privateKey, nuint privateKeyLength)
qscabi_falcon_sign	QscStatus FalconSign(byte* signedMessage, nuint signedMessageLength, nuint* written, byte* message, nuint messageLength, byte* privateKey, nuint privateKeyLength)
qscabi_falcon_verify	QscStatus FalconVerify(byte* message, nuint messageLength, nuint* written, byte* signedMessage, nuint signedMessageLength, byte* publicKey, nuint publicKeyLength)
qscabi_slhdsa_public_key_size	nuint SlhDsaPublicKeySize()
qscabi_slhdsa_private_key_size	nuint SlhDsaPrivateKeySize()
qscabi_slhdsa_signature_size	nuint SlhDsaSignatureSize()
qscabi_slhdsa_signed_message_size	nuint SlhDsaSignedMessageSize(nuint messageLength)
qscabi_slhdsa_generate_keypair	QscStatus SlhDsaGenerateKeypair(byte* publicKey, nuint publicKeyLength, byte* privateKey, nuint privateKeyLength)

Native entry point	Managed P/Invoke declaration
qscabi_slhdsa_sign	QscStatus SlhDsaSign(byte* signedMessage, nuint signedMessageLength, nuint* written, byte* message, nuint messageLength, byte* privateKey, nuint privateKeyLength)
qscabi_slhdsa_verify	QscStatus SlhDsaVerify(byte* message, nuint messageLength, nuint* written, byte* signedMessage, nuint signedMessageLength, byte* publicKey, nuint publicKeyLength)

Table 26.14 Native declarations in Interop/QscAbiPqcNative.cs

26.15 Interop/QscAbiStatusNative.cs (1 imports)

Native entry point	Managed P/Invoke declaration
qscabi_status_string	nint StatusString(QscStatus status)

Table 26.15 Native declarations in Interop/QscAbiStatusNative.cs

26.16 Interop/QscAbiSymmetricNative.cs (27 imports)

Native entry point	Managed P/Invoke declaration
qscabi_aes_block_size	nuint AesBlockSize()
qscabi_aes128_key_size	nuint Aes128KeySize()
qscabi_aes256_key_size	nuint Aes256KeySize()
qscabi_aes_gcm_nonce_size	nuint AesGcmNonceSize()
qscabi_aes_gcm_max_nonce_size	nuint AesGcmMaxNonceSize()
qscabi_aes_gcm_tag_size	nuint AesGcmTagSize()
qscabi_chacha20_key_size	nuint ChaCha20KeySize()
qscabi_chacha20_nonce_size	nuint ChaCha20NonceSize()
qscabi_chacha20_poly1305_tag_size	nuint ChaCha20Poly1305TagSize()
qscabi_csx_key_size	nuint CsxKeySize()
qscabi_csx_nonce_size	nuint CsxNonceSize()
qscabi_csx_tag_size	nuint CsxTagSize()
qscabi_rcs_key_size	nuint RcsKeySize(QscRcsMode mode)
qscabi_rcs_nonce_size	nuint RcsNonceSize()
qscabi_rcs_tag_size	nuint RcsTagSize(QscRcsMode mode)
qscabi_aes256_ctr_transform	QscStatus Aes256CtrTransform(byte* output, nuint outputLength, byte* input, nuint inputLength, byte* key, nuint keyLength, byte* nonce, nuint nonceLength, QscAesCtrCounter counterMode)
qscabi_aes128_gcm_encrypt	QscStatus Aes128GcmEncrypt(byte* output, nuint outputLength, nuint* written, byte* plaintext, nuint plaintextLength, byte* key, nuint keyLength, byte* nonce, nuint nonceLength, byte* aad, nuint aadLength)
qscabi_aes128_gcm_decrypt	QscStatus Aes128GcmDecrypt(byte* output, nuint outputLength, nuint* written, byte* ciphertext, nuint ciphertextLength, byte* key, nuint keyLength, byte* nonce, nuint nonceLength, byte* aad, nuint aadLength)
qscabi_aes256_gcm_encrypt	QscStatus Aes256GcmEncrypt(byte* output, nuint outputLength, nuint* written, byte* plaintext, nuint plaintextLength, byte* key, nuint keyLength, byte* nonce, nuint nonceLength, byte* aad, nuint aadLength)

Native entry point	Managed P/Invoke declaration
qscabi_aes256_gcm_decrypt	QscStatus Aes256GcmDecrypt(byte* output, nuint outputLength, nuint* written, byte* ciphertext, nuint ciphertextLength, byte* key, nuint keyLength, byte* nonce, nuint nonceLength, byte* aad, nuint aadLength)
qscabi_chacha20_transform	QscStatus ChaCha20Transform(byte* output, nuint outputLength, byte* input, nuint inputLength, byte* key, nuint keyLength, byte* nonce, nuint nonceLength)
qscabi_chacha20_poly1305_encrypt	QscStatus ChaCha20Poly1305Encrypt(byte* output, nuint outputLength, nuint* written, byte* plaintext, nuint plaintextLength, byte* key, nuint keyLength, byte* nonce, nuint nonceLength, byte* aad, nuint aadLength)
qscabi_chacha20_poly1305_decrypt	QscStatus ChaCha20Poly1305Decrypt(byte* output, nuint outputLength, nuint* written, byte* ciphertext, nuint ciphertextLength, byte* key, nuint keyLength, byte* nonce, nuint nonceLength, byte* aad, nuint aadLength)
qscabi_csx_encrypt	QscStatus CsxEncrypt(byte* output, nuint outputLength, nuint* written, byte* plaintext, nuint plaintextLength, byte* key, nuint keyLength, byte* nonce, nuint nonceLength, byte* info, nuint infoLength, byte* aad, nuint aadLength)
qscabi_csx_decrypt	QscStatus CsxDecrypt(byte* output, nuint outputLength, nuint* written, byte* ciphertext, nuint ciphertextLength, byte* key, nuint keyLength, byte* nonce, nuint nonceLength, byte* info, nuint infoLength, byte* aad, nuint aadLength)
qscabi_rcs_encrypt	QscStatus RcsEncrypt(byte* output, nuint outputLength, nuint* written, QscRcsMode mode, byte* plaintext, nuint plaintextLength, byte* key, nuint keyLength, byte* nonce, nuint nonceLength, byte* info, nuint infoLength, byte* aad, nuint aadLength)
qscabi_rcs_decrypt	QscStatus RcsDecrypt(byte* output, nuint outputLength, nuint* written, QscRcsMode mode, byte* ciphertext, nuint ciphertextLength, byte* key, nuint keyLength, byte* nonce, nuint nonceLength, byte* info, nuint infoLength, byte* aad, nuint aadLength)

Table 26.16 Native declarations in Interop/QscAbiSymmetricNative.cs

26.17 Interop/QscAbiTlsNative.cs (40 imports)

Native entry point	Managed P/Invoke declaration
qscabi_tls_status_string	nint TlsStatusString(QscStatus status)
qscabi_tls_socket_options_default	void TlsSocketOptionsDefault(QscAbiTlsSocketOptionsNative* options)
qscabi_tls_ticket_policy_default	void TlsTicketPolicyDefault(QscAbiTlsTicketPolicyNative* policy)
qscabi_tls_context_create	QscStatus TlsContextCreate(nint* context)
qscabi_tls_context_destroy	void TlsContextDestroy(nint context)
qscabi_tls_context_set_policy	QscStatus TlsContextSetPolicy(QscTlsContextHandle context, int policy)
qscabi_tls_context_set_socket_options	QscStatus TlsContextSetSocketOptions(QscTlsContextHandle context, QscAbiTlsSocketOptionsNative* options)
qscabi_tls_context_set_alpn_protocols	QscStatus TlsContextSetAlpnProtocols(QscTlsContextHandle context, nint* protocols, nuint protocolCount, int required)

Native entry point	Managed P/Invoke declaration
qscabi_tls_context_clear_alpn_protocols	QscStatus TlsContextClearAlpnProtocols(QscTlsContextHandle context)
qscabi_tls_context_set_sni_required	QscStatus TlsContextSetSniRequired(QscTlsContextHandle context, int required)
qscabi_tls_context_set_client_auth	QscStatus TlsContextSetClientAuth(QscTlsContextHandle context, int requestClientAuth, int requireClientAuth)
qscabi_tls_context_set_ticket_policy	QscStatus TlsContextSetTicketPolicy(QscTlsContextHandle context, QscAbiTlsTicketPolicyNative* policy)
qscabi_tls_context_load_trust_anchor_pem	QscStatus TlsContextLoadTrustAnchorPem(QscTlsContextHandle context, byte* pem, nuint pemLength, int selfSigned)
qscabi_tls_context_load_server_identity_pem	QscStatus TlsContextLoadServerIdentityPem(QscTlsContextHandle context, byte* chainPem, nuint chainPemLength, byte* privateKeyPem, nuint privateKeyPemLength, int signatureScheme)
qscabi_tls_connection_create	QscStatus TlsConnectionCreate(nint* connection)
qscabi_tls_connection_destroy	void TlsConnectionDestroy(nint connection)
qscabi_tls_client_connect_host	QscStatus TlsClientConnectHost(QscTlsConnectionHandle connection, QscTlsContextHandle context, byte* hostname, byte* service)
qscabi_tls_connection_send	QscStatus TlsConnectionSend(QscTlsConnectionHandle connection, byte* input, nuint inputLength, nuint* written)
qscabi_tls_connection_receive	QscStatus TlsConnectionReceive(QscTlsConnectionHandle connection, byte* output, nuint outputLength, nuint* readLength)
qscabi_tls_connection_send_frame	QscStatus TlsConnectionSendFrame(QscTlsConnectionHandle connection, byte* input, nuint inputLength)
qscabi_tls_connection_receive_frame	QscStatus TlsConnectionReceiveFrame(QscTlsConnectionHandle connection, byte* output, nuint outputLength, nuint* readLength)
qscabi_tls_connection_shutdown	QscStatus TlsConnectionShutdown(QscTlsConnectionHandle connection)
qscabi_tls_connection_cancel	QscStatus TlsConnectionCancel(QscTlsConnectionHandle connection)
qscabi_tls_connection_is_connected	int TlsConnectionIsConnected(QscTlsConnectionHandle connection)
qscabi_tls_connection_is_handshake_complete	int TlsConnectionIsHandshakeComplete(QscTlsConnectionHandle connection)
qscabi_tls_connection_get_selected_alpn	QscStatus TlsConnectionGetSelectedAlpn(QscTlsConnectionHandle connection, byte* output, nuint outputLength, nuint* written)
qscabi_tls_connection_get_peer_info	QscStatus TlsConnectionGetPeerInfo(QscTlsConnectionHandle connection, QscAbiTlsPeerInfoNative* peerInfo)
qscabi_tls_listener_create	QscStatus TlsListenerCreate(nint* listener)
qscabi_tls_listener_destroy	void TlsListenerDestroy(nint listener)
qscabi_tls_listener_set_socket_options	QscStatus TlsListenerSetSocketOptions(QscTlsListenerHandle listener, QscAbiTlsSocketOptionsNative* options)
qscabi_tls_listener_bind	QscStatus TlsListenerBind(QscTlsListenerHandle listener, QscTlsContextHandle context, byte* address, ushort port, int family)
qscabi_tls_listener_accept	QscStatus TlsListenerAccept(QscTlsListenerHandle listener, QscTlsConnectionHandle connection)
qscabi_tls_listener_close	void TlsListenerClose(QscTlsListenerHandle listener)
qscabi_tls_server_create	QscStatus TlsServerCreate(nint* server)

Native entry point	Managed P/Invoke declaration
qscabi_tls_server_destroy	void TlsServerDestroy(nint server)
qscabi_tls_server_configure	QscStatus TlsServerConfigure(QscTlsServerHandle server, QscTlsContextHandle context, byte* address, ushort port, int family)
qscabi_tls_server_set_max_clients	QscStatus TlsServerSetMaxClients(QscTlsServerHandle server, nuint maxClients)
qscabi_tls_server_start	QscStatus TlsServerStart(QscTlsServerHandle server)
qscabi_tls_server_start_concurrent	QscStatus TlsServerStartConcurrent(QscTlsServerHandle server)
qscabi_tls_server_stop	void TlsServerStop(QscTlsServerHandle server)

Table 26.17 Native declarations in Interop/QscAbiTlsNative.cs

26.18 Interop/QscAbiX509Native.cs (24 imports)

Native entry point	Managed P/Invoke declaration
qscabi_x509_certificate_create	QscStatus X509CertificateCreate(nint* certificate)
qscabi_x509_certificate_destroy	void X509CertificateDestroy(nint certificate)
qscabi_x509_certificate_load_memory	QscStatus X509CertificateLoadMemory(QscX509CertificateHandle certificate, byte* data, nuint dataLength)
qscabi_x509_certificate_load_pem	QscStatus X509CertificateLoadPem(QscX509CertificateHandle certificate, byte* pem, nuint pemLength)
qscabi_x509_certificate_load_der	QscStatus X509CertificateLoadDer(QscX509CertificateHandle certificate, byte* der, nuint derLength)
qscabi_x509_certificate_export_der	QscStatus X509CertificateExportDer(QscX509CertificateHandle certificate, byte* output, nuint outputLength, nuint* written)
qscabi_x509_certificate_export_pem	QscStatus X509CertificateExportPem(QscX509CertificateHandle certificate, byte* output, nuint outputLength, nuint* written)
qscabi_x509_certificate_subject_string	QscStatus X509CertificateSubjectString(QscX509CertificateHandle certificate, byte* output, nuint outputLength, nuint* written)
qscabi_x509_certificate_issuer_string	QscStatus X509CertificateIssuerString(QscX509CertificateHandle certificate, byte* output, nuint outputLength, nuint* written)
qscabi_x509_certificate_subject_common_name	QscStatus X509CertificateSubjectCommonName(QscX509CertificateHandle certificate, byte* output, nuint outputLength, nuint* written)
qscabi_x509_certificate_subject_dns_name_count	nuint X509CertificateSubjectDnsNameCount(QscX509CertificateHandle certificate)
qscabi_x509_certificate_subject_dns_name	QscStatus X509CertificateSubjectDnsName(QscX509CertificateHandle certificate, nuint index, byte* output, nuint outputLength, nuint* written)
qscabi_x509_certificate_match_hostname	QscStatus X509CertificateMatchHostname(QscX509CertificateHandle certificate, byte* hostname)
qscabi_x509_certificate_check_role	QscStatus X509CertificateCheckRole(QscX509CertificateHandle certificate, int role, byte* hostname)
qscabi_x509_chain_create	QscStatus X509ChainCreate(nint* chain, nuint certificateCapacity)
qscabi_x509_chain_destroy	void X509ChainDestroy(nint chain)
qscabi_x509_chain_load_memory	QscStatus X509ChainLoadMemory(QscX509ChainHandle chain, byte* data, nuint dataLength)

Native entry point	Managed P/Invoke declaration
qscabi_x509_chain_count	nuint X509ChainCount(QscX509ChainHandle chain)
qscabi_x509_store_create	QscStatus X509StoreCreate(nint* store, nuint anchorCapacity)
qscabi_x509_store_destroy	void X509StoreDestroy(nint store)
qscabi_x509_store_add_anchor_memory	QscStatus X509StoreAddAnchorMemory(QscX509StoreHandle store, byte* data, nuint dataLength, int selfSigned)
qscabi_x509_store_count	nuint X509StoreCount(QscX509StoreHandle store)
qscabi_x509_verify_peer_certificates	QscStatus X509VerifyPeerCertificates(QscX509ChainHandle chain, QscX509StoreHandle store, int preset, byte* hostname)
qscabi_x509_status_string	nint X509StatusString(QscStatus status)

Table 26.18 Native declarations in Interop/QscAbiX509Native.cs

26.19 Interop/QscAbiXofNative.cs (3 imports)

Native entry point	Managed P/Invoke declaration
qscabi_shake_128_compute	QscStatus Shake128Compute(byte* output, nuint outputLength, byte* input, nuint inputLength)
qscabi_shake_256_compute	QscStatus Shake256Compute(byte* output, nuint outputLength, byte* input, nuint inputLength)
qscabi_shake_512_compute	QscStatus Shake512Compute(byte* output, nuint outputLength, byte* input, nuint inputLength)

Table 26.19 Native declarations in Interop/QscAbiXofNative.cs

Special audit note: qscabi_status_string is declared by private static partial StatusString in QscAbiStatusNative.cs and uses QscAbiNative.LibraryName. The 329-symbol parity fails if an automated inventory filters only imports whose first argument is literally `LibraryName`. The actual source has 329 imports, not 328.

27: Implementor Release Acceptance Checklist

- Restore the full authorized QSCCS solution, managed examples, build scripts, QRCS-CASL distribution files and external QSC/QSCABI dependencies.
- Pin .NET SDK version, platform/architecture, QSCABI native revision, compiled QSC CPU ISA/AES feature profile and native hash.
- Audit all 329 LibraryImport symbols and verify exact prototypes against QSCABI 0.10 headers, including qscabi_status_string.
- Build Debug and Release on supported platform targets; log actual compiler commands and errors.
- Validate Try* status/failure semantics, short buffer checks, managed key zeroization and native SafeHandle release.
- Exercise KEM, signatures, AEAD tamper checks, explicit parameter selection and streaming lifecycle negative cases.
- Qualify X.509 chains, hostname, CSR/CRL issuance and role checks with signed trusted fixtures.
- Run live network TLS client/server/mTLS interoperability separately from lifecycle tests.
- For hardware, run physical device sign/agree/attestation and provider disconnection tests on installed backends.
- Run the full xUnit suite and 17 console sections, retaining individual PASS/FAIL/SKIP diagnostics and `--no-pause` logs.
- Retain authorized QRCS-CASL legal source notice and verify each packaged artifact includes the controlling license terms.
- Do not claim skipped/unexecuted tests passed or equate C# import parity with cryptographic certification.

This specification records the uploaded source state. No managed compilation, native cryptographic execution, or independent third-party certification was performed for this paper.

Annex A: Public Managed Types and Enums

Public type	Declaration source / type form
QscEcdh	Asymmetric/QscEcdh.cs — public static class QscEcdh
QscEcdsa	Asymmetric/QscEcdsa.cs — public static class QscEcdsa
QscEddh	Asymmetric/QscEddh.cs — public static class QscEddh
QscEddsa	Asymmetric/QscEddsa.cs — public static class QscEddsa
QscAbiCapabilities	Common/QscAbiCapabilities.cs — public enum QscAbiCapabilities : ulong
QscException	Common/QscException.cs — public sealed class QscException : Exception
QscMemory	Common/QscMemory.cs — public static class QscMemory
QscResult	Common/QscResult.cs — public readonly struct QscResult
QscStatus	Common/QscStatus.cs — public enum QscStatus
QscVersion	Common/QscVersion.cs — public static class QscVersion
QscEncoding	Encoding/QscEncoding.cs — public static class QscEncoding
QscEntropy	Entropy/QscEntropy.cs — public static class QscEntropy
QscExplicitKeyPair	Explicit/QscExplicitOperations.cs — public sealed class QscExplicitKeyPair : IDisposable
QscExplicitKemOutput	Explicit/QscExplicitOperations.cs — public sealed class QscExplicitKemOutput : IDisposable
QscExplicitKem	Explicit/QscExplicitOperations.cs — public static class QscExplicitKem
QscExplicitCurve	Explicit/QscExplicitOperations.cs — public static class QscExplicitCurve
QscExplicitSignature	Explicit/QscExplicitSignatures.cs — public static class QscExplicitSignature
QscExplicitSlhDsa	Explicit/QscExplicitSignatures.cs — public static class QscExplicitSlhDsa
QscKemAlgorithm	Explicit/QscExplicitTypes.cs — public enum QscKemAlgorithm : int
QscCurveAlgorithm	Explicit/QscExplicitTypes.cs — public enum QscCurveAlgorithm : int
QscAttachedSignatureAlgorithm	Explicit/QscExplicitTypes.cs — public enum QscAttachedSignatureAlgorithm : int
QscExplicitSlhDsaAlgorithm	Explicit/QscExplicitTypes.cs — public enum QscExplicitSlhDsaAlgorithm : int
QscSlhDsaPrehash	Explicit/QscExplicitTypes.cs — public enum QscSlhDsaPrehash : int
QscDigestAlgorithm	Explicit/QscExplicitTypes.cs — public enum QscDigestAlgorithm : int
QscStreamingXofAlgorithm	Explicit/QscExplicitTypes.cs — public enum QscStreamingXofAlgorithm : int
QscStreamingMacAlgorithm	Explicit/QscExplicitTypes.cs — public enum QscStreamingMacAlgorithm : int
QscStreamingCtrAlgorithm	Explicit/QscExplicitTypes.cs — public enum QscStreamingCtrAlgorithm : int
QscHardwareResult	Hardware/QscHardware.cs — public enum QscHardwareResult : int
QscHardwareCallbackSet	Hardware/QscHardware.cs — public unsafe sealed class QscHardwareCallbackSet
QscHardwareProvider	Hardware/QscHardware.cs — public sealed unsafe class QscHardwareProvider : IDisposable
QscHardwareKey	Hardware/QscHardware.cs — public sealed unsafe class QscHardwareKey : IDisposable
QscSha2	Hash/QscSha2.cs — public static class QscSha2
QscSha3	Hash/QscSha3.cs — public static class QscSha3
QscHkdf	Kdf/QscHkdf.cs — public static class QscHkdf
QscPbmac1Kdf	Kdf/QscPbmac1Kdf.cs — public static class QscPbmac1Kdf
QscHmac	Mac/QscHmac.cs — public static class QscHmac
QscKmac	Mac/QscKmac.cs — public static class QscKmac
QscPbmac1	Mac/QscPbmac1.cs — public static class QscPbmac1

Public type	Declaration source / type form
QscPbmac1Hash	Mac/QscPbmac1Hash.cs — public enum QscPbmac1Hash
QscPoly1305	Mac/QscPoly1305.cs — public static class QscPoly1305
QscQmac	Mac/QscQmac.cs — public static class QscQmac
QscFalcon	Pqc/QscFalcon.cs — public static class QscFalcon
QscHqc	Pqc/QscHqc.cs — public static class QscHqc
QscMcEliece	Pqc/QscMcEliece.cs — public static class QscMcEliece
QscMlDsa	Pqc/QscMlDsa.cs — public static class QscMlDsa
QscMlKem	Pqc/QscMlKem.cs — public static class QscMlKem
QscPqcKeyPair	Pqc/QscPqcTypes.cs — public sealed record QscPqcKeyPair(byte[] PublicKey, byte[] PrivateKey) : IDisposable
QscKemEncapsulation	Pqc/QscPqcTypes.cs — public sealed record QscKemEncapsulation(byte[] Ciphertext, byte[] SharedSecret) : IDisposable
QscSlhDsa	Pqc/QscSlhDsa.cs — public static class QscSlhDsa
QscIncrementalDigest	Streaming/QscStreaming.cs — public sealed class QscIncrementalDigest : IDisposable
QscIncrementalXof	Streaming/QscStreaming.cs — public sealed class QscIncrementalXof : IDisposable
QscIncrementalMac	Streaming/QscStreaming.cs — public sealed class QscIncrementalMac : IDisposable
QscCtrStream	Streaming/QscStreaming.cs — public sealed class QscCtrStream : IDisposable
QscCustomizedShake	Streaming/QscStreaming.cs — public static class QscCustomizedShake
QscAes	Symmetric/QscAes.cs — public static class QscAes
QscAesCtrCounter	Symmetric/QscAesCtrCounter.cs — public enum QscAesCtrCounter
QscChaCha	Symmetric/QscChaCha.cs — public static class QscChaCha
QscChaChaPoly1305	Symmetric/QscChaChaPoly1305.cs — public static class QscChaChaPoly1305
QscCsx	Symmetric/QscCsx.cs — public static class QscCsx
QscRcs	Symmetric/QscRcs.cs — public static class QscRcs
QscRcsMode	Symmetric/QscRcsMode.cs — public enum QscRcsMode
QscSymmetricConstants	Symmetric/QscSymmetricConstants.cs — public static class QscSymmetricConstants
QscTlsAddressFamily	Tls/QscTlsAddressFamily.cs — public enum QscTlsAddressFamily
QscTlsConnection	Tls/QscTlsConnection.cs — public sealed class QscTlsConnection : IDisposable
QscTlsContext	Tls/QscTlsContext.cs — public sealed class QscTlsContext : IDisposable
QscTlsListener	Tls/QscTlsListener.cs — public sealed class QscTlsListener : IDisposable
QscTlsOptions	Tls/QscTlsOptions.cs — public sealed class QscTlsOptions
QscTlsPeerInfo	Tls/QscTlsPeerInfo.cs — public sealed class QscTlsPeerInfo
QscTlsPolicy	Tls/QscTlsPolicy.cs — public enum QscTlsPolicy
QscTlsServer	Tls/QscTlsServer.cs — public sealed class QscTlsServer : IDisposable
QscTlsSignatureScheme	Tls/QscTlsSignatureScheme.cs — public enum QscTlsSignatureScheme
QscTlsTicketPolicy	Tls/QscTlsTicketPolicy.cs — public sealed class QscTlsTicketPolicy
QscX509Certificate	X509/QscX509Certificate.cs — public unsafe sealed class QscX509Certificate : IDisposable
QscX509CertificateRole	X509/QscX509CertificateRole.cs — public enum QscX509CertificateRole
QscX509Chain	X509/QscX509Chain.cs — public sealed class QscX509Chain : IDisposable
QscP256DerSigner	X509/QscX509Issuance.cs — public sealed unsafe class QscP256DerSigner
QscX509Csr	X509/QscX509Issuance.cs — public sealed unsafe class QscX509Csr : IDisposable

Public type	Declaration source / type form
QscX509CrlBuilder	X509/QscX509Issuance.cs — public sealed unsafe class QscX509CrlBuilder : IDisposable
QscX509ProfilePreset	X509/QscX509ProfilePreset.cs — public enum QscX509ProfilePreset
QscX509TrustStore	X509/QscX509TrustStore.cs — public sealed class QscX509TrustStore : IDisposable
QscShake	Xof/QscShake.cs — public static class QscShake

Table A.1 Public managed type declaration inventory

Some public enums encode algorithm identifiers or native protocol values; their numeric discriminants are part of interoperability and must not be inferred from their ordinal position. Some managed classes expose properties representing borrowed native metadata; a property query is not itself proof of trust or validated peer identity.

Annex B: Test Case and Source Inventory

QSCCSTest/Program.cs invokes 112 named case delegates through 17 direct test section calls. Each test is listed below in the order used by the console. The exact output status on a real host requires running those calls against a native QSCABI binary; only the source invocation names were inspected.

Test class/group	Explicit console cases
Abi010ExplicitTests	8
Abi010PolicyIssuanceHardwareTests	5
Abi010StreamingTests	5
AsymmetricTests	7
ConsoleSafeInvalidUsageTests	2
CoreTests	5
EncodingTests	8
EntropyTests	6
HashTests	9
KdfTests	5
MacTests	11
PqcTests	9
StatusTests	6
SymmetricTests	11
TlsTests	6
X509Tests	5
XofTests	4

Table B.1 Console case counts by test class

B.2 Sequential named case invocations 1–30

Order	Name in console runner
1	Abi010ExplicitTests.All329NativeImportsPresent
2	Abi010ExplicitTests.Native010BuildMetadataAvailable
3	Abi010ExplicitTests.P256CurveSizesMatchNative
4	Abi010ExplicitTests.P256CurveAgreementRoundtrip
5	Abi010ExplicitTests.X25519CurveAgreementRoundtrip
6	Abi010ExplicitTests.MlKem512ExplicitRoundtrip
7	Abi010ExplicitTests.Ed25519AttachedSignatureRoundtrip
8	Abi010ExplicitTests.SlhDsaShake128sDetachedRoundtrip
9	Abi010StreamingTests.IncrementalSha256MatchesDotNet
10	Abi010StreamingTests.IncrementalShake256ProducesContinuousOutput
11	Abi010StreamingTests.IncrementalHmacSha256MatchesDotNet
12	Abi010StreamingTests.Aes256CtrRoundtrip
13	Abi010StreamingTests.CustomizedShakeIsDeterministic
14	Abi010PolicyIssuanceHardwareTests.TlsContextAcceptsNative010Preferences
15	Abi010PolicyIssuanceHardwareTests.UnsignedP256CsrCannotVerify
16	Abi010PolicyIssuanceHardwareTests.NativeHardwareAndUtcLayoutsMatch

Order	Name in console runner
17	Abi010PolicyIssuanceHardwareTests.HardwareFacadeCapabilityIsOnlyBuildFeature
18	Abi010PolicyIssuanceHardwareTests.ExplicitEnumSelectorsMatchNativeHeader
19	CoreTests.VersionMetadata_ShouldReturnNonEmptyValues
20	CoreTests.NativeAbi_ShouldRejectUnsupportedMajorVersion
21	CoreTests.NativeCapabilities_ShouldContainExplicitStreamingX509AndTls
22	CoreTests.MemoryAreEqual_ShouldReportEqualAndDifferentBuffers
23	CoreTests.MemoryClear_ShouldZeroCallerOwnedBuffer
24	StatusTests.QscException_ShouldPreserveStatus[0]
25	StatusTests.QscException_ShouldPreserveStatus[1]
26	StatusTests.QscException_ShouldPreserveStatus[2]
27	StatusTests.QscException_ShouldPreserveStatus[3]
28	StatusTests.QscException_ShouldPreserveStatus[4]
29	StatusTests.QscResult_ShouldReflectSuccessState
30	EncodingTests.HexEncode_ShouldMatchKnownValue

Table B.2 Explicit console cases

B.3 Sequential named case invocations 31–60

Order	Name in console runner
31	EncodingTests.HexRoundTrip_ShouldRecoverOriginalBytes
32	EncodingTests.HexDecode_ShouldRejectOddLengthInput
33	EncodingTests.Base64Encode_ShouldMatchKnownValue
34	EncodingTests.Base64RoundTrip_ShouldRecoverOriginalBytes
35	EncodingTests.Base64Decode_ShouldRejectInvalidLengthInput
36	EncodingTests.PemRoundTrip_ShouldRecoverOriginalBytes
37	EncodingTests.PemDecode_ShouldRejectNonPemInput
38	HashTests.Sha2DigestSizes_ShouldMatchExpectedSizes
39	HashTests.Sha2_256_ShouldMatchKnownAnswerForAbc
40	HashTests.Sha2_384_ShouldMatchKnownAnswerForAbc
41	HashTests.Sha2_512_ShouldMatchKnownAnswerForAbc
42	HashTests.Sha2TryCompute_ShouldRejectSmallOutputBuffer
43	HashTests.Sha3DigestSizes_ShouldMatchExpectedSizes
44	HashTests.Sha3_256_ShouldMatchKnownAnswerForEmptyMessage
45	HashTests.Sha3_512_ShouldMatchKnownAnswerForEmptyMessage
46	HashTests.Sha3_128_ShouldBeDeterministicAndUseExpectedSize
47	XofTests.Shake128_ShouldMatchKnownAnswerForEmptyMessage32Bytes
48	XofTests.Shake256_ShouldBeDeterministicForSameInputAndLength
49	XofTests.Shake512_ShouldSupportVariableOutputLengths
50	XofTests.ShakeTryCompute_ShouldRejectEmptyOutputBuffer
51	MacTests.HmacSizes_ShouldMatchExpectedSizes
52	MacTests.HmacSha2256_ShouldMatchRfc4231Vector
53	MacTests.HmacSha2512_ShouldMatchRfc4231Vector
54	MacTests.HmacTryCompute_ShouldRejectSmallOutputBuffer
55	MacTests.Kmac_ShouldBeDeterministic
56	MacTests.KmacXof_ShouldRespectOutputLength

Order	Name in console runner
57	MacTests.Poly1305Sizes_ShouldMatchExpectedSizes
58	MacTests.Poly1305_ShouldMatchRfc8439Vector
59	MacTests.Poly1305Verify_ShouldRejectTamperedCode
60	MacTests.Qmac_ShouldBeDeterministicAndUseExpectedSizes

Table B.3 Explicit console cases

B.4 Sequential named case invocations 61–90

Order	Name in console runner
61	MacTests.Pbmac1_ShouldVerifyAndRejectTamperedCode
62	KdfTests.HkdfPrkSizes_ShouldMatchExpectedSizes
63	KdfTests.HkdfSha2256_ShouldMatchRfc5869Vector
64	KdfTests.HkdfExpand_ShouldRejectEmptyOutputBuffer
65	KdfTests.Pbmac1Kdf_ShouldDeriveDeterministicKey
66	KdfTests.Pbmac1Kdf_ShouldRejectOversizedOutput
67	SymmetricTests.SymmetricSizes_ShouldMatchExpectedSizes
68	SymmetricTests.Aes256Ctr_ShouldMatchNistVector
69	SymmetricTests.Aes256Ctr_ShouldRoundTrip
70	SymmetricTests.AesGcm256_ShouldRoundTripAndRejectTamper
71	SymmetricTests.AesGcm128_ShouldRoundTrip
72	SymmetricTests.ChaCha20_ShouldRoundTrip
73	SymmetricTests.ChaCha20Poly1305_ShouldRoundTripAndRejectTamper
74	SymmetricTests.Csx_ShouldRoundTripAndRejectTamper
75	SymmetricTests.Rcs256_ShouldRoundTripAndRejectTamper
76	SymmetricTests.Rcs512_ShouldRoundTripAndRejectTamper
77	SymmetricTests.TryMethods_ShouldRejectSmallOutputBuffers
78	EntropyTests.EntropySeedMaximums_ShouldBePositive
79	EntropyTests.AcpGenerate_ShouldFillRequestedLength
80	EntropyTests.CspGenerate_ShouldFillRequestedLength
81	EntropyTests.TryGenerate_ShouldRejectEmptyOutput
82	EntropyTests.RdpGenerate_ShouldRespectAvailability
83	EntropyTests.CjpGenerate_ShouldRespectAvailability
84	AsymmetricTests.Ecdh_ShouldComputeMatchingSecrets
85	AsymmetricTests.Eddh_ShouldComputeMatchingSecrets
86	AsymmetricTests.Ecdsa_ShouldSignAndVerify
87	AsymmetricTests.Eddsa_ShouldSignAndVerify
88	AsymmetricTests.EcdhSeededKeypair_ShouldBeDeterministic
89	AsymmetricTests.EddsaPublicFromPrivate_ShouldRecoverPublicKey
90	AsymmetricTests.EddsaPublicFromPrivate_ShouldRejectInvalidSizes

Table B.4 Explicit console cases

B.5 Sequential named case invocations 91–112

Order	Name in console runner
91	PqcTests.PqcKeyPairDispose_ShouldZeroPrivateKeyAndPreservePublicKey
92	PqcTests.KemEncapsulationDispose_ShouldZeroSharedSecretAndPreserveCiphertext

Order	Name in console runner
93	PqcTests.MlKem_ShouldRoundTripSharedSecret
94	PqcTests.Hqc_ShouldRoundTripSharedSecret
95	PqcTests.McEliece_ShouldRoundTripSharedSecret
96	PqcTests.MlDsa_ShouldSignAndVerify
97	PqcTests.SlhDsa_ShouldSignAndVerify
98	PqcTests.Falcon_ShouldExposeSizes
99	PqcTests.MlKem_ShouldRejectSmallBuffers
100	X509Tests.LoadPemReadsIdentityFields
101	X509Tests.HostnamePolicyAcceptsAndRejectsExpectedNames
102	X509Tests.ExportDerAndPemRoundTrip
103	X509Tests.ChainAndTrustStoreVerifyExpectedHost
104	X509Tests.InvalidPemIsRejected
105	TlsTests.LegacyInMemoryTlsCredentialLoading_ShouldReturnUnsupported
106	TlsTests.ContextPolicyAndOptionsShouldSucceed
107	TlsTests.ContextAlpnSniAndClientAuthenticationShouldSucceed
108	TlsTests.ConnectionStartsDisconnectedAndHandshakeIncomplete
109	TlsTests.ListenerAndServerLifecycleShouldSucceedWithoutNetworkStart
110	TlsTests.InvalidAlpnInputShouldReturnInvalidArgument
111	ConsoleSafeInvalidUsageTests.EntropyTryGenerate_ShouldRejectEmptyOutputWithoutThrowing
112	ConsoleSafeInvalidUsageTests.ShakeTryCompute_ShouldRejectEmptyOutputWithoutThrowing

Table B.5 Explicit console cases

Source tests include xUnit Fact/Theory cases that remain accessible to Test Explorer independently of the console direct-runner. ConsoleSkipAttribute is a debugger-visible presentation policy and should never be reported as an executed cryptographic PASS.

Annex C: Source Findings and Packaging Notes

Finding	Implementation consequence
No QSCCS.sln included	Solution-root build commands require the full repository; per-project builds are the available source path.
No examples/ code included	CLI commands described by docs/EXAMPLE_COMMANDS.md cannot be independently verified against this ZIP.
No QSCABI DLL/SO included	Native runtime, even for the managed tests, is an external prerequisite.
No scripts/ directory included	The documented scripts/audit_native_imports.py is absent from this archive; do not claim it was run.
329 distinct imports present	Status-string import is private in QscAbiStatusNative.cs; narrow pattern returns a false 328.
17 console sections, 112 direct cases	Earlier testing guide's 14/100 description is historical/inconsistent with supplied current Program.cs.
No full commercial license agreement in ZIP	The source contains authorized QRCS-CASL notices; the agreement must be supplied in distribution.
No executed test logs	Source inspection, managed signature inventory and DOCX render checks do not establish runtime qualification.

Table C.1 Project-specific source findings

For current native ABI signatures, the independently supplied QSCABI header archive can be used as an additional reference; the binary actually installed by the customer is the ultimate runtime target and must be checked for export names and supported ABI versions. Any QSC/QSCABI dependency patch is out of scope for this specification.